PUBLIC

Document Version: 1.0.0 – 2026-04-13

Joule Development Guide

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

Content

1 Joule Development. 3

2 Pro-Code Development Tools for Joule. 4

2.1 Joule Studio CLI. 4

Install and Update the Joule Studio CLI. 4

Joule Studio CLI Commands. 6

Troubleshooting Guide. 15

2.2 Joule Studio Code Editor. 18

Download and Install the Joule Studio Code Editor. 18

Joule Studio Code Editor Features. 20

Walkthrough. 28

Handling Linter Rules. 32

3 Building Capabilities for Joule 35

3.1 Design Time Artifact (DTA) Specification. 35

3.2 Resource Limits for Capability Files. 40

3.3 Build a Capability. 41

Defining Scenarios. 42

Implementing Functions. 57

Agent Integration. 189

4 Compile. 208

4.1 Compile the Design-Time Artifact. 208

5 Deploy. 209

5.1 Deploy the Runtime Artifact. 209

1 Joule Development

This development guide provides a detailed step-by-step description on how to develop an assistant using the Joule Studio CLI and the Joule Studio code editor pro-code development tools. It also describes how to compile an assistant and deploy it in your tenant in SAP Business Technology Platform (BTP).

> **Note**
>
> To build and deploy capabilities for Joule, you must be assigned the role of an extensibility developer. For more information, see Assign Roles.

> **Note**
>
> Always ensure you are working with the latest versions of the tools.
>
> For information about installing and updating the CLI tool, see Install and Update the Joule Studio CLI [page 4]. For information about downloading and installing the code editor, see Download and Install the Joule Studio Code Editor [page 18].

2 Pro-Code Development Tools for Joule

The pro-code development tools for Joule provide a complete environment for building, validating, and managing assistants. They consist of the Joule Studio CLI tool and the Joule Studio code editor.

> **Note**
>
> You must have created a service instance of Joule. For more information, see Initial Setup.

> **Note**
>
> To build and deploy capabilities for Joule, you must be assigned the role of an extensibility developer. For more information, see Assign Roles.

2.1 Joule Studio CLI

Joule Studio CLI is a command line interface (CLI) tool that enables developers to work directly with Joule capabilities from the terminal, integrating seamlessly into local development and CI/CD pipelines. It supports secure authentication, project compilation and deployment to target runtimes.

2.1.1 Install and Update the Joule Studio CLI

Prerequisites

- You have installed Node.js (minimum version 20.12.0 and maximum 24). You can download a version from https://nodejs.org/en/download/releases/ .
- You have a terminal provided by your operating system. (Windows PowerShell or Cmd app, MacOS Terminal app)
- If you are using Linux, you have installed libsecret, gnome-keyring or another keyring desktop application compatible with libsecret.

Installation

Open the terminal and run the following commands to -

- Get the latest delivered official version

Sample Code

Install joule

```
npm install -g @sap/joule-studio-cli
```

- Get a specific delivered official version

Sample Code

Install joule

```
npm install -g @sap/joule-studio-cli@<version>
```

Replace `<version>` with the version number, for example, `<1.0.90>`.

- Check version
  - Default registry used for automatic version checks:

    Sample Code

    ```
    joule -V
    ```

  - If JOULE_NPMJS_URL is set, its value is used instead of the default for all registry requests that joule performs for version metadata.
    - Unix / macOS (bash, zsh)

      Sample Code

      ```
      JOULE_NPMJS_URL=<registry_url> joule -V
      ```

    - Windows PowerShell

      Sample Code

      ```
      $env:JOULE_NPMJS_URL="<registry_url>"; joule -V
      ```

Update the CLI

With every subsequent update of the CLI version, you must update your existing version.

Open a terminal and run the following command:

Sample Code

Update joule

```
npm install -g @sap/joule-studio-cli --force
```

Uninstall

Open a terminal and run the following command to uninstall the CLI.

{·} Sample Code

Uninstall joule

```
npm uninstall -g @sap/joule-studio-cli
```

ⓘ Note

Make sure that you run Joule logout before executing the uninstall command, if you want to delete all the private and public properties stored locally. Uninstalling the *@sap/joule-studio-cli* package will not clear them.

2.1.2 Joule Studio CLI Commands

The following section describes the available command usages. Each command supports the -h option for help and the -d option to generate a debug file. More details are available in the Troubleshooting Guide [page 15].

ⓘ Note

Custom capabilities must be defined in the *joule.ext* namespace. Custom capabilities can be added to SAP Digital Assistant only if they are defined in the *joule.ext* namespace.

ⓘ Note

You must have created a service instance of Joule. For more information, see Initial Setup.

2.1.2.1 joule version

Description: Show the version of the current joule-studio-cli installation

Command: joule

Options:

Parameter	Description
--version, -V	Show the version number

{·} Sample Code

```
joule --version
```

2.1.2.2 joule help

Description: Shows all the commands and root options available with the CLI

Command: joule

Options:

Parameter	Description
--help, -h	Display help for the command

Sample Code

```
joule --help
```

2.1.2.3 joule debug

Description: Create a debug log file to trace errors

Command: joule [command]

Options:

Parameter	Description
--debug, -d	Show and store extra debug logs

Sample Code

```
joule [command] -d
```

2.1.2.4 joule monochrome

Description: Displays the output of the command in a monochrome format

Command: joule [command]

Options:

Parameter	Description
--no-color, -w	Display monochrome output

Sample Code

```
joule [command] --no-color
```

2.1.2.5 joule login

Description: Logs in to the production landscape

Command: joule login [options]

Options:

Parameter	Value	Description
--authurl, -a	<authurl>	Authentication URL
--apiurl	<apiurl>	API URL
--clientid, -c	<clientid>	Instance Client ID
--clientsescret, -s	<clientsecret>	Instance Client Secret
--username, -u	<username>	Username
--password, -p	<password>	Password
--default-idp, -i	N/A	Use the Default Identity Provider
--unsecure-storage	-e	Store the secrets in the local public store (Not recommended)
--sso-passcode	[passcode]	One-time passcode
--use-env	[env]	Use environment variables for login
--store-password	N/A	Enable password storage (default settings: password storage and log-in auto-retries are disabled)

Examples:

- `joule login`
  Login to the production landscape. Run the command, follow the prompts, and provide the required information to fulfill the login steps.
- `joule login --default-idp`
  By default, you login to the `sap.custom` IDP. Use this command to switch to the `sap.default` IDP.
- `joule login --unsecure-storage`
  Use this option when running the login command in a Docker container or Linux script as it ignores the OS credentials storage mechanism.

  > ⚠ Caution
  >
  > Credentials are normally stored in a secure store managed by the operating system. This option is not recommended as the credentials are stored in a public store.
  >
  > IDP. Use this command to switch to the Authentication information will be stored locally and used for authentication objectives onFurthermore, this software will access and transmit your local DTA capabilities when some others commands are utilized, as this is indispensable for the correct operation of this software.

- `joule login --sso-passcode`

> **Note**
>
> The *--sso-passcode* authentication option is currently available only for XSUAA authentication flow.

Use this option to login using the one-time passcode instead of your password.

- `A prompt with a URL will be displayed.`
- `Open the link, authenticate if needed, and copy the generated passcode into the prompt`

- `joule login --use-env [path]`
  `Available environment variables are the following:`
  - JOULE_API_URL=
  - JOULE_USERNAME=
  - JOULE_PASSWORD=
  - JOULE_AUTH_URL=
  - JOULE_CLIENT_ID=
  - JOULE_CLIENT_SECRET=
  - JOULE_AUTH_SESSION = {"accessToken": "<Token>", "refreshToken": "<Token>"}
  - JOULE_DEFAULT_IDP=<false or true, default false>

  Use this option to login using the environment variables specified in file located in specified path. If path is omitted then it will look for `.env` file in the current working directory. This option can be used to partially load variables like authentication url and client id and secret variables.
  You can login either using your credentials or specifying Token to be used for command line feature that requires authentication.

- `joule login –store-password: Enable storing passwords.`
  By default, passwords are not stored, and automatic authentication retries will be performed only if refresh_token are available.

By default, the CLI remembers your Authentication and API URLs. In this case to change the tenant, you have two options:

1. `joule logout` and `joule login`
2. `joule login -a --<authurl>` and follow the prompts.

2.1.2.6 joule logout

Description: Logs out of a landscape

Command: joule logout

Options: N/A

> **Sample Code**
>
> ```
> joule logout
> ```

2.1.2.7 joule status

Description: Show the login status

Command: joule status

Options: N/A

Sample Code

```
joule --status
```

2.1.2.8 joule compile

Description: Compiles the Design Time Artifact (DTA) to generate a Runtime Artifact (RTA) file and saves it locally as .daar

Command: joule compile [options] [source folder] [target folder]

Options:

Parameter	Value	Description
--hide-warnings	N/A	Hide warnings
--batch-size, -b	<number>	Set the batch size of local capabilities to be compiled at once. Default is 8.

Arguments:

Parameter	Description
source target	Source design time artifact folder to compile (default: ".")
target folder	Target folder or file name to store compiled runtime artifact (default: ".")

Examples:

- joule compile (The DTA folder should contain one or multiple valid capabilities and one valid digital assistant (DA) config (da.sapdas.yaml))
- joule compile . <my_target_folder>
- joule compile <my_source_foled> <my_target_folder> --hide-warnings
- joule compile -b 10

2.1.2.9 joule deploy

Description: Deploy the runtime artifact to your tenant

Command: joule deploy [options] [source file]

Options:

Parameter	Value	Description
--daars-input-dir, -i	<folder with compiled capabilities>	Folder with compiled capabilities
--name	<digital assistant name>	Target digital assistant name
--compile, -c	N/A	Compile the local capabilities before deploying
--batch-size, -b	<number>	Set the batch size of local capabilities to be compiled at once. Default is 8.

Arguments:

Parameter	Description
source file	Source digital assistant configuration file to compile, package, and deploy (default: ".")

Examples:

- joule deploy
- joule deploy ./my_first_digital_assistant -n <your_digital_assistant_name>
- joule deploy ./my_first_digital_assistant/da.sapdas.yaml -n <your_digital_assistant_name>
- joule deploy ./my_first_digital_assistant/someprefix.da.sapdas.yaml -n <your_digital_assistant_name>
- You can use a single command to compile, package, and deploy a multi-capability digital assistant to your tenant in SAP BTP: joule deploy --compile ./my_first_digital_assistant
- To deploy a digital assistant with the precompiled capabilities stored in a custom folder, use: joule deploy ./my_first_digital_assistant --daars-input-dir ./my_daar_list_folder

> **Note**
>
> Extensibility Developers cannot deploy assistants named after sap_digital_assistant.

2.1.2.10 joule update

Description: Adds/updates capabilities in deployed Digital Assistant

Command:joule update <digital_assistant_name> --capability-file <file_path> [options]

Options:

Parameter	Description
-h, --help	Display help for command
--capability-file	Path(s) to a capability file(s) (capability `.sapdas.yaml` or `.daar` file)

Parameter	Description
-hide-warnings	Suppress warning messages during the update process

Arguments:

Parameter	Description
<digital_assistant_name>	The name of the deployed digital assistant to update
<file_path>	Path(s) to a capability file(s) (capability `.sapdas.yaml` or `.daar` file)

2.1.2.11 joule list

Description: Gets the list of all the assistants deployed in your tenant

Command: joule list [options]

Options:

Parameter	Value	Description
--sort	<field>	Specify an optional sort field

Examples:

- joule list
- joule list --sort name

2.1.2.12 joule get

Description: Gets the details of the active or latest version of the deployed assistant

Command:joule get [options] <digital assistant name>

Options:

Parameter	Value	Description
--capability, -c	<capability name>	Display the capability details of the digital assistant

Arguments:

Parameter	Description
digital assistant name	Name of the digital assistant

Examples:

- joule get <your_digital_assistant_name>
- To display the capability details of the digital assistant, use: joule get <your_digital_assistant_name> -c <capability name>

> **Note**
>
> The get details command on Joule only returns custom capabilities (namespace *joule.ext* namespace).

2.1.2.13 joule launch

Description: Test and interact with your deployed digital assistant using the Web Client. Once the command is executed, the default web browser will open with the Web Client page connected to your digital assistant. In case of failure using the default open browser command of the Operating System, it will fall back to sensible-browser to open the link. In case of consecutive failures, it will display the link to the Web Client page.

Command: joule launch <digital assistant name>

Arguments:

Parameter	Description
digital assistant name	Name of the digital assistant

Example:

joule launch <your_digital_assistant_name>

2.1.2.14 joule delete

Description: Deletes a digital assistant

Command: joule delete <digital assistant name>

Arguments:

Parameter	Description
digital assistant name	Name of the digital assistant to be deleted

Example:

joule delete <your_digital_assistant_name>

> **Note**
>
> Extensibility Developers cannot delete assistants named after sap_digital_assistant.

2.1.2.15 joule remove

Description: Removes capabilities from deployed Digital Assistant

Command:joule remove <digital_assistant_name> --capability <capability namespace>:<capability name>

Arguments:

Parameter	Description
digital assistant name	The name of the deployed digital assistant
capability namespace	Specific capability(ies) to remove in format name-space:name

2.1.2.16 joule lint

Description: Execute static linter

Command: joule lint [options] [file/dir/glob]

Options:

Parameter	Value	Description
--format, -f	<format...>	Formatter (default: ["pretty"])
--severity-level	<info|warning|error>	SeverityLevel (default: "warning")

Arguments:

Parameter	Description
file/dir/glob	Path to the capability or glob (default: "")

Examples:

- joule lint
- joule lint -f json:json-result.json --severity-level error
- Available lint rules are::
  - **UniqueScenarioDescription**: each scenario must have a unique description across digital assistant. Rule scope depends on where joule lint runs:
    - Root Level (folder with `*da.sapdas.yaml`): Lints all scenarios in every capability (any folder depth) and enforces global uniqueness,
    - Capability Level (folder with `capability.sapdas.yaml`): Lints only scenarios in that capability and won't check the scenarios outside the specified capability.
  - **NoEmptyDescription**: each scenario must have a non-empty description
  - **NoHardCodedUrl**: no hard coded urls are allowed
  - **OdataRequestPagination**: odata Joule action should have $top pagination query parameter

  Grace period features:

- Some linter rules may include a grace period during which they appear as Warnings instead of Errors.
- The message will indicate when the rule will transition to an Error.
- After the grace period ends, the rule will automatically return its default severity level (Error).
- Grace periods are configured by the development team and cannot exceed 3 months.

2.1.3 Troubleshooting Guide

CLI Login Error Scenarios and Steps to fix them

Login Options

Following are the 3 login options:

- **Option 1** (for production environment)
  Run `joule login`.
  Enter sequentially *Authentication URL*, *Client ID*, *Client Secret*, *Username* and *Password*. API URL is not needed.
- **Option 2**
  Run `joule login --apiurl <apiurl>`.
  Enter sequentially *Authentication URL*, *Client ID*, *Client Secret*, *Username* and *Password*.
- **Option 3**
  Run the following command with entire data: `joule login --apiurl <apiurl> -a <authurl> -c <clientid> -s <clientsecret> --username <username> --password <password>`.

Possible errors and ways to debug and fix them

Wrong Authentication URL

Make sure the URL format is "https://<tenant>.authentication.<landscape>.hana.ondemand.com" if you are using the XSUAA login service, or "https://<IAS_tenant>.accounts*.ondemand.com" if you are using the IAS login service.

API URL is invalid

To ensure that the URL is valid, open the link in a browser. You should see a white page with the message "Service up and running".

Request failed with status code 401

Make sure you use the correct Client ID, Client Secret, Username, or Password.

Request failed with status code 403

(Request can not be processed with your role): To fix the issue, ensure that you have the correct roles during the Onboarding Step as described in Assign Roles.

Wrong type of quotes

Use single quotes to wrap parameter values when using inline parameters, not prompts.

Handling Special Characters

If the Client Secret or the Password contains a $ symbol, it must be replaced with \$ by adding a backslash (\) before the $ sign.

No User password asked

The system cannot work with the library *keytar* (managing secrets). This will not prompt to specify the *password* and the *instance client secret*. For more information, see Store secrets in Linux .

As shown earlier, the Joule Studio CLI command directly logins after having asked for the Authentication URL, Instance Client ID, Username without asking for the User password. The joule is running under a Unix system (Mac OS, Linux, WSL, ...) with some required libraries missing.

As mentioned here:

- Install and Update the CLI.
- If you are using Linux, you should have installed libsecret, gnome-keyring or another keyring desktop application compatible with libsecret.
- Install any other components as per the official documentation.

As temporary workaround, use the *--unsecure-storage* option on the login command line. For more information, see Joule Studio CLI Commands [page 6]. This is not recommended as this will store credentials in clear instead of storing them in the Operating System Credentials Store. This option is only designed to be used for automated process where the necessary Operating System Libraries cannot be installed.

> **Note**
>
> Ensure to understand the risks before using this option.

Environment specific error

Error: auth token fetching failed: () (code: AUTH_FETCH_TOKEN_FAILED)

When using **Node.js v20 or higher**, certain internal changes may cause network-dependent tools (like *joule login*) to fail unexpectedly. These issues are often environment-specific and can be hard to investigate. One observed cause relates to automatic IP family selection (IPv4 vs IPv6) that may lead to connectivity issues on some systems or configurations.

To mitigate this, you can disable automatic IP family detection by setting the NODE_OPTIONS environment variable.

Linux/MacOS:

> **Sample Code**
>
> ```
> export NODE_OPTIONS="--no-network-family-autoselection"
> ```

Windows:

> **Sample Code**
>
> ```
> set NODE_OPTIONS=--no-network-family-autoselection
> ```

This workaround is documented officially by *Node.js: NODE_OPTIONS* documentation.

Debug File

The debug file contains information such as the command executed, the result of the command, and some information about the error in case of failure. You need to send this file to support when creating a ticket for an error. By adding the option *-d* to your command (login, compile, deploy,...) you can generate a debug file.

Sample Code

Example: joule login *-d*

```
Executing: login
Logged out
API URL: https://myapiurl.cloud.sap
error: auth token fetching failed: (Request failed with status code 401)
(code: AUTH_FETCH_TOKEN_FAILED)
debug-login-2023-12-13T14-14-10.390Z.json generated.
```

Sample Code

```
[
  {
    "timestamp": "2026-02-06T09:26:33.687Z",
    "command": "login",
    "level": "info",
    "message": "API URL: https://myapiurl.cloud.sap
  },
  {
    "timestamp": "2026-02-06T09:26:33.689Z",
    "command": "login",
    "level": "debug",
    "message": "request header",
    "data": {
      ""
    }
  },
  {
    "timestamp": "2026-02-06T09:26:33.694Z",
    "command": "login",
    "level": "debug",
    "message": "authentication url",
    "data": "https://myauthurl.ondemand.com"
  },
  {
    "timestamp": "2026-02-06T09:26:34.113Z",
    "command": "login",
    "level": "debug",
    "message": "error response",
    "data": {
      "error": "unauthorized",
      "error_description":
"{\"error\":\"invalid_grant\",\"error_description\":\"User authentication
failed: Unauthorized\"}"
    }
  },
  {
    "timestamp": "2026-02-06T09:26:34.122Z",
    "command": "login",
    "level": "error",
    "message": "error: auth token fetching failed: (Request failed with
status code 401) (code: AUTH_FETCH_TOKEN_FAILED)",
    "data": null,
    "error": {
```

```
      "message": "auth token fetching failed: (Request failed with status
code 401)",
      "name": "CliError",
      "errorCode": "AUTH_FETCH_TOKEN_FAILED",
      "stacktrace": "CliError: auth token fetching failed: (Request failed
with status code 401)\n
      "causedBy": {
        "message": "Request failed with status code 401",
        "name": "AxiosError",
        "stacktrace": "AxiosError: Request failed with status code 401.....)"
      }
    }
  }
]
```

2.2 Joule Studio Code Editor

The **Joule Studio code editor** VS Code Extension is a tool created to facilitate the build of **Joule Assistants**, offering various features like, *Generate from template*, *Validate the content*, *Quick Fix*, *Schema Update*, *Lint the content*, *Create connections*, *Compile capabilities*, *Deploy*, *Launch*, *Delete Assistants,* and more.

2.2.1 Download and Install the Joule Studio Code Editor

Prerequisites

- You have installed Node.js (minimum version 20.12.0 and maximum 24). You can download a version from https://nodejs.org/en/download/releases/ .
- You have installed the recent Visual Studio Code version `(x64 - 64 bit)` from https://code.visualstudio.com/download (minimum version 1.99.0). You can check the version by navigating to *Code* > *About Visual Studio Code* (macOS) or go to *Help* > *About (Windows).*

  > → Tip
  >
  > Make sure to install the `x64` version of Visual Studio Code.

Installation

Extension is available on the Microsoft Marketplace. Search for "Joule Studio Code Editor" on https://marketplace.visualstudio.com/ .

Create a new project

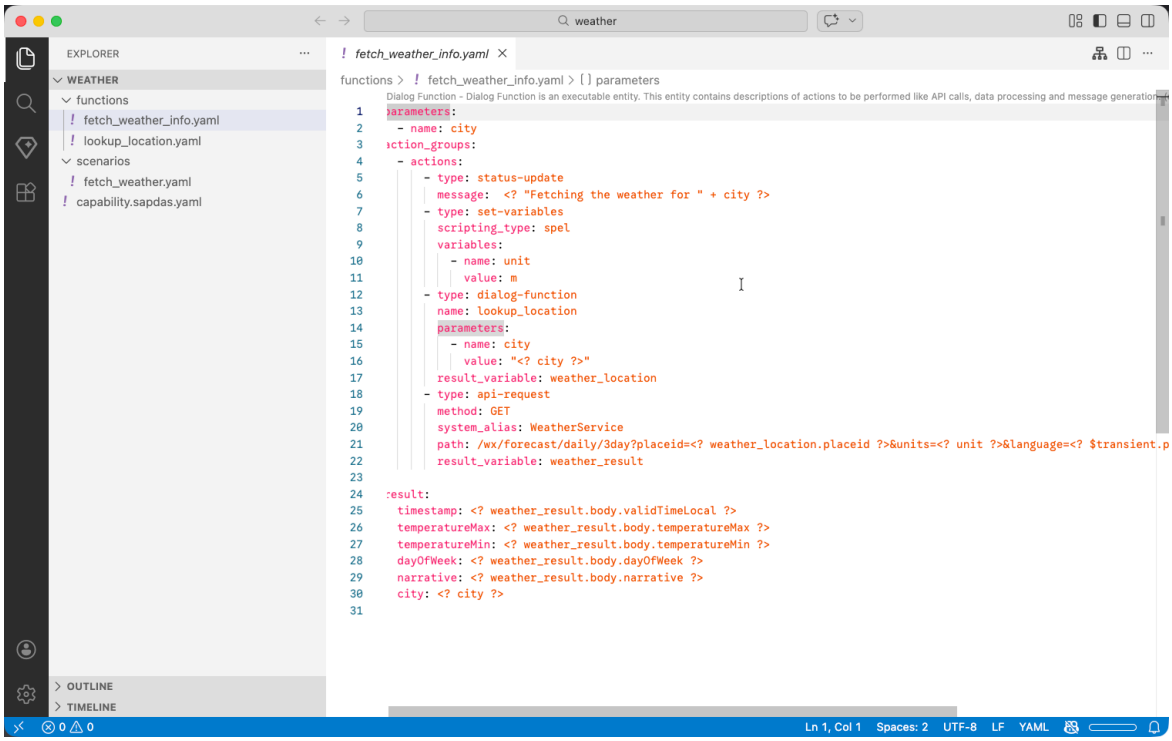
1. From Visual Studio, open a folder that contains the assistant folder.

2. Open the command palette (`ctrl`+`shift`+`P` or `command`+`shift`+`P`).
3. Search for Template Wizard.
4. Select *Generators* > *Joule*.
5. Select a Template (e.g. Weather) and follow the wizard.
   The folder structure should be:

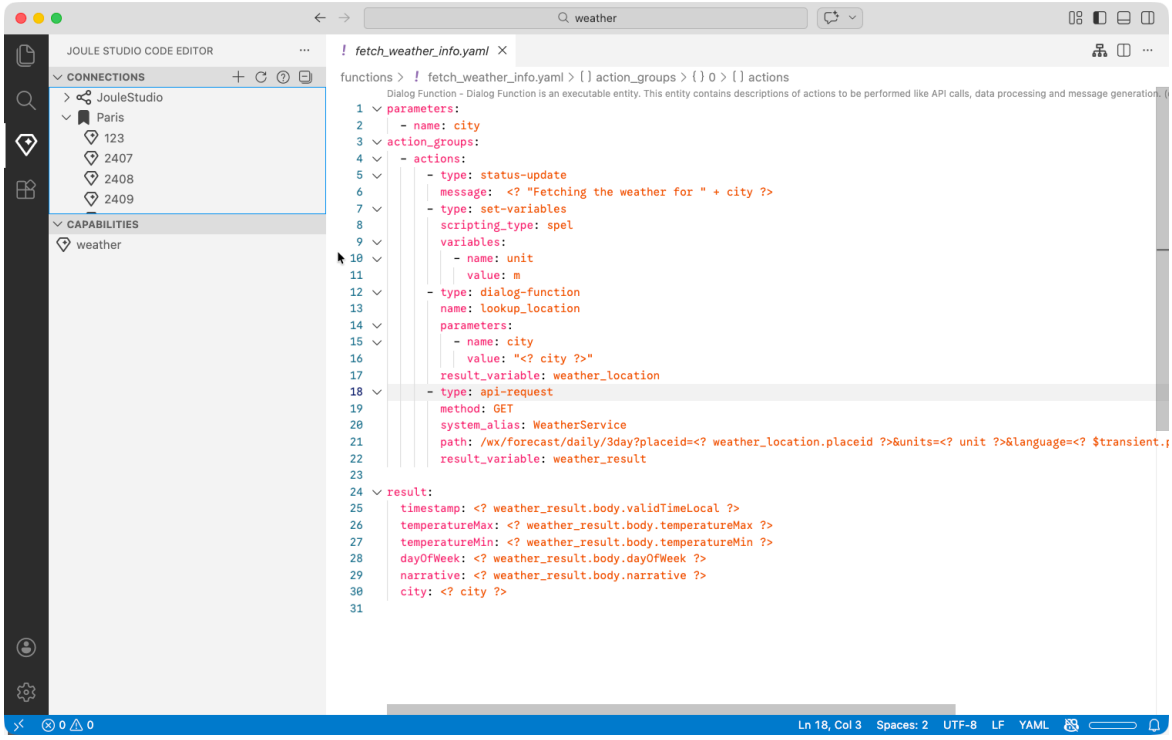
   Sample Code

   ```
   └── weather
         ├── weather_capability
         │   ├── functions
         │   │   └── fetch_weather.yaml
         │   ├── scenarios
         │   │   └── weather.yaml
         │   └── capability.sapdas.yaml
         ├── weather.da.sapdas.yaml
         └── da.sapdas.yaml
   ```

   The main folder contains one or multiple `da.sapdas.yaml` or `any name*.da.sapdas.yaml` file and one or more capability folders.
6. Open the explorer view.
   The left part shows the tree of objects with functions and scenarios.



7. Open the Joule explorer by clicking the Joule icon in the Activity bar on the left. You can connect to your Joule tenant in SAP BTP using the plug icon. Once the connection to the tenant is successfully established, the assistants deployed in your tenant are visible for each tenant. Right-click an assistant, you can delete or launch the selected assistant. For more information, see Joule Studio Code Editor Features.



8. You can register connections to multiple Joule tenants, inspect deployed assistant, compile capabilities, deploy assistants...

2.2.2 Joule Studio Code Editor Features

Extensibility developers can use the Joule Studio code editor tool to update and validate a design-time artifact.

The tool is activated in Visual Studio Code when an assistant exists in the workspace and a YAML file is open in the code editor.

The code editor provides the following features:

- Validation
- Quick fix
- Autocompletion
- Wizard to build a full assistant design-time artifact or enhance an existing one
- Management of connections to your Joule tenants in SAP BTP
- Compile capabilities
- Deployment of assistants to your Joule tenant
- Dependency graph
- Vibe coding

2.2.2.1 Validation

The following table describes the validations that are displayed while creating each DTA element.

Element	Validation
Capability	• filenames (excluding the file extension) should not have more than 30 characters • filenames should not contain any hyphens • local capability paths must exist
Dialog Functions	• filenames (excluding the file extension) should not have more than 30 characters • filenames and the names of their parent directories should not contain any hyphens • maximum depth of folders under dialog functions should be 3 (`capability/dialog/functions/depth1/depth2/depth3/your_function.yaml`) • dialog function folder names should not exceed 30 characters • system_alias must be defined in the `capability.sapdas.yaml` file in actions with api_request type • an error is displayed for each duplicate parameter in the function definition • initial_context variable must be defined in result section in Initialize hook when used in a function in a spel script (for example, <? $initial_context.initialize_result_variable ?>) • a warning is displayed if the non optional frontend action(s) is not in the scenario *required_frontend_actions* list

Element	Validation
Scenario	• target function must exist • target function parameters must be defined as slots or target parameters • response_context name list match function result • response_context names must be prefixed by $target_result • capability_context names must exist in the `capability_context.yaml` file • capability_context value must be prefixed by $target_result if you type dialog function result variables • an error is displayed for each duplicated slot or target parameters • an error is displayed when the slot refers to the same scenario • • message keys referenced in $i18n variables must exist in the `i18n/messages.properties` file. • a warning is displayed on the target function name in a scenario if the list does not contain non optional frontend action(s). • an error is displayed when a scenario title references an i18n message that contains placeholders

Joule Configuration File	• When the type of capability is *local*, the path of the capability folder or the path of the artifact is validated
	• Errors are displayed when the following rules are not fulfilled:
	  • The relative path of a folder does not exist
	  • The referred folder does not contain a `capability.sapdas.yaml` file
	  • The relative path of a file does not exist

Agent	• Errors are displayed when the following rules are not fulfilled:
	  • The target function does not exist.
	  • Parameters not defined in the target function are provided.
	  • The mandatory target function parameters are missing.
	  • Agent output format option field is missing or file doesn't exist when the output format is set to **json**.
	• Warnings are displayed when:
	  • The optional target function parameters are missing

2.2.2.2 Quick Fix

Element	Actions

Dialog Function	• If a call is made to a Dialog Function that doesn't exist, an error is displayed, and a Quick Fix is proposed to create the missing function.
	• In a Dialog Function call, when referencing the parameters:
	  • If the parameters do not exist, a quick fix is proposed to remove it from the current file
	  • If a mandatory parameter is missing, a quick fix is proposed to add this parameter to the function call
	• In a Dialog Function if the *system_alias* doesn't exist in the actions with the api-request type, an error is displayed, and a Quick Fix is proposed to replace it with the best 3 matches of system aliases available in the `capability.sapdas.yaml file`.

Scenario	• If the target function mandatory parameters are not covered by either slots of target parameters (3.5.0 or later)
	  • create the slot with parameter name
	  • create a target parameter with parameter name
	• If capability_context name doesn't exist, an error is displayed, and Quick Fix is proposed to add a context variable in the `capability_context.yaml` file (3.7.0 or later)
	• If the *required_frontend_actions* list does not contain non optional frontend action(s), a warning is displayed, and Quick Fix is proposed to:
	  • adds the missing frontend function(s) to the *required_frontend_actions*
	  • adds the required_frontend_actions and the functions otherwise above the *target* item

Capability	Update to the latest version.

2.2.2.3 Autocomplete

Element	Validation

Joule Assistant Configuration File	• When the type of capability is `local`, if you add the path of:
	  • the capability folder, you are prompted with folders containing a `capability.sapdas.yaml` file in the current directory and its subdirectories
	  • the artifact, you are prompted with files that have `.daar` extension.
	• If you type deployment identifiers in systems, you are prompted with a list of suffixes available in the product filter for conversational search (1.4.0).

Dialog Functions	• if you type a function name in the YAML file, you are prompted with a list of functions if the action type is dialog-function
	• if you type a function parameter name in the YAML file, you are prompted with a list of parameters available in function if the following conditions are fulfilled:
	  • when function specified as action exists
	  • when function specified as action has parameters
	  • when you have not specified all available parameters
	• if you type a system alias in actions with api-request type, you are prompted with a list of system aliases available in the `capability.sapdas.yaml` file

Element	Validation

Scenarios	• if you type a functions name as target name in the YAML file, you are prompted with a list of functions if the target type is function
	• if you type a function parameter name as slot name in the YAML file, you are prompted with a list of parameters available in target function if the following condition are fulfilled:
	  • when function specified as a target exists
	  • when function specified as a target has parameters
	  • when you have not specified all available parameters
	• response context names are completed with target function result variables
	• capability context names are completed with list of values in `capability_context.yaml` file
	• if you type a value in the capability context using $target_result, you are prompted with a list of target function result variables
	• when entering $i18n. followed by a string, autocomplete will suggest matching message keys from the `i18n/messages.properties file`.

Agent	• if you type a functions name as target name in the YAML file, you are prompted with a list of functions when the target type is function
	• if the default ouput format is **json**, you are prompted with a list of **json** files in the agents directory
	• if you type a function parameter name as parameter's name in the YAML file, you are prompted with a list of parameters available in target function when the following condition are fulfilled:
	  • when function specified as a target exists
	  • when function specified as a target has parameters

2.2.2.4 Wizard to Build the Assistant

The Joule wizard prompts you to provide all expected information (name, description and so on) to create a new assistant.

- You are prompted to provide all mandatory information for creating scenarios and functions.
- Once you have provided the requested information for each step, the structure of the DTA is automatically created with all the expected directories and files fulfilling the DTA Schema requirements.
- Yeoman Wizard fields requirements follow schema validation.

- The files that are created, fulfill the DTA schema requirements for all types of wizards.

2.2.2.5 Using Joule Explorer

To open *Joule Explorer*, in the activity bar, click the *Joule* icon (on the left).

In the *Joule Explorer* view, you can manage connections to your Joule tenants in SAP BTP and carry out the following tasks:

Action	Details
Register connection	Define a new connection by clicking the *Register Connection* icon in the header of the *Joule Explorer* and follow the VS Code Command Palette wizard.
Sort assistants	Right-click an existing connection and choose *Sort assistants*. Clicking on this button will sort assistants through nameAsc-nameDesc-default.
Edit connection	Right-click an existing connection and choose *Edit connection*. Follow the wizard to edit the connection.
Unregister connection	Right-click an existing connection and choose *Unregister connection*. A confirmation step will be displayed to confirm the command. **Note** Removing a connection will only delete the connection in Joule Extension. The Joule tenant and the assistants will not be deleted.
Delete assistant	Right-click an assistant and choose *Delete assistant*. A confirmation step will be displayed to confirm the command. **Note** Deleting an assistant will delete the assistant from your Joule tenant in SAP BTP. You will not be able to launch the assistant any more using the *Launch assistant* option.
Launch assistant	Right-click an assistant and choose *Launch assistant*. The assistant opens in the default browser.

2.2.2.6 Using the Explorer View

To open the *Explorer* view, in the activity bar, click the *Explorer* icon.

You can carry out the following tasks in the *Explorer* view:

Action	Details
Compile the capability	Right-click a capability (`capability.sapdas.yaml`) and choose *Compile this capability*. The VS Code Command Palette will be displayed listing the already registered connections. Select a connection from the list. If only one connection is registered, the VS Code Command Palette will not be displayed and the registered connection will be used by default. To check the status of the compile operation, open the terminal in VS Code and click *Output* tab. Choose *Joule: CLI* from the dropdown menu displayed in the header of the tab.
Deploy the artifact	Right-click an assistant YAML file (`*.da.sapdas.yaml`) and choose *Deploy Assistant*. The VS Code Command Palette will be displayed listing the already registered connections. Select a connection from the list. If only one connection is registered, the VS Code Command Palette will not be displayed and the registered connection will be used by default. To check the status of the deployment operation, open the terminal in VS Code and click *Output* tab. Choose the *Joule: CLI* from the dropdown menu displayed in the header of the tab. If the assistant is successfully deployed in your Joule tenant in SAP BTP, it will be added in the list of deployed assistants under the connection chosen for this operation in the *Joule Explorer* view.
Open the assistant in your Web browser	Right-click an assistant YAML file (`*.da.sapdas.yaml`) and choose *Open in your Web Browser*. The VS Code Command Palette will be displayed listing the already registered connections. Select a connection from the list. If only one connection is registered, the VS Code Command Palette will not be displayed and the registered connection will be used by default. Joule Web Client opens in your default browser. If there is no assistant available for the chosen connection that corresponds to the name defined in the assistant file, an error is displayed in the Joule Web client.

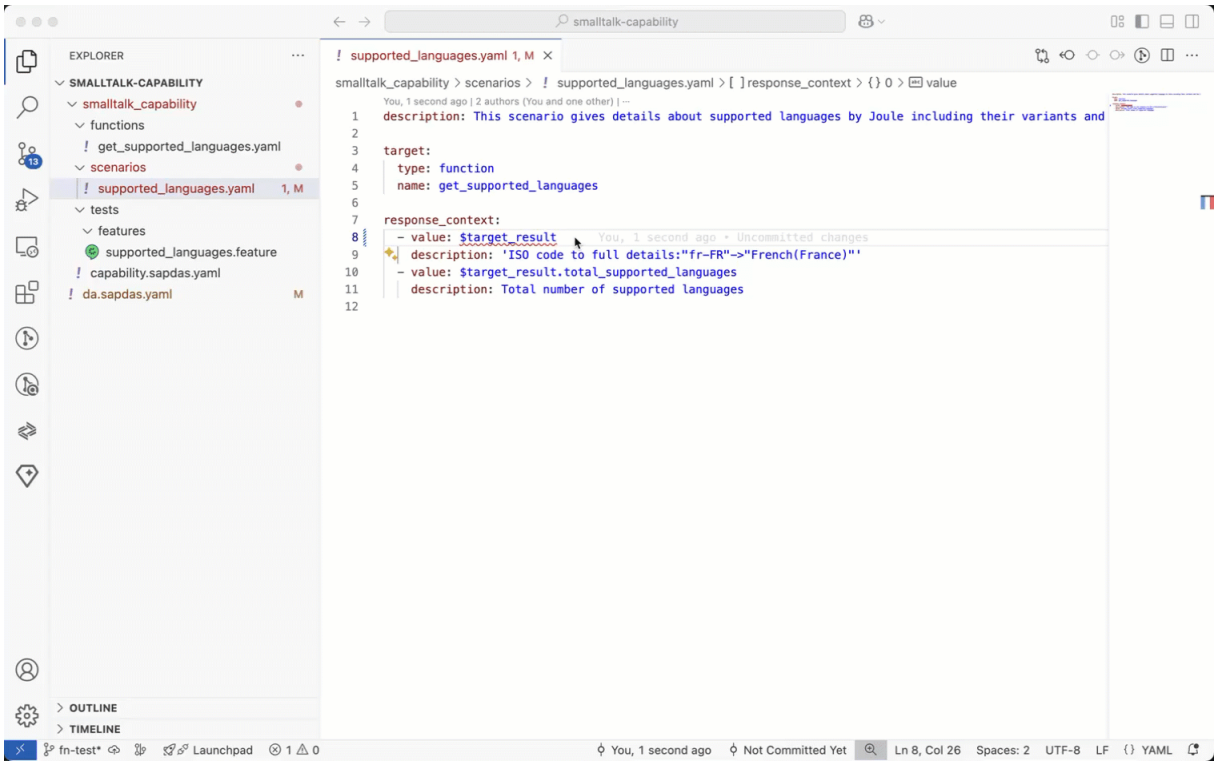
2.2.2.7 GitHub Copilot for Building Joule Assistants

You can use GitHub Copilot with the custom Joule Agent to create assistants or resources (such as capabilities, scenarios, or functions) by prompting the agent using natural language. For more information, see Walkthrough [page 28].

2.2.3 Walkthrough

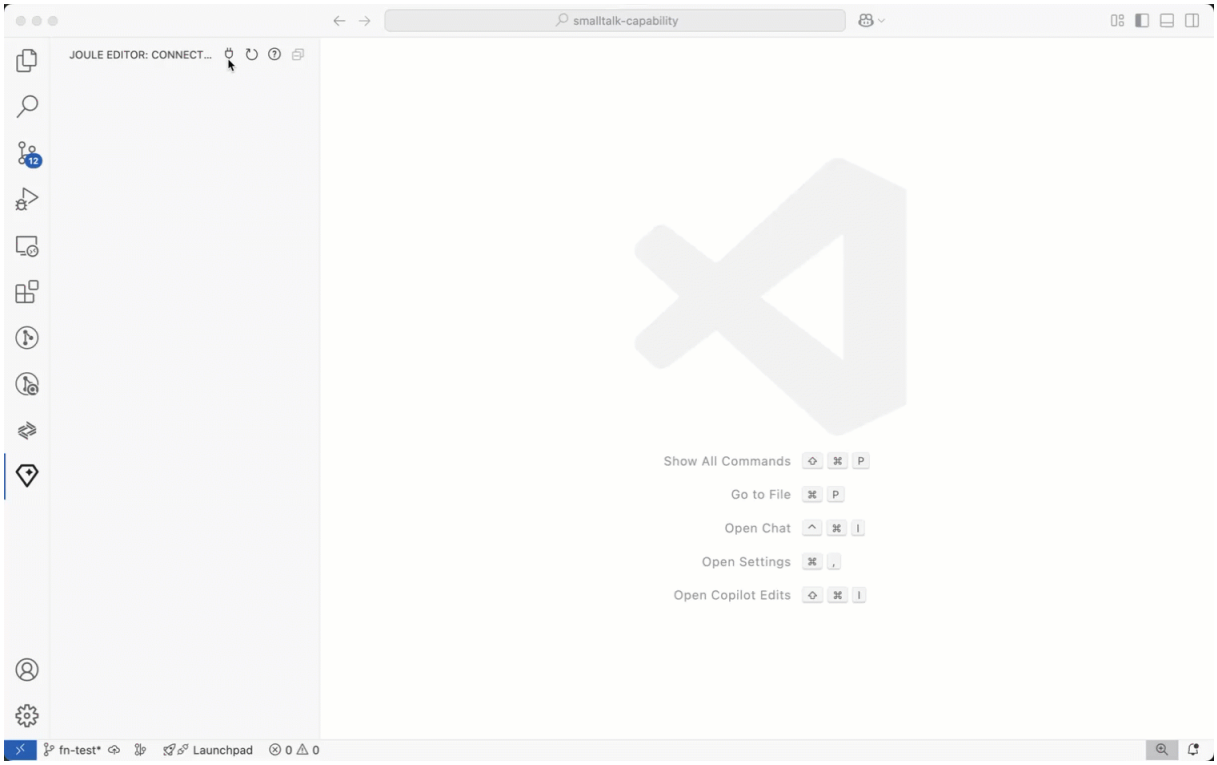
Getting Started with the Joule Studio code editor

Explore the features that will accelerate and change the way you build Joule Assistants.



Create a Connection

In this step, we will create a connection with the Joule tenant.



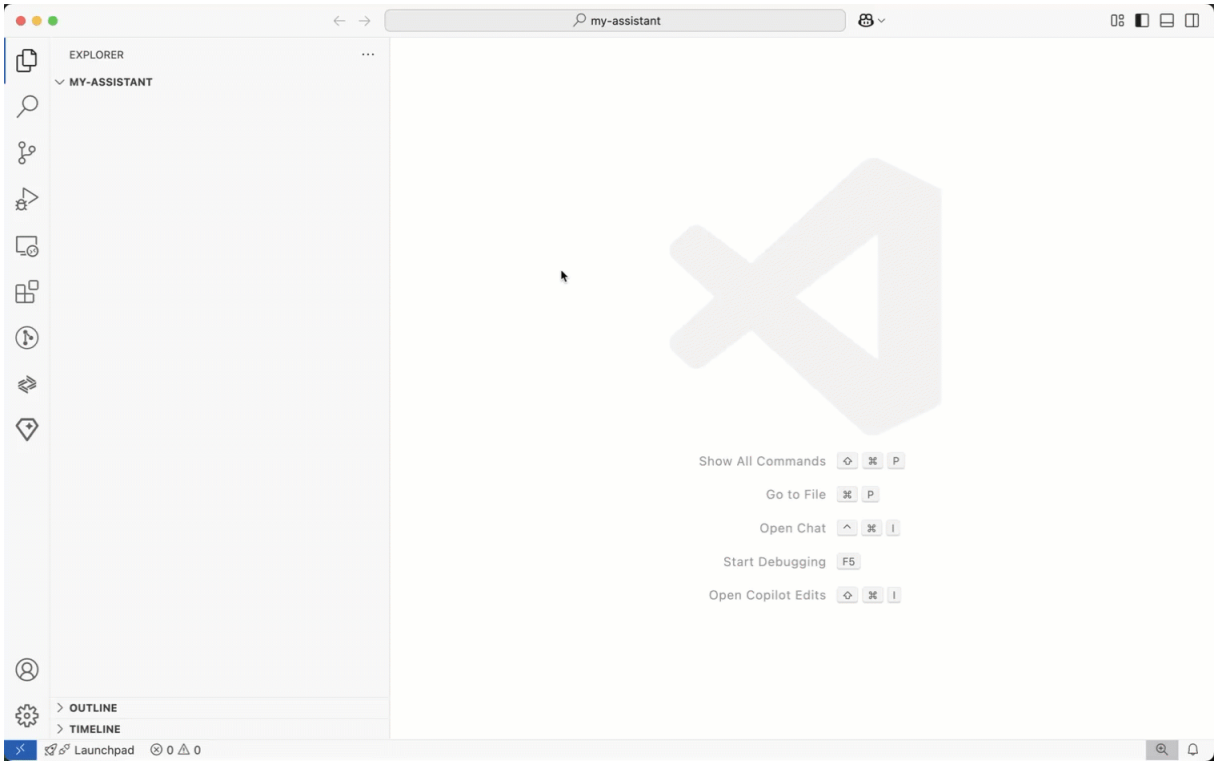
In order to **Compile** or **Manage Assistants**, a connection to the tenant is required.

To create a connection, you will need to pass the following parameters:

- API URL (For example, https://mytenant.eu10.sapdas.cloud.sap
- AUTH URL (For example, https://mytenant.authentication.eu10.hana.ondemand.com)
- CLIENT ID
- CLIENT SECRET
- USERNAME
- PASSWORD / PASSCODE

Generate from a Template

In this step, we will generate an assistant from a template.



Use the Command Pallet (SHIFT + COMMAND + P) to find and run the command.

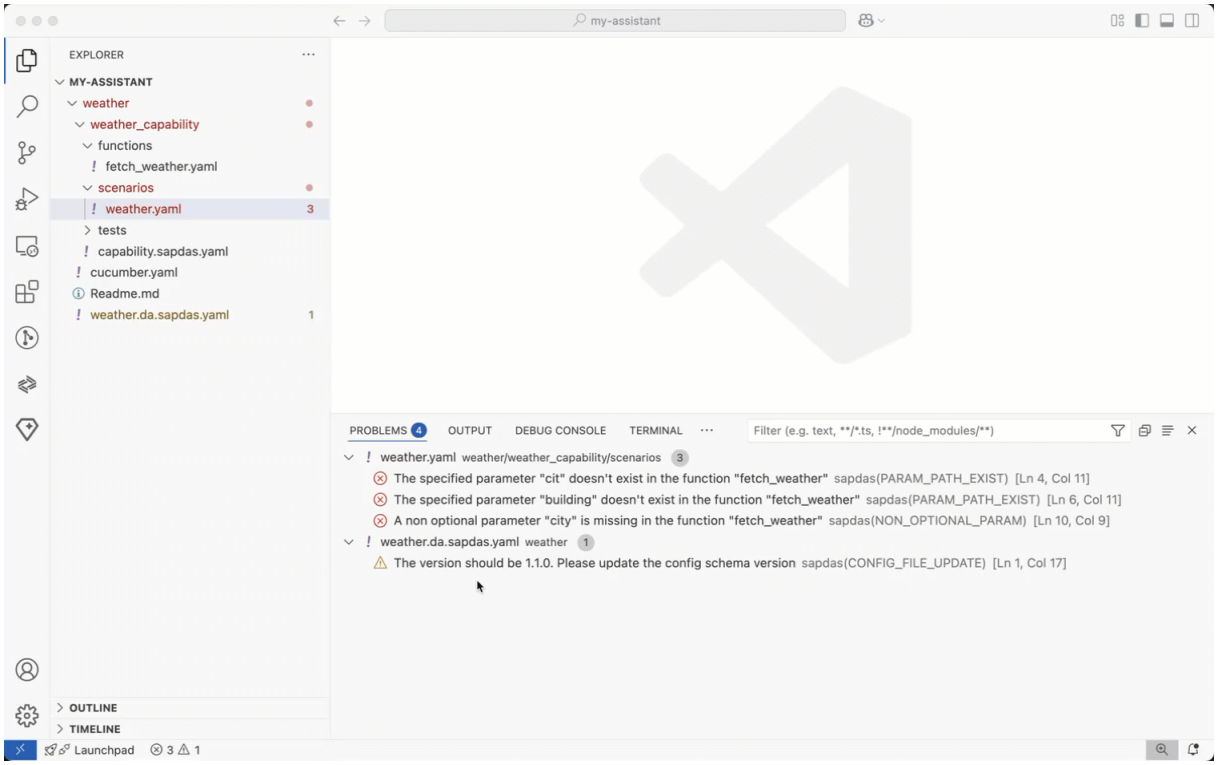
```
Open Template Wizard
```

Select the `Joule` Generator and start building your assistant, either using the **Weather Template** or the **Custom Template** which gives you more control over the data.

To learn more on how to update and create capabilities that fit your needs, see Build a Capability for Joule [page 41].

Validate and Quick Fix

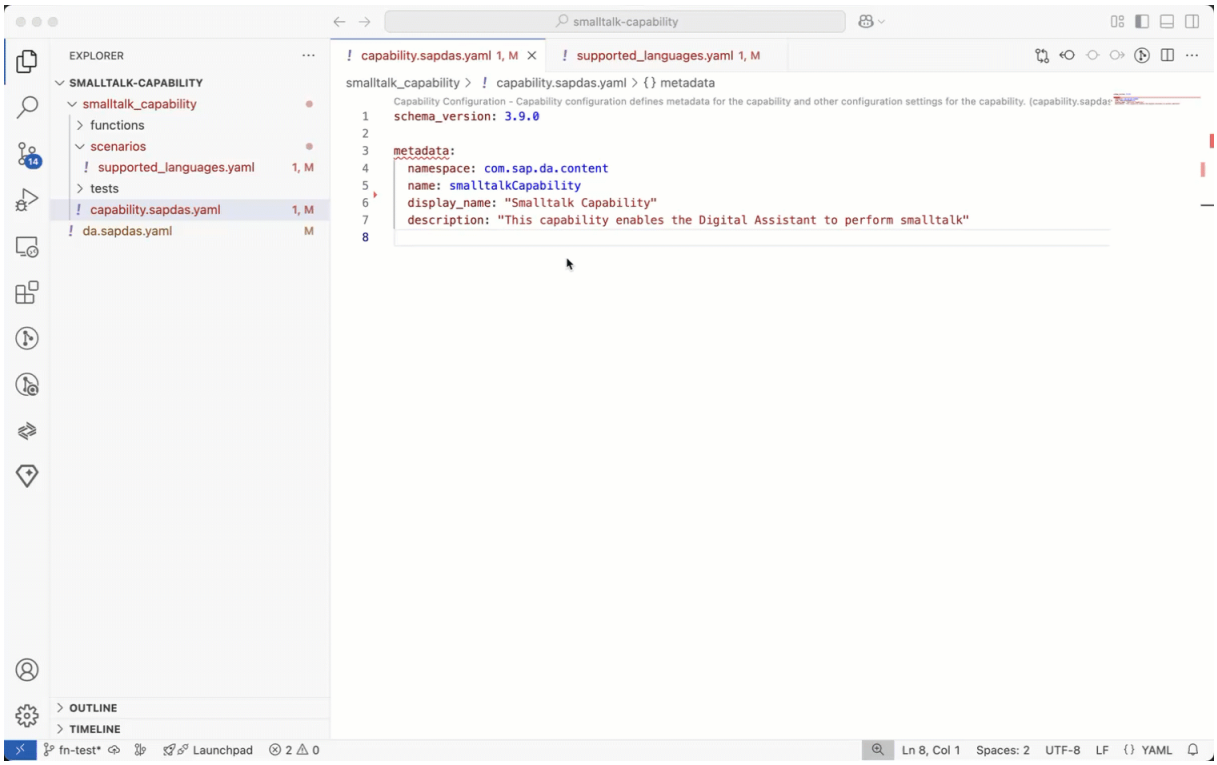
In this step, we will validate some content and apply quick fixes.



Utilizing the built-in validation and quick fix will help maintain clean and consistent content, preventing unused parameters, typos, or invalid properties.

Completion of Values

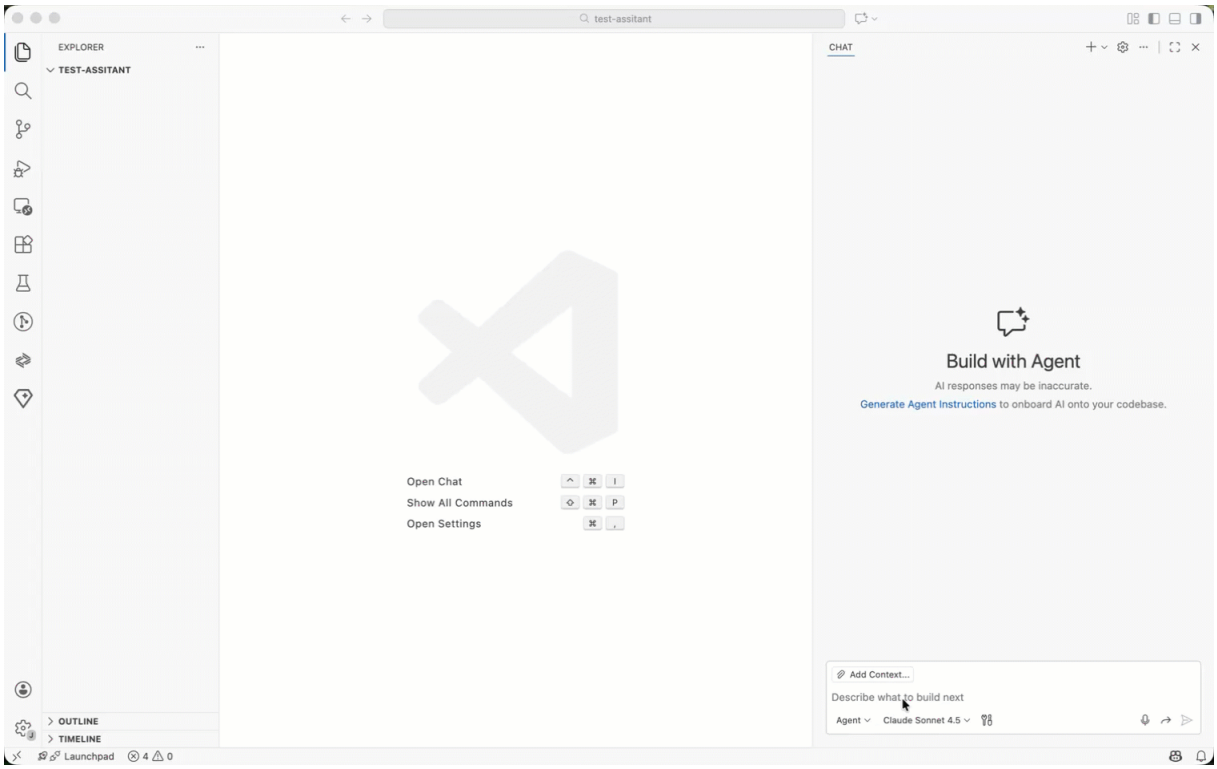
In this step, we will see how to use completion for values.



Utilizing the completion option will assist you in selecting only valid property values and preventing mistypes.

Joule and GitHub Copilot

In this step, we will see how to use GitHub Copilot to build Joule Assistants.



Leverage the power of AI. With **GitHub Copilot** and **@joule Helper**, you can create scenarios and functions that fit your business needs.

2.2.4 Handling Linter Rules

At times, it is necessary to disable linter warnings in YAML DTA files when they are unresolvable, irrelevant, or need to be temporarily ignored during development.

Annotations

To disable rules in part of a file, use the `joulelint-disable` annotation in the following format:

```
parameters:
  - name: city
action_groups:
  - actions:
      # joulelint-disable
      - type: api-request
        method: GET
        path: https://weather.com/wx/forecast/daily/3day
```

```
        result_variable: weather_result
      # joulelint-enable
```

You can also specify the rules that you need to temporarily disable:

```
parameters:
  - name: city
action_groups:
  - actions:
      # joulelint-disable NoHardCodedUrl
      - type: api-request
        method: GET
        path: https://weather.com/wx/forecast/daily/3day
        result_variable: weather_result
      # joulelint-enable NoHardCodedUrl
```

> **Note**
>
> `# joulelint-enable` is intended to be used only in conjunction with `# joulelint-disable`, otherwise it will be ignored

To disable rules for specific lines, you can use the following syntax:

```
parameters:
  - name: city
action_groups:
  - actions:
      # ...
      - type: api-request
        method: GET
        path: https://weather.com/wx/forecast/daily/3day # joulelint-disable-line
        result_variable: weather_result
      # ...
      - type: api-request
        method: GET
        # joulelint-disable-next-line
        path: https://weather.com/wx/forecast/daily/3day
        result_variable: weather_result
```

Both `# joulelint-disable-line` and `# joulelint-disable-next-line` accept the list of rules that need to be disabled.

```
parameters:
  - name: city
action_groups:
  - actions:
      # ...
      - type: api-request
        method: GET
        path: https://weather.com/wx/forecast/daily/3day # joulelint-disable-
line NoHardCodedUrl, OdataRequestPagination
        result_variable: weather_result
```

In the example above, two rules will be disabled for the url: OdataRequestPagination and NoHardCodedUrl

Additional information on `# joulelint-enable`

`# joulelint-enable` can be used to enable only specific rules for a portion of a file

For example, you have a sample function where you want to disable all the rules, but enable one rule for a specific part of the file. To do this, you can use the following:

```
# joulelint-disable
parameters:
  - name: city
action_groups:
  - actions:
      # joulelint-enable NoHardCodedUrl
      - type: api-request
        method: GET
        path: https://weather.com/wx/forecast/daily/3day
        result_variable: weather_result
      # joulelint-disable
      - type: api-request
        method: GET
        # joulelint-disable-next-line
        path: https://weather.com/wx/forecast/daily/3day
        result_variable: weather_result
```

In the example above, only first api-request action will be reported for using the hardcoded url.

3 Building Capabilities for Joule

A developer can start by defining a design time artifact (DTA) which is a YAML formatted human readable textual representation of a group of resources that formulate the capability. It consists of a dataset, dialog flow, and configurations. An assistant configuration is a YAML format file which specifies a set of capabilities that should be assembled together to form the assistant.

Some of the advantages of creating an assistant with a DTA:

- A DTA can be treated like other forms of source code and managed using the same tools such as IDEs and version control systems, such as git.
- It allows multiple developers to collaborate on the development of the assistant capabilities, and enables important functionalities such as merging, conflict resolution, and an audit trail of changes.

Components of a DTA

Capability

A capability is a set of skills that semantically belong together alongside with the language understanding in the respective domain. For example, the capability *Purchasing* holds *order creation* and *check delivery status* skills along with the language understanding specific for each skill. Developers create assistants and capabilities using YAML formatted text to implement their business scenarios, which can be treated like other forms of source code and managed using the same tools such as IDEs and version control systems, for example, git. This approach allows multiple developers to collaborate on the development of assistants and enables important functionalities such as merging, conflict resolution, and an audit trail of changes.

Scenario

A scenario is the semantic representation of a dedicated use case. It contains the description of the use case, semantic relations to other use cases and the target to be executed. A scenario's target can be a function in which case a function must be specified to carry out the relevant scenario.

Dialog Functions

A dialog function is an executable block of instructions that is defined by a developer to perform multiple tasks. The complete execution of the dialog function results in either producing data or a UI model that will be displayed to the user.

Related information:

Design Time Artifact (DTA) Specification

3.1 Design Time Artifact (DTA) Specification

The Design Time Artifact (DTA) Specification is a proprietary set of structural requirements that developers must follow when they create artifacts. The specification includes a predefined folder structure for an artifact

and specific YAML structures for each resource that describe different parts of an artifact, such as the data set, the conversation flow, and the capability configuration settings.

DTA Specification Versions

New features are implemented with new versions of the DTA Specification. To use new features, you must update the version in the `capability.sapdas.yaml` file to the corresponding version.

> **Note**
>
> Later minor versions for a given major version are always backward compatible.

Types of Design Time Artifacts (DTA)

DTA	Description
Capability	Source code representation of a capability in a human-readable file format that adheres to a predefined folder structure. A capability comprises a data set, the conversation flow, which is defined using dialog functions, and the capability configuration. The data is expressed in YAML format in accordance with the resource schema definition.

Structure of a Capability

The following example shows the structure of design time artifact, that is, a capability:

> **Sample Code**
>
> Version 3.x

```
└── sample-capability
    ├── scenarios
    │   └── sample_scenario.yaml
    ├── functions
    │   └── sample_function.yaml
    ├── agents
    │   └── agent_1.yaml
    │   └── agent_2.yaml
    │   └── agent_1.schema.json
    │   └── agent_2.schema.json
    ├── i18n
    │   ├── messages_de.properties
    │   ├── messages_en.properties
    │   └── messages.properties
    ├── capability_context.yaml
    ├── deployment_extension.yaml
```

```
    └── capability.sapdas.yaml
```

The DTA Specification follows the *Convention over Configuration* paradigm.

The following table lists all objects that are part of a DTA structure and recommendations for creating these objects:

File	Optional/Mandatory	Description
`sample-capability`		Although recommended to have the same name as that of the capability, a developer is free to choose any name for this folder. It must have a mandatory `capability.sapdas.yaml` file at the root containing the capability configuration.
`sample-capability/capability.sapdas.yaml`	Mandatory	For more information, see Build a Capability [page 41].
`sample-capability/deployment_extension.yaml`	Optional	For use with system context filtering to control capability or scenario deployment.
`sample-capability/capability_context.yaml`	Optional	For use when defining statically referenced capability context variables.
`sample-capability/functions`	At least one function should be present if there is at least one scenario with type function	Up to three levels of sub folders are allowed in this folder for organization. For more information, see Dialog Functions [page 58].
`sample-capability/hooks`	Optional	Predefined life cycle functions should be in this folder. Currently, only initialize is supported and should be defined using file name `initialize.yaml`. For more information, see Initialize Conversation [page 78].
`sample_da/sample_capability/scenarios`	Mandatory starting 2.0.0 version and 3.0.0 version	At least one scenario should be present. You can choose a name for the scenario yaml files. Subfolders up to a maximum nesting of three are allowed inside this folder.
`sample_da/i18n`	Optional starting in 3.7.0	Used to store properties files to define localized text. Properties files for all languages to be stored here.

Digital Assistant (DA) Configuration

Capabilities are independent artifacts, which can be assembled into a digital assistant (DA). Metadata, like version and name for the DA, along with data on the capabilities that should be assembled together in the DA , should be specified in a configuration file by name `*.da.sapdas.yaml`.

Sample Code

Example of DA configuration file (local development)

```
schema_version: 1.0.0 #DA Config Schema Version
name: MyTestDigitalAssistant #name of the Digital Assistant
capabilities: # capabilities to be assembled
  - type: local
    folder: ./my_capability # folder path to a local capability DTA
  - type: local
    folder: ./my_second_capability
  - type: local
    path: ./my_third_capability/my_third_capability.daar # path to a local
pre-compiler daar file

conversational_search: # optional: enable conversational search feature
  enabled: true
  product_filter: # products the search will be perfomed on (SAP Help Product
IDs)
    - SAP_SUCCESSFACTORS
```

Naming Conventions

DA Configuration

Resource Type	Character Length	Constraints
Assistant name	50	It can contain only alphanumeric and underscore characters. Spaces, consecutive underscores are not allowed.

Capability DTA

Resource Type	Character Length	Constraints
Capability name	50	It can contain only alphanumeric and underscore characters. Spaces, consecutive underscores are not allowed.
Capability namespace	100	It can contain only alphanumeric characters.
Capability version	20	Semantic version qualifier → only SNAPSHOT supported

Resource Type	Character Length	Constraints
Capability display name	128	It can contain alphanumeric, underscore and hyphen characters. Cannot contain carriage return(\r), newline(\n), or tab(\t) characters.
Capability description	512	Cannot contain carriage return(\r), newline(\n), or tab(\t) characters.
Capability context variable	64	It can contain alphanumeric, underscore and hyphen characters. Cannot contain carriage return(\r), newline(\n), or tab(\t) characters.
Resource file names	30	They can contain only alphanumeric and underscore characters. Spaces, consecutive underscores are not allowed.
Resource grouping folder names	30	It can contain only alphanumeric and underscore characters. Spaces, consecutive underscores are not allowed.
Resource descriptions	128	Cannot contain carriage return(\r), newline(\n), or tab(\t) characters. Formatting using block scalars is allowed.
Context key (dialog functions)	64	• It must contain only alphanumeric and underscore characters. • It must not contain the character sequences `public` and `private`. • It must not be equal to `request`, `skip_user_input`, `transient`, `user`, `conversation`.
Dialog function properties (system alias, parameters name, result-variable)	64	It can contain only alphanumeric and underscore characters.
Dialog function: API Request Path	2048	Should be encoded.

Capability File Sizes

For more information, see Resource Limits for Capability Files [page 40].

3.2 Resource Limits for Capability Files

The following table describes resource limits that are relevant for the files used to build capabilities for Joule.

Resource	Limit
Maximum size of capability project zip file (**compressed**, that is, `.daar` file as returned from the compiler)	15 MB
Maximum size of capability project zip file (**uncompressed**, that is, unzipped `.daar` file)	35 MB
Maximum number of scenarios per capability	200
Maximum depth for scenario folders	3
Maximum size allowed for a scenario file	50 KB
Maximum number of functions per capability	500
Maximum depth for function folders	3
Maximum size allowed for a dialog function file	50 KB
Maximum size allowed for capability context file	200 KB
Maximum size allowed for initialize file	50 KB
Maximum size allowed for deployment extension YAML file	50 KB
For dialog requests, conversation request or utterance length for type `text`	2000 characters
Localization i18n file	512 KB **Note** As of DTA Specification version 3.22, a properties file cannot exceed 512 KB.
Localization i18n file total	16000 KB **Note** As of DTA Specification version 3.22, the total size of all properties files cannot exceed 16000 KB.

For information about the naming conventions for design time artifacts, see Naming Conventions [page 38].

3.3 Build a Capability

A capability is a set of skills that semantically belong together alongside with the language understanding in the respective domain. For example, a capability *Purchasing* holds *order creation* and *check delivery status* skills along with the language understanding specific for each skill. A capability contains the scenarios that contain the semantic representations of functionality, conversation flow defined using dialog functions, capability configurations, all expressed in a YAML format adhering to a resource schema definition.

Creating a capability at a minimum involves:

- Creating a scenario to define the functionality. Scenarios are representations of functionalities and can be a type of function
- In the case of a function scenario, a function to execute the functionality
- A mandatory configuration file to define the metadata and configurations for the capability.

Step 1: Create folder structure for the capability

Create a folder with sub-folder hierarchy as follows:

> **Sample Code**
>
> Bot project file structure
>
> ```
> ├── sample-capability
> │   ├── functions
> │   │   └── fetch_weather_info.yaml
> │   ├── scenarios
> │   │   └── fetch_weather.yaml
> │   └── capability.sapdas.yaml
> ```

> **Note**
>
> It is mandatory to use the specified folder structure for defining resources.

Step 2: Create resources in the capability project

Create a function to fetch weather and a scenario to semantically represent the functionality.

1. Create a function `fetch_weather_info.yaml` under the functions folder.

> **Sample Code**
>
> fetch_weather_info.yaml
>
> ```
> parameters:
>  - name: city
> action_groups:
>  - actions:
>  - type: api-request
>  method: GET
>  system_alias: WeatherService
>  path: /wx/forecast/daily/3day?placeid=<?city ?> &language=en-
> US&format=json
>  result_variable: weather_result
> result:
>  timestamp: <? weather_result.body.validTimeLocal ?>
>  temparatureMax: <? weather_result.body.temperatureMax ?>
> ```

```
  city: <? city ?>
```

Create a scenario `fetch_weather.yaml` under the scenarios folder.

Sample Code

fetch_weather.yaml

```
description: This function fetches the weather for a given city
slots:
  - name: city
    description: Weather information fetched for this city
target:
  type: function
  name: fetch_weather_info
```

Step 3: Configure the capability

Add the capability configuration details in the `capability.sapdas.yaml` file.

Sample Code

capability.sapdas.yaml

```
schema_version: <schema version>

metadata:
  display_name: Weather Capability
  namespace: com.sap.example
  name: sample_capability
  version: 1.0.0-SNAPSHOT
  description: >-
   Weather Capability
```

3.3.1 Defining Scenarios

In Joule, scenarios represent the use cases that are supported by a capability. The descriptions of the individual scenarios are used to intelligently select the most suitable scenario in response to a user's input. Furthermore, slots are used to gather additional input, and the response context is used to produce an intelligent result.

Folder and File Structure

Scenario files must be placed in the `scenarios` folder, which is located at the same hierarchical level as the `functions` folders.

Sample Code

Basic Structure of a Capability

```
sample_capability
```

```
├── functions
├── scenarios
├── capability_context.yaml
└── capability.sapdas.yaml
```

Code Syntax

Scenario File

```
description: <The provided scenario description will serve as the basis to
appropriately route a request to the correct processor>

slots:
  - name: <Parameter name. If used with dialog functions, it should match
with one of the function's parameters>
    description: <Some information that is required to execute the scenario>

target:
  type: function
  name: <name of the dialog function>
  parameters:
    - name: <parameter name>
      value: <parameter value either as static string or value name from
capability context with $capability_context prefix>

response_context:
  - value: <result variable>
    description: <description for response context>
```

Definition of a Scenario

A scenario has the following attributes:

Attributes of a Scenario

Attribute	Type	Description	Required	Rules	Limit	Available As of Version
`title`	string	Title of the scenario	false	Must be in the English language or a reference to an i18n text, for example, `$i18n.VIEW_PRODUCT_INFO`	Maximum number of characters: 32	3.16.0

Attribute	Type	Description	Required	Rules	Limit	Available As of Version
`description`	string	Serves as grounding information to route a request to the intended scenario	true	See Guidelines [page 47] to define a scenario description.	Maximum number of characters: 1000	3.0.0
`feature`	string	Feature ID for the scenario	false	Must be a string	Maximum number of characters: 64	3.23.0
`slots`	array	Contains all the information required for the execution of the target	false	See slots [page 45]	Maximum number of elements: 60	3.0.0
`target`	object	Reference to the dialog function to execute	true	See target [page 46]		3.0.0
`response_context`	array	Defines how the result of a dialog is mapped to the response_context used for response generation, response generation is skipped if response_context is absent	false	See Gen AI Response Generation [page 48]	Maximum number of elements: 10	3.3.0
`capability_context`	array	Through the capability context variables, you can maintain a technical context that can be maintained during an active conversation with the user	false	See Capability Context Variables and Static Parameters [page 53]		3.5.0

Attribute	Type	Description	Required	Rules	Limit	Available As of Version
`visibility_condition`	object	Defines a condition to filter scenarios	Defines a condition to filter scenarios	See Role-Based Scenario Access [page 50]	See Role-Based Scenario Access [page 50]	3.14.0
`required_frontend_actions`	array	List of required frontend actions	false			3.24.0

Definition of a Slot

A slot has the following attributes:

Attributes of a Slot

Attribute	Type	Description	Required	Limit	Available As of Version
`name`	string	Parameter name If used with dialog functions, it must match one of the function's parameters.	true	Maximum number of characters: 64	3.0.0
`description`	string	Some information that is required to execute the scenario	true	Maximum number of characters: 512	3.0.0

> **Note**
>
> For each slot, only one string value can be injected.

Definition of a Target

A target has the following attributes:

Attributes of a Target

Attribute	Type	Description	Required	Rules	Limit	Available As of Version
`type`	enum	Target type to be executed	true	Must be of type `function` to define slots, response context, capability context or target parameters.		3.0.0
`name`	string	Reference to the dialog function to be executed	true	Must reference an existing function that is defined in a dialog function. For information, see Dialog Functions [page 58].	Maximum number of characters: 256	3.0.0
`parameters`	object	Defines the function parameters passed by executing the function	false	See parameters [page 54]		3.5.0

For more information, see Gen AI Response Generation [page 48].

Example of a Scenario File

A function exists in the directory `sample_capability_2/functions` with the file name `greet.yaml`. This function will be referenced in the target by the following scenario file in `sample_capability_2/scenarios/greeting.yaml`.

Sample Code

Example of a scenario of type function

```
description: This functionality can be used to answer greetings.

target:
  type: function
  name: greet
  parameters:
    - name: client_name
```

```
      value: $capability_context.client_name
    - name: greeting_message
      value: Hello
```

Guidelines

Do

- Use natural and simple English language.
- Describe your scenario within 2 to 4 sentences and a maximum 1000 characters.
- Be as specific and precise but also as short as possible.
- Ensure to include all relevant key words.
- If you need to put in many different key words or capabilities you can also build long sentences (Example: use comma separated lists).
- Try to avoid repetitions in non key words.
  Example: `The leave request app allows to...` (1st sentence). `With the leave request functionality you can...`(2nd sentence).
- Primarily describe what the feature can do and not what a user wants to do. If users tend to use special vocabulary, you must also consider those specific key words in your scenario description.
- If certain terms are not part of the common (non-business) language, consider describing them in the descriptions.
- Formulate the functionality in descriptive words.
- Be specific while formulating descriptions, always indicating what the user can achieve using this functionality.
- Avoid using personal pronouns such as *you* and *I* in the perspective of the user, rather describe the scenario from the perspective of Joule.

Don't

- Don't mention slots in the scenario descriptions.
- Don't mention personas (for example, manager, HR admin).
- If only certain roles should have access to a scenario, this should be modeled via Role-Based Scenario Access [page 50].
- Don't mention access restrictions. Access restrictions should be modeled via Role-Based Scenario Access [page 50].
- Don't use any instructions in the descriptions. Especially avoid any instruction which are targeting the LLM behavior.
- Don't use superfluous filler words and avoid expressing preferences or sentiments.
  Those words do not describe functionality. They influence the Large Language Model (LLM). Don’t use words or phrases like *straightforward*, *better use*, etc.
- Don't rank or highlight capabilities.
  List the capabilities in a neutral manner to avoid influencing the LLM.
- Don't reference other features within your description.
  This decreases the probability of Joule finding your feature. You must be sure in which context your scenario should be found and in which context a referenced feature should be found and describe them separately.

- Don't describe what a scenario is unable to do. Describe the capabilities.
- Don't use generic trigger words in the scenario description. Using words like *delete*, *remove*, *erase* will trigger this function. Such words are not allowed.
- Don't use abbreviations without having the word written out first, and the abbreviation used in brackets behind.

3.3.1.1 Gen AI Response Generation

You can describe the context that might be relevant for the response generation in the scenario DTA. The context must reference the attributes defined in the result of target dialog function. Context definition cannot use Spring Expression Language (SpEL) or any other form of scripting but can only reference root-level attributes.

The response generation occurs using the following rules:

- There can only be one `response_context` section per scenario descriptor.
- If the `response_context` section is present, a response message will be generated.
- The `response_context` section must be in a valid YAML format.
- The response context has a maximum size of 10000 bytes.
- A maximum of 10 fields can be defined for the response context.

The `response_context` will implicitly influence the type of response and the information that will be present in it. The exclusive definition in the scenario descriptor will help filter out non-relevant data from the dialog function result.

Definition of response_context

Attribute	Type	Description	Required	Limit
`value`	string	Dialog function result variable name	true	Maximum number of characters: 50
`description`	string	Description for that value in response generation prompt	true	Maximum number of characters: 50

Sample Code

Example of a dialog function

```
parameters: ...

action_groups: ...

result:
  totalEntitlement: <? body.total ?>
  leftVacation: <? body.available ?>
```

Sample Code

Example of a scenario with response generation

```
description: This function checks for vacation entitle-ments

target:
  type: function
  name: hr/create_leave_request

response_context:
  - value: totalEntitlement
    description: These are total leaves for an employee
  - value: leftVacation
    description: Total available leaves for booking
```

Supported Message Types

Note

Gen AI based message generation is a continually developing feature and doesn't currently support all aspects of existing messages described in Messages [page 158]. For example, markdown is currently only supported for text-based messages.

Text

Texts are generated to answer questions targeted to a single value or aggregation. The following text attributes are supported:

Attribute	Description
`content`	String content of the text message

Card

Cards are generated to display multiple details of a single entity. The following card attributes are supported:

Attribute	Description
`title`	Descriptive title for the displayed entity
`subtitle`	Descriptive subtitle for the displayed entity
`description`	Description consisting of relevant attributes of the displayed entity
`status`	Status of the displayed entity (e.g. availability)
`statusState`	Status state derived from the semantics of the status of the displayed entity
`elements[*].imageJsonPath`	Base64 encoded image, image URL, or SAP icon
`sections`	Sections are attributes of type text, email, and phone number

Attribute	Description
`elements[*].attributes`	Up to five additional attributes of type text, email, and phone number
`elements[*].urlButtons`	Array of URL buttons allowing navigation based on a URL
`elements[*].navigationButtons`	Array of buttons allowing cross-product navigation

List

Lists are generated to display high level details of multiple entities. The following list attributes are supported:

Attribute	Description
`title`	Title describing all entities displayed
`subtitle`	Subtitle describing all entities displayed
`description`	Description giving some context to the entities displayed
`elements[*].title`	Descriptive title for each displayed entity
`elements[*].subtitle`	Descriptive subtitle for each displayed entity
`elements[*].description`	Description consisting of relevant attributes of each displayed entity
`elements[*].status`	Status of each displayed entity (for example, availability)
`elements[*].statusState`	Status state derived from the semantics of the status of each displayed entity
`elements[*].imageUrl`	Base64 encoded image, image URL, or SAP icon
`elements[*].buttons`	Navigation buttons (with IBN targets including parameters or URL), phone number buttons and postback buttons

3.3.1.2 Role-Based Scenario Access

To filter scenarios based on a condition such as a user role or position, you can configure a visibility condition for that scenario. If the visibility condition is `true`, the scenario is taken into account by Joule during a conversation. If the condition is `false`, the user cannot access the scenario.

Definition of a Visibility Condition

A condition can be defined by specifically stating a semantic object and action for consumption by the SAP Build Work Zone Navigation Service. Refer to the definition below for stating intent-based navigation parameters.

If no explicit visibility condition is provided, the default value is `true`, and the node is always considered.

Attribute	Type	Description	Required	Allowed Values	Since Version
`type`	string	Type of object for which the visibility condition is configured.	false	`ibn_targets` `ias_attributes`	3.14.0
`intents[*].semantic_object`	string	Semantic object for the intent-based navigation target	true	Only alphanumeric characters and underscores are allowed.	-
`intents[*].action`	string	Action for the intent-based navigation target	true	Only alphanumeric characters and underscores are allowed.	-

> **Note**
>
> As of DTA Specification version 3.15.0, both `ibn_targets` and `ias_attributes` types can coexist for scenarios of type `function`.
>
> As of DTA Specification version 3.23.0, attribute `sap_licenses` is allowed within `ias_attributes`.

Example Usage for IBN Targets

The following example shows how visibility conditions can be used within a scenario. These attributes must match the attribute name in the IAS token. The threshold is 50.

> **Sample Code**

```
visibility_condition:
  objects: # only one entry for each type is allowed!
    - type: ibn_targets
      match: ALL
      intents:
        - semantic_object: semantic_object1     # sample semantic object
          action: action1                       # sample action
        - semantic_object: semantic_object2     # sample semantic object
          action: action2                       # sample action
```

The user roles for the current logged in users are dynamically fetched within the initialize hook using an API request action and user information.

`visibility_condition` also supports the `ias_attributes` type. `ias_attributes` restricts the scenarios to be taken into account only for specific attributes. To restrict the scenarios to be taken into account only for specific users or users who are assigned to `user_groups`, or restrict the scenarios based on certain sap_licenses, Joule adds an object of type `ias_attributes` under `visibility_condition`.

Example Usage for IAS Attributes

The following example shows how visibility conditions can be used within a scenario. These attributes must match the attribute name in the IAS token. If the attribute value is `sap_licenses`, the threshold is 10; otherwise, it is 50.

count for values.

Sample Code

```
visibility_condition:
  objects: # only one entry for each type is allowed!
    - type: ias_attributes
      attributes:
        - attribute: groups  # Sample group matching to attribute name in IAS
token
          match: "< ANY | ALL>",
          values:
            - group1           # sample group
            - group2           # sample group
        - attribute: user_uuid
          match: "< ANY | ALL>",
          values:
            - user_uuid1              # sample user_id
            - user_uuid2              # sample user_id
        - attribute: attribute3
          match: "< ANY | ALL>",
          values:
            - val1                  # sample attribute3
            - val2                  # sample attribute3
```

CDM-Managed Intent-Based Navigation (IBN) Targets

You can specify intent-based navigation (IBN) targets in visibility conditions that are evaluated by user context via the SAP Build Work Zone navigation service. To use the navigation service, you need to configure a destination with the name **`NavigationService`** for your Joule subscription. For more information, see Configure Cross-Product Navigation [page 172].

Note

If the destination is missing or wrongly configured (or when the service call fails), no navigation targets are displayed, regardless of the user context. In such cases, back-end calls might fail due to insufficient user authorization.

Character Limits for Visibility Conditions

- Maximum number of characters: 1000
- The following escape sequences are not allowed:
  - **\r**: Carriage return

    - **\n**: Newline
    - **\t**:Horizontal tab
- YAML block scalars are allowed.

3.3.1.3 Capability Context Variables and Static Parameters

Capability context variables are used to maintain a technical context during an active conversation with the user. when the conversation expires, the context is cleared.

The capability context is shared with all Joule functions that belong to the same capability. Access to the context variables of other capabilities is not supported. Furthermore, the capability context for Joule functions is only consumable in Joule scenarios (read and write).

Define the Capability Context Model

To define the model of the capability context, you need to create a new file called `capability_context.yaml` in your capability. In this file, you must define the names of all context variables that are supposed to be available in the capability.

Capability folder structure

```
sample_capability
├── functions
├── scenarios
├── capability_context.yaml <-- new file
└── capability.sapdas.yaml
```

Capability_context.yaml

```
variables:
  - name: <capability context variable name>
  - ...
```

Set the Values of the Capability Context Variables

You can use two approaches to set the value of the capability context variable:

1. Directly use the result variables of the Joule function. Use the `$target_result.<result_key>` notation to achieve this.
2. Providing static strings by omitting the `$target_result` prefix.

> **⚠ Restriction**
>
> Scripting is not permitted in the `capability_context` section of the scenario definition.
>
> Access to Joule function results is restricted to root-level keys only; path-based access is not possible.

Scenario definition to write Joule functions results into the capability context

```
capability_context:
  - name: station_id  <-- name of the capability context variable
    value: $target_result.id # <-- $target_result indicates that the station_id context variable will have the value of the id result key of the Joule Function.
  - name: last_executed_scenario
    value: search_trainstation # <-- static string value, not coming from the result of the function
```

> **Note**
>
> If you want to use the result variables of the Joule Function to set the capability context variable value using the `$target_result.<key>` notation, you should ensure that key is defined in the result set of the respective Joule function.

Pass Variables from the Capability Context As Joule Function Parameters

In addition to slots that are filled by Joule during conversation with the user, Joule functions support also static parameter injection that can be either:

1. Values from the capability context by using the `$capability_context` prefix.
2. Static string values

To pass either capability context variables or static strings as parameters to a dialog function, the target section of the respective scenario definition can be extended as in the example below.

> **Restriction**
>
> In both cases, scripting is not allowed within the parameters section of the scenario definition.
>
> Access to capability context variables is restricted to root-level keys only; path-based access is not possible.

Using capability context variables as Joule function parameters

```
target:
 type: function
 name: train/search_station
 parameters:   # <-- Define parameters that should be passed by executing the function
   - name: from_station_id
     value: $capability_context.from_station_id  # <-- $capability_context
   - name: to_station_id
     value: Wiesloch-Walldorf # static string
```

> **Restriction**
>
> The declared parameter names must also be defined in the respective dialog function as **optional** input parameters. You need to ensure null-safety yourself.

> **Note**
>
> Dialog function parameters can be either filled via slots or via target parameters. Therefore, every dialog function parameter can only be referenced either in the slots section or the `target.parameters` section of the scenario file. It is not possible to reference it in both.

Full Example

The following example is passing static parameters to a Joule function and the results of the executed Joule function are written into the capability context.

```
description: Search a train station using the provided information

slots:
 - name: date
   description: Date of the flight
 - ...

target:
 type: function
name: train/search_station
parameters:   # <-- Define parameters that should be passed by executing the function
    - name: from_station_id
      value: $capability_context.from_station_id  # <-- $capability_context No scripting allowed
    - name: to_station_id
      value: Wiesloch-Walldorf # static string

response_context:
  - description: Train Station Name
    value: id # dialog function result variable name

capability_context:
  - name: station_id
    value: $target_result.id # <-- $target_result mandatory and at the moment only possible key. No SpEL
  - name: station_capacity
    value: $target_result.capacity
  - name: last_executed_scenario
    value: search_trainstation # <-- static string value, not coming from the result of the function
```

Capability context definition

```
variables:
    - name: from_station_id
    - name: station_id
    - name: station_capacity
    - name: last_executed_scenario
```

3.3.1.4 Scenario Dependencies

Scenario dependencies in Joule allow developers to model complex business interactions where the output of one scenario can serve as the input to another. This functionality supports the creation of intricate business

workflows without the need for extensive backend logic, as dependencies manage the flow of data between scenarios seamlessly.

Definitions

Types of Results

Dependencies can yield three types of results:

1. **Single Result**: A unique value that directly fills a slot.
2. **Multiple Results**: Several values from which the user must choose to fill a slot.
3. **No Result**: Situations where the dependency does not return any usable output, prompting a user request.

Creating Scenario Dependencies

Design Time Artifact Modifications

You need to declare dependencies within the sources block of the scenario's DTA (Design Time Artifact) file, specifying up to five dependencies per slot. Below is a snippet illustrating how to define dependencies for a purchase requisition id slot:

Sample Code

```
slots:
  - name: pr_id
    description: ID of the purchase requisition the new purchase order is
based on.
    sources:
      - type: scenario
        name: purchase/get_purchase_requisition
        key: $target_result.id
      - type: scenario
        name: purchase/create_purchase_requisition
        key: $target_result.id
```

Execution Groups (Optional)

For scenarios where multiple slots are filled through a single execution, you can group these slots using the execution_group parameter:

Sample Code

```
slots:
  - name: departure_latitude
    description: Departure latitude for the flight
    sources:
      - type: scenario
        name: get_location
        execution_group: departure
        key: $target_result.latitude
  - name: departure_longitude
    description: Departure longitude for the flight
    sources:
```

```
      - type: scenario
        name: get_location
        execution_group: departure
        key: $target_result.longitude
  - name: destination_latitude
    description: Destination latitude for the flight
    sources:
      - type: scenario
        name: get_location
        execution_group: destination
        key: $target_result.latitude
  - name: destination_longitude
    description: Destination longitude for the flight
    sources:
      - type: scenario
        name: get_location
        execution_group: destination
        key: $target_result.longitude
```

Referencing Dependency Results (Result Mapping)

Dependency results can be referenced using the key property within the slot sources. The key property refers to a result variable of the target (usually the dialog function) of the source scenario.

The key must be part of the *response_context* of the source scenario.

Sample Code

```
sources:
  - type: scenario
    name: pur/get_purchase_requisition
    key: $target_result.id
```

Constraints

- Only non-cyclic dependency trees are supported.
- Each slot within a scenario can define multiple sources, but only one will be executed based on the scenario logic.
- Dependencies are managed by the system and do not adhere to a capability developer-defined sequence.
- Dependencies can only be defined on mandatory slots.
- Each scenario supports up to 5 dependencies per slot.
- Scenario dependencies can only be defined within a capability. Cross-capability dependencies are not supported.
- Only scenarios with response generation can be used as dependencies.

3.3.2 Implementing Functions

A dialog defines the flow of the conversations in the form of a logic function. It matches user intents determined using natural language processing based on the dataset built for your assistant and uses the context of the dialog with the user and returns the defined responses.

Dialog Functions

Defined under the *functions* sub-folder of a capability. For more information, see Dialog Functions [page 58].

Central Assistant Functionality

An assembled assistant comes with centrally provided functionality that includes:

1. Welcome functionality
   With the start of a conversation, a dedicated welcome message will be displayed to the user including possible triggers a user may utter to start a capability use case.
2. Fallback functionality
   If the assistant does not recognize the given input or can't match the user utterance to a suitable functionality, a fallback functionality will be triggered where the assistant displays a specific message including possible trigger utterances to start a capability use case.
3. Initialize functionality
   Initialize a capability using the initialize conversation feature. For more information, see Initialize Conversation [page 78].

3.3.2.1 Dialog Functions

Multiple dialog functions can be defined in one assistant and referenced by various function scenarios. Functions can be nested up to 3 levels of subfolders.

Dialog functions are executable entities defined by developers. These entities contain descriptions of the actions related to API calls, data processing, and message generation that the Joule runtime performs.

Specification for Function Definition

> **Note**
>
> - The nesting of dialog functions, that is, the call from one function to another or recursive calls is limited to 10.
> - You can use up to 64 alphanumeric characters for each parameter variable name.

A function can be defined using the following attributes:

Attribute	Type	Description	Required	Rules
`parameters`	array	Input to the dialog function	false	See Parameters [page 77].

Attribute	Type	Description	Required	Rules
`action_groups`	array	Group of actions to execute	true	See Action Group [page 59].
`result`	object	output of a dialog function execution	false	See Result [page 77]

Parameters

Attribute	Type	Description	Required	Rules
`name`	string	Name of the parameter available in the dialog function context	true	
`optional`	boolean	Indicates whether the parameter is optional	false	Default value is `false`

Action Groups

Attribute	Type	Description	Required	Rules
`actions`	array	Actions to execute	true	At least one action
`condition`	string	Condition for the action group	false	

3.3.2.1.1 Action Group

An action group contains actions in an ordered list while each action defines a specific instruction to execute.

Instructions in a dialog function can be bundled in one or multiple action groups.

Conditions can be defined at the action group level using the **SpEL** syntax. If the condition for an action group is evaluated to `true` at runtime, the actions defined in the action group are executed. Action groups that don't contain a condition are always executed. Action groups and the actions in them are always executed in the same order as they are defined by the developer.

{·} Code Syntax

Action Groups in Dialog Function Definition

```
action_groups:
  - condition: <spel group condition> #optional
    actions:
      <dialog function actions>
```

Action

An action is an element in an action group, defining an instruction. The following table describes the different action types:

Action Type	Description	Since Schema Version
API Request Action [page 61]	This type of action performs an API call and gathers the result into the defined local-context variable.	-
Set Variables Action [page 61]	This type of action will perform modifications of local context variables and / or parameters by using **SpEL** or **Handlebars** syntax.	-
Message Action [page 62]	This type of action generates a UI message that is returned by the parent dialog function. There are different types of messages that can be generated through the design time specification.	-
Dialog Function Action [page 62]	This type of action performs a dialog function call.	-
OData Batch Action [page 62]	This type of action performs an odata-batch call and gathers all results into a defined local-context variable.	-
Status Update Action [page 63]	This type of action provides a status update message to the user during dialog function execution.	3.6.0
Streamed Message Action [page 64]	This type of action provides a streamable message using HTTP SSE APIs.	3.11.0
File Upload Action [page 65]	This type of action allows users to upload files.	-
Frontend Action [page 67]	This type of action interacts with applications running next to Joule via the front end.	3.24.0
User Confirmation Action [page 66]	This type of action allows users of Joule to review and confirm their API requests before they are sent.	-
Agent Request Action [page 69]	This type of action is used to call an agent.	3.18.0

API Request Action

Code Syntax

API Request Action

```
- type: api-request
  method: <GET | POST | OPTIONS | DELETE | PUT | PATCH>
  async: <TRUE | FALSE > #optional, available from 3.18.0 version (defaults
to false)
  system_alias: <system alias name defined in capability.sapdas.yaml>
  path: <url path> #optional
  headers: #optional
     <header key>: <header value>
  timeout: 45 #optional
  body: <body value> #optional, must be a string if a header 'content-type'
is defined and eq 'application/octet-stream', disallowed if 'content-type' is
'multipart/form-data'
  form_body: #allowed if `content-type` is 'multipart/form-data'
    - type: text
      key: mykey
      headers: #optional, such as content-type and content-disposition
         key: value
      value: <? result.body.file.id ?> # file id generated in previous
response should be provided here.
  result_variable: <result context variable> #optional
```

In an API request action, scripting can only be used with the **SpEL** syntax and is limited to the path, headers, and body properties.

Timeout values are not honored for asynchronous API requests because the maximum allowed timeout for asynchronous requests is 5 minutes. For more information, see Asynchronous API Request [page 73].

Note

The size of fetched data must not exceed 1 MB.

The API request action can be used to query customer-specific documents via Document Grounding in Joule. This allows SAP Built Agents to retrieve relevant document chunks based on metadata filters for enhanced context during conversations.

For information about creating a *SharePoint* site, configuring the **Document Grounding** instance and pipeline, and creating a respective API request action, see Using an API Request Action for Document Retrieval with Joule [page 70].

Set Variables Action

This type of action is used to set variables in the local context of the dialog function while already existing variables are overwritten.

Code Syntax

Set Variables Action

```
- type: set-variables
  scripting_type: <spel (default) | handlebars> #optional
  variables:
    - name: <key 1>
      value: <value 1>
    - name: <key 2>
      value: <value 2>
```

```
    ...
    - name: <key n>
      value: <value n>
```

The variable values can be dynamically set using either **SpEL** (default) or **handlebars** scripting syntax. If **handlebars** is used to evaluate the variable value, the `scripting_type` must be set to `handlebars`; otherwise, **SpEL** is used as default.

Message Action

Message actions allow returning a message to the bot user. The messages are shown in the same order as the message actions are defined in the dialog function.

Code Syntax

Message Action

```
- type: message
  scripting_type: <spel (default) | handlebars> #optional
  message:
    type: text
    content: Weather in {{location}} is
{{weatherServiceResponse.response.body.current}}
```

In addition to text, the message types card, buttons, quick replies, carousel, list, media, client_data, ui_integration_card, illustrated_message, and likert_scale can be used. For more information on these message types, see Messages [page 158].

The message content can be dynamically created using either **SpEL** (default) or **handlebars** scripting syntax. If you want to use **handlebars** to evaluate the message content, the `scripting_type` must be set to `handlebars` otherwise **SpEL** is used as default.

Dialog Function Action

The dialog function action is used to execute another dialog function. The result of the called dialog function is then stored under the specified `result_variable` in the local context of the calling dialog function.

Code Syntax

Dialog Function Action

```
- type: dialog-function
  name: <dialog function name (full path)>
  parameters: #optional
    - name: <name>
      value: <value>
  result_variable: <optional result context variable> #optional
```

Note

In dialog function actions, scripting can only be used with **SpEL** syntax and is limited to the dialog function parameters.

OData Batch Action

The OData batch action is used to make an OData batch call to an external API.

Code Syntax

OData Batch Action

```
- type: odata-batch
  system_alias: <system alias name defined in capability.sapdas.yaml>
  path: <base url path> #optional
  headers: #optional
    <header key>: <header value>
  timeout: 45 #optional, available from 1.5.0 version
  requests:
    - method <GET | POST | DELETE | PUT | PATCH | MERGE>
      path: <relative path>
      headers: #optional
        <header key>: <header value>,
        Content-ID: <header value> #optional
      body: <body value> #optional
      result_key: <result key name>
    - ...
  result_variable: <result context variable> #optional
```

All single requests that should belong to the OData batch can be defined as a list of entries under the `requests` property.

In OData batch actions, scripting can be used only with **SpEL** syntax and is limited to the following properties:

- `path`
- `headers`
- `request.path`
- `request.headers`
- `request.body`

The result of each single request within one batch will be stored in the provided `result_key`.

Note

When you configuring OData Batch requests for SAP S/4HANA integration, you must explicitly define the `Accept` header. Relying on the default `multipart/mixed` value may cause runtime errors if the back end returns an error response.

To ensure correct processing, set the header as follows:

```
- type: odata-batch
  system_alias: <system alias name defined in capability.sapdas.yaml>
  path: <your url path>
  headers:
    Accept: multipart/mixed, application/json
  requests: ....
```

Status Update Action

The status update action is used to define a status update message and provide updates to the user for long running actions in a dialog function.

```
- type: status-update
  scripting_type: < spel | handlebars> #optional
  message: <text message, may include scripting>
```

The message for the status update can be dynamically set using either **SpEL** (default) or **handlebars** scripting syntax. If **handlebars** is used to evaluate the variable value, the `scripting_type` parameter must be set to `handlebars` otherwise **SpEL** is used as default.

Streamed Message Action

The streamed message action is similar to the api-request action, where a capability developer defines a request to a service that supports Server-Sent Event (SSE). The API definition is configured in the BTP Destination service similar to api-request action.

Client side streaming needs to be enabled to see the end effect. If the streaming is disabled, it is displayed as a standard message. In case of execution errors no message is displayed and the capability developer is required to handle it gracefully in the following actions.

Each streamed chunk is referenced by Joule, using the reference path in the streamed response event of the API.

A final aggregated message is automatically created by Joule for the streaming, so capability developers do not need to create additional messages from the streaming response.

Nevertheless the complete streaming response is available as part of the result variable in the corresponding dialog function.

> **Code Syntax**
>
> API Request Action
>
> ```
> - type: streamed-message
>  method: <GET | POST | OPTIONS | DELETE | PUT | PATCH>
>  system_alias: <system alias name defined in capability.sapdas.yaml>
>  path: <url path> #optional
>  headers: #optional
>  <header key>: <header value>
>  Accept: "text/event-stream" #SSE required value
>  Connection: "keep-alive" #SSE required value
>  Cache-Control: "no-cache" #SSE required value
>  body: <body value> #optional
>  response_chunk_ref: <path to chunk in streamed message request>
>  result_variable: <result context variable> #optional
> ```

The headers *Accept*, *Connection* and *Cache-Control* are mandatory for the *streamed message* action type. If these headers are not provided, compiler will throw a validation warning and add them by default with the values mentioned above. If these headers are a different value, compiler will issue an error with suggestions to correct the values.

> **Note**
>
> The streamed message action supports Server-Sent Events in JSON and Plain Text data formats.

JSON

In the case of JSON data format, *response_chunk_ref* should be defined based on the structure of the JSON response from the target API (for example, *response_chunk_ref = $.delta.content*):

```
data: {"delta":{"content":"Test"}}

data: {"delta":{"content":" text"}}
```

```
---------------------------------------

Result:
Test text
```

Plain Text

In the case of plain text data format, *response_chunk_ref* should point to the start of the received data value from the target API (for example, *response_chunk_ref* = *$*):

```
data: Test

data:  text

----------------------

Result:
Test text
```

In order to receive a properly formatted plain text response with all **SPACE** symbols, the target API must send Server-Sent-Events using one of the following options:

- Send the data field with 2 leading spaces in the value:

  ```
  data:-Test

  data:--text

  ----------------------

  Result:
  Test-text
  ```

- Send the data field with a space at the end of the value:

  ```
  data:-Test-
  data:-text
  ----------------------
  Result:
  Test-text
  ```

Otherwise, the result will be the following with the first space character being ignored:

```
data:-Test
data:-text
----------------------
Result:
Testtext
```

File Upload Action

The file upload action allows users to choose a file to upload. After selection, the file can be either uploaded or the upload can be canceled.

Source Code

```
- type: file-upload
  scripting_type: "spel|handlebars"
  content:
    message: <message>
    on_cancel:
      title: <title>
```

```
      message: <message>
      button_labels:
        cancel: "yes"
        continue: "no"
  allowed_file_types:
    - pdf
    - docx
    - jpg
  max_file_size: <max_file_size>
  max_files_allowed: <max_files_allowed>
  result_variable: <result context variable> #optional
```

- **message**: The message that will be displayed to the user regarding the file upload action.
- **on_cancel**:
  - **title**: The title of the cancel confirmation dialog.
  - **message**: The confirmation message displayed when the user attempts to cancel the file upload.
  - **button_labels**:
    - **cancel**: The label for the button that confirms the cancellation (**yes** in this case).
    - **continue**: The label for the button that continues with the file upload process (**no** in this case).
- **allowed_file_types**: A list of file types that are permitted for upload. The following file name extensions are supported:
  - `.bmp`
  - `.csv`
  - `.doc`
  - `.docx`
  - `.gif`
  - `.htm`
  - `.html`
  - `.jpeg`
  - `.jpg`
  - `.mle`
  - `.pdf`
  - `.png`
  - `.ppt`
  - `.pptx`
  - `.rtf`
  - `.txt`
  - `.xls`
  - `.xlsx`
- **max_file_size**: The maximum size allowed for individual files being uploaded is 10 MB.
- **max_files_allowed**: The maximum number of files that can be uploaded in a single operation is 10.

User Confirmation Action

The user confirmation action allows users of Joule to review and confirm their API requests before they are sent. A confirmation step can be used to get user approval before continuing the dialog function execution.

Sample Code

User Confirmation

```
- type: user-confirmation
  scripting_type: "spel|handlebars"
  content:
    title: "<title text>"
    subtitle: "<subtitle text>"
    button_labels:
      confirm: "I confirm"
      cancel: "Cancel"
  confirmation_variable: "<variable name>"
```

Mandatory fields:

- Title
- Subtitle
- Confirm and cancel button labels

Scripting type is optional (default SpEL).

Scripting is enabled for *title*, *subtitle*, and both button labels.

Frontend Action

A frontend action is used to interact with the application running next to Joule using the frontend. This is useful for scenarios where:

- A user wants to interact with the current application like "summarize the current screen" or "help me with the error".
- An application does not offer specific backend APIs.
- After an API request action, to reflect the current state with the help of a refresh frontend action.

A frontend action behaves similarly to an API request action, allowing developers to pass parameters to the frontend action and receive data via the `result_variable` from the frontend, which can be used in succeeding actions.

Note

For optimal user experience, frontend actions should execute as quickly as typical UI interactions in the application. If the application fails to respond within the timeout period configured in the Joule Web Client, an error will be triggered. Timeout duration is 15 seconds.

Sample Code

Frontend Action

```
- type: frontend-action
  name: <technical_frontend_action_name>
  optional: <TRUE | FALSE>  # Optional
  parameters:  # Optional
    - name: <parameter_name_1>
      value: <parameter_value_1>
    - name: <parameter_name_2>
      value: <parameter_value_2>
      ...
    - name: <parameter_name_n>
```

```
      value: <parameter_value_n>
  result_variable: <result_context_variable> # Optional
```

- **name:** technical name of the frontend action to be called. Naming convention is <lob_namespace>.<application_name>.<frontend_action_name>.<version>
- **optional:**Default is false and therefore mandatory. The scenario will not be executed if the frontend action is not available for the request. Can be set to `optional = true` for non essential frontend actions like refresh.
- **parameters:**list of parameter key/value pairs which are passed to the frontend action
- **result_variable:** name of the variable where the result of the frontend action call is stored
  - **status:**of the frontend action call which can be either
    - **success:**frontend action call was successful
    - **error:** only for technical issues during execution. Business errors result in a successful execution, and business error information must be part of the payload.
    - **skipped:**if the frontend action is optional and not part of current list of frontend actions. If the status is skipped the result_variable does not have a payload. Optional frontend actions can be implemented as fire and forget events by omitting the result_variable.
  - **content:** payload of the frontend action result

Sample Code

Frontend Action Example

```
action_groups:
  - actions:
    - type: frontend-action
      name: sap.ui.summarize.v1
      result_variable: frontend_action_result
  - condition: frontend_action_result.status == "success"
    actions:
      - type: set-variables
        variables:
          - name: success_payloadWhen a scenario includes nested dialog
functions (for example, Scenario A calls
                    Dialog Function X, wh
    actions:
      - type: set-variables
        variabl
                    frontend action), only mandatory frontend act
            value: frontend_action_result.content
result:
  status: <? frontend_action_result.status ?>the
                    scenario's `visibility_condition`.
action_groups:
  - actions:
  content: <? frontend_action_result.content ?>summarize.v1
      result_variable: frontend_action_result
  - condition: frontend_action_result.status == "success"
```

For frontend actions invoked deeper in the call hierarchy (i.e., within nested dialog functions), the scenario developer must explicitly declare these dependencies using the `required_frontend_actions` field in the scenario definition. These declared frontend actions are then appended to the `visibility_condition` during compilation, ensuring that relevant required frontend actions—whether from top-level or nested dialog functions—are considered for scenario preselection.

Sample Code

Sample Code for nested Dialog function scenario

```
description: This functionality can be used to answer greetings.
required_frontend_actions:
  - com.sap.ui.refresh.v1
  - com.sap.ui.setTheme.v2
  - com.sap.ui.changeCountryCode.v2 target:
type: function
   name: module/another_function
   parameters:
     - name: location
       value: "Berlin"
```

This explicitly declares that these three frontend actions are required for the scenario to be considered during scenario preselection.

This means:

- These frontend actions may be invoked from nested dialog functions (i.e., not directly from the scenario's top-level dialog function).
- The scenario developer is responsible for listing all such required and relevant frontend actions, as they are not automatically detected by the compiler if they are only referenced in nested dialog functions.
- During compilation, these declared frontend actions are appended to the scenario's `visibility_condition`. At runtime, scenario preselection will only include this scenario if all three frontend actions are available in the current frontend context.
- If any of these actions are missing from the available frontend actions at runtime, the scenario will be filtered out and not offered to the user.

In summary, listing frontend actions under `required_frontend_actions` ensures that all necessary frontend actions called via nested dialog functions—are considered for scenario visibility and preselection.

Frontend actions can only be successfully executed if the application running next to Joule offers the corresponding frontend actions:

- For each request Joule queries the application for the list of currently available frontend actions. For the Joule Web Client this is done using the `getFrontendActions()` callback.
- This list is then used to filter the available scenarios based on the list of available frontend actions and the optional flag.

Agent Request Action

Agent request actions are used to define requests that call agents as part of a dialog function. For more information, see Content-Based Agents [page 190].

Related Information

Asynchronous API Request [page 73]

3.3.2.1.1.1 Using an API Request Action for Document Retrieval with Joule

An API request action, that is, an action of type `api-request`, can be used to query customer-specific documents via document grounding in Joule. This allows agents to retrieve relevant document chunks based on metadata filters and thereby provide enhanced context during conversations.

Set Up Document Grounding for Retrieving Documents via API Request Action

> **Note**
>
> If the document grounding instance has already been created and provided, skip steps 1 and 2.

1. Create a SharePoint site.
   If a SharePoint site does not already exist, create one. For more information, see Set Up Document Grounding.
2. Create a document grounding instance and pipeline.
   Configure your own metadata that will be used in the API request action (in the request body). For information about setting up a document grounding instance and pipeline, see Set Up Document Grounding.
3. Create an API request action.
   Use the template below to create your API request action that leverages the configured metadata.

   > **Sample Code**
   >
   > API request action with filter for document grounding

```
- type: api-request
  method: POST
  headers:
    content-type: application/json
  system_alias: JOULE_GLOBAL_DOC_GROUNDING // It's a hardcoded value. No
need to create destination for it at all. Just define system alias with
this as destination_name in capability.sapdas.yaml
  path: "/retrieval/search"
  body: >
    {
      "query": "<? query ?>",  // example: "What about Global Health
Safety?"
      "filters": [
            {
               "id": <? filter_id ?>, // mandatory, but could be any
random string. It's needed for the consumer to identify the results.
example: "dummy"
               "searchConfiguration": {
                    "maxChunkCount": 1
                },
               "dataRepositories": [
                  "*"  // It can be put as a wildcard
               ],
               "dataRepositoryType": "string", // can be type can be agent
so SPR avoids. example: "vector"
```

```
                "dataRepositoryMetadata": [{key="agent-name",
value="accurals-agent"}], // example: [{"key":"pipeline/e2e_test","value":
["agent"]}] - Make sure to insert the metadata fields that were configured

           //when creating the pipeline. These fields will be used as
filters for retrieving relevant document chunks.
                "documentMetadata": [],
                "chunkMetadata": []
             }
    }
  result_variable: result_retrieval_search
```

Relevant documents can now be retrieved via document grounding using Joule.

3.3.2.1.1.2 Attachment Handling in Joule

Using API Request Action for Uploading/Downloading file to/from Attachment Service with Joule

Application/octet-stream flow

Application/octet-stream flow is used when you need to send one file to the external service in a single request. In the first api-request you can download the file from the external service, upload it to the attachment service, and then upload the file to the external service via **octet-stream**.

Create an api-request action (**octet-stream**) flow.

Sample Code

API-request actions for octet-stream flow

```
action_groups:
  - actions:
      - type: api-request
        method: GET
        headers:
          Accept: image/jpeg #To invoke attachment service flow Accept header
need to be specified and the value should be equal with value of Content-Type
header from binary response (file that should be downloaded)
        system_alias: JpgAlias
        path: /images/link-icon-png/link-icon-png-12.jpg
        result_variable: file_metadata

      - type: api-request
        method: POST
        headers:
          Content-Type: application/octet-stream #To invoke the octet-stream
flow
          Accept: application/json
        system_alias: WebHookAlias
        path: /574eeaad-584f-4afc-9c31-229e1017947e #Path to the external
service where we want to upload the file
        body: <? file_metadata.body.file.id ?> #In case of octet-stream body
will be file-reference id
        result_variable: responseData
```

Multipart/form-body flow

Multipart/form-body flow is used when you need to upload multiple files to the external service in a single request. In the first api-request you can download a file from the external service, upload it to the attachment service, and then upload the file to the external service via **form-body** flow.

Create an api-request action (**form-body**) flow.

Sample Code

API-request actions for form-body flow

```
action_groups:
  - actions:
      - type: api-request
        method: GET
        headers:
          Accept: image/jpeg #To invoke attachment service flow Accept header
need to be specified and the value should be equal with value of Content-Type
header from binary response (file that should be downloaded)
        system_alias: JpgAlias
        path: /images/link-icon-png/link-icon-png-12.jpg
        result_variable: file_metadata

      - type: api-request
        method: POST
        headers:
          Content-Type: multipart/form-body #To invoke the form-body flow
          Accept: application/json
        system_alias: WebHookAlias
        path: /d50ddc33-2919-4531-9b5c-0622bf4eff53
        form_body:
          - type: file
            key: myFile
            headers: # further headers to be provided for each form element
              content-type: application/pdf
            value: <? file_metadata.body.file.id ?> #File id generated in
previous response should be provided here.
          - type: text
            key: myTextKey
            value: some test string
        result_variable: fileResponse
```

Note

To invoke the new Attachment Service flow you must set the *Accept* header to the same value as the *Content-Type* header in the binary download response.

Files Supported by the Attachment Service

- The maximum accepted file size is 10MB.
- Allowed file type extensions: bmp, csv, doc, docx, gif, htm, html, jpeg, jpg, mle, pdf, png, ppt, pptx, rtf, txt, xls, xlsx.

Attachment upload

To upload attachments to a target destination you can use the action type *file-upload* in your action group. For more information, see Action Group [page 59].

With this action the end user is requested to upload a file within your dialog-function execution. You should define a message that is displayed when the user is requested to upload a file from where the end user can decide to upload a file or cancel the process. Additionally as part of the file-upload action you can define the number of files, the size of the file, and file type depending on what you backend system expects.

- The current maximum number of files is 10.
- The maximum size per file is 10MB.
- The currently supported file types are: bmp, csv, doc, docx, gif, htm, html, jpeg, jpg, mle, pdf, png, ppt, pptx, rtf, txt, xls, xlsx.

3.3.2.1.1.3 Asynchronous API Request

Important Considerations

> ⓘ Note
>
> This functionality requires additional integration effort. Please assess integration readiness before using this feature.

Platform Requirements:

- Bi-directional communication (callback support) is currently only supported for IAS-based integrations.
- Non-IAS-based integrations are not yet compatible with this asynchronous callback model.

For products already integrated with Joule and intending to adopt asynchronous APIs:

- The backend systems must be updated to support callback requests to Joule.
- Joule-side configurations (security/trust setup) must also be updated to accept and process these callbacks.
- This applies even if the system is already connected to Joule.

Technical Prerequisites

To enable asynchronous API requests:

- IAS-enabled subaccount: The BTP subaccount must be integrated with SAP Identity Authentication Service (IAS).
- For callback support your back end has to:
  - Perform the actual asynchronous task.
  - Send a callback to Joule once complete.
- A trust configuration must be set up between the back-end system and Joule, which requires implementation effort with regard to Joule integration configuration.

Implementation Guide

Step 1: Define the Asynchronous API Request

Use the normal api-request action but add `async: true`

Sample Code

Asynchronous API Request

```
action_groups:
  - actions:
      - type: api-request
        method: POST
        async: true               # This enables async behavior
        system_alias: Orbit
        path: "/thisIsMyEndpoint"
        headers:
          someHeaderKey: "someHeaderValue"
        body: {
          "someBodyKey": "someBodyValue"
        }
        result_variable: api_call_result
```

Step 2: Handle the Initial Request

Your endpoint receives the normal request plus the following headers, that will be **automatically** added by Joule. The added headers provide the needed information to execute the callback later.

- `x-correlationId`
- `x-callback-target`

It is necessary that the called endpoint immediately responds with a **non-4xx/5xx HTTP status**. This response serves as a necessary acknowledgment and without it, Joule will assume an error happened and will show a processing exception. After your endpoint responded to the initial request, the dialog function execution will be paused.

Step 3: Implement the Callback

After completing your task, send a callback with the following structure:

URL	<value from x-callback-target>
HTTP Method	POST
Headers	• content type: <defaults to application/json> • x-correlationId: <value from x-correlationId header> // Cannot be empty • x-joule-callback-request-type: result //The value has to be "result"
Body	The request body can contain any valid payload. This endpoint doesn't enforce a specific structure; the payload will be forwarded as-is.

The execution will be paused for a **maximum of 5 minutes.** If a callback is not received within this time frame, Joule will bypass the paused execution state and any callback received thereafter will be disregarded.

Step 4: Use the results

The callback payload becomes available in your `result_variable`:

Sample Code

Use result of asynchronous api request

```
action_groups:
  - actions:
      - type: message
        message:
          type: text
          content: "Result: <? api_call_result.body ?>"
```

Complete Example

Sample Code

complete example of async api request and following message action

```
action_groups:
  - actions:
      - type: api-request
        method: POST
        async: true
        system_alias: Orbit
        path: "/thisIsMyEndpoint"
        headers:
          someHeaderKey: "someHeaderValue"
        body: {
          "someBodyKey": "someBodyValue"
        }
        result_variable: api_call_result
      - type: message
        message:
          type: text
          content: "Result: <? api_call_result.body ?>"
```

Response Codes

When your backend makes the callback, Joule responds with:

- **200 OK**: Callback was successfully executed
- **400 Bad Request**: Callback doesn't meet API specification
- **404 Not Found**: No matching paused dialog found

3.3.2.1.1.4 Configure API Request Timeout

When an API call is triggered during runtime, it is expected to return the response within a certain time period. You can set the HTTP request timeout period based on your business needs or scenarios.

Example of Timeout Configuration

The timeout feature is an optional attribute for an action group and for OData batch actions. The minimum value for timeout is one second and you can set the maximum value up to 45 seconds. If the defined timeout exceeds the maximum limit, the system defaults to the highest allowable timeout.

If the timeout is not specified in the action, a 15-second timeout will be applicable for web requests.

> **Note**
>
> The timeout can be configured for API calls in the BTP canary landscape only. In live landscape, the default timeout is 15 seconds.

> **Sample Code**
>
> OData-Batch

```
- type: odata-batch # only spel can be used in odata-batch
  system_alias: <System Alias Name>
  path: <base url path>
  headers:
    <header key>: <header value>
  timeout: <integer in range 1 - 45>  #optional
  requests:
    - method <GET | POST | DELETE | PUT | PATCH>
      path: <relative url path>
      headers:
        <header key>: <header value>
      body: <body value>
      result_key: <result key name>
  result_variable: <result context variable>
```

> **Sample Code**
>
> API Request

```
- type: api-request # only spel can be used in api-requests
  method: <GET | POST | OPTIONS | DELETE | PUT | PATCH>
  system_alias: <System Alias Name>
  path: <url path>
  headers:
    <header key>: <header value>
  timeout: <integer between 1 - 45 seconds>  #optional
  body: <body value>
  result_variable: <result context variable>
```

3.3.2.1.2 Parameters

A dialog function can be described as an in and out black box. The input for the dialog function execution is provided via parameters.

Each parameter is defined by a variable name and a flag indicating if the parameter is optional or mandatory. In case a mandatory parameter is not passed in a dialog function call, a notification is received during compilation. Optional parameters that are not set are mapped to null.

Code Syntax

```
parameters: #optional
  - name: <parameter name>
    optional: <true | false>
```

Parameters are loosely typed following the YAML specification. The supported data types are string, boolean, number, array, object, and null.

Sample Code

Call Sample Function with Various Parameters

```
- type: dialog-function
  name: sample-function-various-parameters
  parameters:
    - name: stringParam
      value: "some string"
    - name: numberParam
      value: 10
    - name: booleanParam
      value: true
    - name: objectParam
      value: {"key":"value"}
    - name: arrayParam
      value: ["element1","element2"]
  result_variable: various_parameters
```

3.3.2.1.3 Result

The result of a dialog function is defined as the output of a dialog function execution and consists of an arbitrary number of result variables.

The result variables can then be used for further processing by the dialog function caller under the respective result variable namespace that was passed to the dialog function. While defining the result, you can use SpEL to map dialog function internal information to result variables.

Code Syntax

Result in Dialog Function Definition

```
result: #optional
  <key>: <value>
```

The following example shows a dialog function that fetches weather data from an API and maps the response data to the result variables:

Sample Code

Dialog Function Example for Result Mapping

```
parameters:
  - name: weather_location
action_groups:
  - actions:
      - type: api-request
        method: GET
        system_alias: WeatherService
        path: /wx/forecast/daily/3day?placeid=<? weather_location.placeid ?
>&units=m&language=en-US&format=json&apiKey=<api key>
        result_variable: weather_result
result:
  success: <? weather_result.status_code == 200 ?>
  city: <? weather_location.city ?>
  minTemp: <? weather_result.body.calendarDayTemperatureMin[0] ?>
  maxTemp: <? weather_result.body.calendarDayTemperatureMax[0] ?>
  dayOfWeek: <? weather_result.body.dayOfWeek[0] ?>
  narrative: <? weather_result.body.narrative[0] ?>
```

Note

- Initialize.yaml does not need to be explicitly referenced, it is executed automatically upon the start of conversation.
- You cannot use parameters, messages, or call other dialog functions.

3.3.2.2 Initialize Conversation

In order to initialize a conversation by providing dedicated preprocessing or to manipulate the context with specific information or variables that are needed during the dialog handling, `initialize` dialog function can be used.

It supports the following actions:

- Action Group
  - set-variable actions
  - api-request actions
- Result definition

The capability developer can access the defined result variables in the `initial_context` object.

For example, when the dialog function contains a result variable user, it can be accessed from the same capability referring to `initial_context.user`.

The defined result variables are accessible in the `initial_context` object.

For example, when the dialog function contains a result variable weather, it can be accessed from the same capability referring to `initial_context.weather`.

The capabilities initialize dialog function is placed in the `/hooks/` folder. The content of the `initialize.yaml` is the same as of a normal dialog function.

> **Note**
>
> `Initialize.yaml` does not need to be explicitly referenced in the dialog function. It is executed automatically upon the start of conversation.

> **Sample Code**
>
> Sample structure when using initialize
>
> ```
> sample_capability
> ├── scenarios
> │   └── sample_scenario.yaml
> ├── hooks
> │   └── initialize.yaml
> ├── functions
> │   └── sample_function.yaml
> ├──
> └── capability.sapdas.yaml
> ```

The content of the `initialize.yaml` is same as a normal dialog function. It supports:

Dialog Function Elements	Values	Details
Action Group [page 59]	• `api-request` • `set-variable`	The defined result variables are accessible in the `initial_context` object. For example, if the dialog function contains a result variable `weather`, it can be accessed from the same capability referring to `initial_context.weather`.
Result [page 77]	Result [page 77] definition	

> **Note**
>
> `Initialize.yaml` has access to `$transient.user`.

Usage Example

`initialize.yaml` (initialize Dialog Function)

> **Sample Code**
>
> Sample structure when using initialize
>
> ```
> parameters:
>  - name: user
> action_groups:
> - condition: "true"
> ```

```
  actions:
    - type: set-variables
      scripting_type: spel
      variables:
        - name: userEmail
          value: <? user.email ?>
result:
  email : <? userEmail ?>
```

`greetings.yaml` (dialog function in a Joule Functions scenario)

Sample Code

Sample structure when using initialize

```
action_groups:
  - actions:
      - type: message
        scripting_type: spel
        message:
          type: text
          content: "Hello <? $initial_context.email ?>"
```

Restriction

It's not possible to use parameters (except `user`), messages, or to call other dialog functions.

3.3.2.3 Scripting

Joule provides the capability to process Spring Expression Language (SpEL) or handlebars scripting.

The type of scripting that is supposed to be used must be specified in the design time artifact. The default scripting language is set to SpEL.

Scripting and Dialog Functions

Scripting is supported within different locations within the files (DTA). However, not all locations support all scripting types and methods.

As a general rule of thumb, Spring Expression Language (SpEL) scripting is enabled in all scriptable fields, while handlebars scripting is only supported in dialog functions of the type *set-variables* and *message* actions.

Dialog Functions

Within dialog functions, scripting is enabled on the condition and body level.

Action conditions

Action conditions only support SpEL scripting.

{·} Sample Code

Action condition example

```
parameters:
  - name: recipient_name
    optional: true
action_groups:
  # John gets a personalized greeting
  - condition: 'recipient_name.equalsIgnoreCase("John")'
    actions:
      - type: message
        message:
          type: text
          content: 'Hi John! How is your day?'
  # default greeting
  - condition: '!recipient_name.equalsIgnoreCase("John")'
    actions:
      - type: message
        message:
          type: text
          content: 'Hi there! Welcome to SAP's Digital Assistant.'
```

Action body

Action bodies fully support SpEL scripting.

Handlebars scripting is supported for the message action and the set variables action.

Message action

Within a message action, scripting is generally supported. The default is SpEL, but handlebars can be used if defined accordingly.

Example of SpEL:

{·} Sample Code

Message action example

```
parameters:
  - name: recipient_name
action_groups:
  - actions:
      - type: message
        scripting_type: spel
        message:
          type: text
          content: 'Hello <? recipient_name.toUpperCase() ?>! Welcome to
SAP's Digital Assistant.'
```

Example of Handlebars:

{·} Sample Code

Message action example

```
parameters:
```

```
  - name: recipient_name
action_groups:
  - actions:
      - type: message
        scripting_type: handlebars
        message:
          type: text
          content: 'Hello {{uppercase recipient_name}}! Welcome to SAP's
Digital Assistant.'
```

API request action

API requests can be modified using SpEL scripting on:

- headers
- body
- path

Example:

Sample Code

API request example

```
parameters:
  - name: city
  - name: access_token
  - name: api_key
  - name: body
action_groups:
  - actions:
      - type: api-request
        method: POST
        headers:
          x-sap-crm-token: '<? access_token ?>'
          Content-Type: application/json
        body: '<? body ?>'
        system_alias: WeatherService
        path: '/location/search?query=<? city.toUpperCase() ?>&language=en-
US&format=json&apiKey=<? api_key ?>'
        result_variable: lookup_result
result:
  api: '<? lookup_result ?>'
```

Set Variables Action

The set variables action supports scripting with both SpEL and handlebars.

Example of SpEL:

Sample Code

Set variable action example

```
action_groups:
  - actions:
      - type: set-variables
        variables:
          - name: today
            value: '<? new Date() ?>'
result:
```

```
  today: '<? today ?>'
```

Example of handlebars:

Sample Code

Set variable action example

```
parameters:
  - name: filters
action_groups:
  - actions:
      - type: set-variables
        scripting_type: handlebars
        variables:
          - name: odatafilter
            value: '{{odataFilter filters}}'
result:
  filter: '<? odatafilter ?>'
```

Dialog Function Action

Dialog functions can call other dialog functions with scripted variables in parameters.

Note

A dynamic reference to a dialog function by scripting the target name is not supported.

Example:

Sample Code

Execute dialog function example

```
parameters:
  - name: city
  - name: api_key
action_groups:
  - actions:
      - type: dialog-function
        name: weather/lookup-location
        parameters:
          - name: city
            value: '<? city.toUpperCase() ?>'
            - name: api_key
            value: '<? api_key ?>'
            - name: body
            value: '{ "location": "Berlin" }'
        result_variable: weather_location
result:
  location: '<? weather_location ?>'
```

Dialog Function Result Scripting

Dialog functions produce a result that is transferred back.

This result object can be modified by scripting.

Example:

Sample Code

Result scripting example

```
action_groups:
  - actions:
      - type: api-request
        method: GET
        system_alias: WeatherService
        path: '/location/search?query=Berlin&language=en-US&format=json'
        result_variable: lookup_result
result:
  success: '<? lookup_result.status_code == 200 ?>'
  city: '<? lookup_result.body.location.address[0] ?>'
  placeid: '<? lookup_result.body.location.placeId[0] ?>'
```

Related Information

Handlebars [page 84]
Spring Expression Language [page 110]

3.3.2.3.1 Handlebars

For more flexibility while defining actions for your skills, you can use logical expressions and functions using the Handlebars template language . With Handlebars, you can use pre-defined helpers to access and execute operations on the conversation context and to format the values as per your requirement.

Block helpers (starting with #) allow you to wrap content in a condition block or to change the context for variables used within the block, whereas other helpers return a value based on the parameters you pass. Helpers can be combined flexibly by using sub-expressions.

The Handlebars helpers that are currently supported are explained in the following sections:

→ Remember

When using a simple expression (like `{{variable}}` or `{{variable.array.[0]}}`), if the variable or array element does not exist, the variable will be ignored.

When using a variable with a helper, if a required parameter of the helper is missing or is incorrect, the scripting will not be executed. For example, if you have used a helper with an incorrect parameter to define an action, the action will be skipped.

Note

Mixing big integers and float numbers is not supported.

Context Used

Sample Code

```
Data{
  "api_service_response": {
    "body": {
      "results": [
        {
          "Name": "Dayum",
          "Price": "100",
          "Available":true,
          "lookMeUp": "foundMe",
          "to_ProductCategory": {
            "MainProductCategory": "Computer Systems"
          }
        },
        {
          "Name": "test",
          "Price": "200",
          "Available":true,
          "lookMeUp": "foundMe",
          "to_ProductCategory": {
            "MainProductCategory": "Computer Systems"
          }
        },
        {
          "Name": "wow",
          "Price": "50",
          "Available":false,
          "lookMeUp": "foundMe",
          "to_ProductCategory": {
            "MainProductCategory": "Computer Systems"
          }
        }
      ],
      "lookup" : {
          "foundMe": {"works": "works=true"}
        },
      "stringArray" : ["a", "b", "c", "d", "e", "e"]
    }
  }

}
```

3.3.2.3.1.1 Working with Arrays

Helper	Use
`{{#each array}} ... {{/each}}`	Iterates over the elements in an array or object and sets the current item as context in the inner block. The following keywords can be used in the inner block: • `{{this}}` to access the current item • `{{@root}}` to access other variables, for example, `{{@root.memory.field}}` to access a memory field • `{{@index}}` to get the index of the current item (starting with 0) • `{{@first}}` and `{{@last}}` to check if the current item is the first or last item • `{{@key}}` for objects only to get the attribute key
`{{#eachJoin array}} ... {{/eachJoin}}`	Iterates over the elements in an array or object and sets the current item as context in the inner block. A comma is added between the values that are returned by the inner block of each iteration, unless it returns an empty value. This helper is especially useful when building a JSON based on an array or object. Conditions in the inner block can be used to skip some items in the array. The following keywords can be used in the inner block: • `{{this}}` to access the current item • `{{@root}}` to access other variables, for example, `{{@root.memory.field}}` to access a memory field • `{{@index}}` to get the index of the current item (starting with 0) • `{{@first}}` and `{{@last}}` to check if the current item is the first or last item • `{{@key}}` for objects only to get the attribute key
`{{#inArray array value}} ...` `{{else}} ... {{/inArray}}`	If `value` is in array this helper returns the first "..."-value else returns the second "..."-value.
`{{length array}}`	Returns the count of objects inside the `array` object provided.

Helper	Use
`{{pluck array subpath}}`	Create and return an array from child properties of the given `array`. The `subpath` refers to a child property inside of an array element in `array`. The `subpath` can also be used with the "." syntax to go deeper inside the object.
`{{unique array}}`	Unique helper returns an array with only unique values. It does not return a string (example with "each" and "eachJoin" for a string).
`{{sum numberArray}}`	Returns the sum of all numbers in the provided `numberArray` object.
`{{join array delimiter}}`	Joins all elements of an array into a string using the provided delimiter.
`{{isArray array}}`	Returns true if the value provided is an array, otherwise false. Can be used as a block: value is `{{#isArray value}}` an `{{else}}` not an `{{/isArray}}` array.

Accessing Elements of an Array

> **Note**
>
> The **new** syntax to access an element in an array is `{{array.[0]}}`. The **old** syntax `{{array[0]}}` is still supported for simple expressions, but cannot be used in combination with helpers and will be deprecated completely with one of the next releases.

Helper	Use
`{{first array n}}`	Returns the first value of the array provided. If an argument is specified, an array with the first n elements is returned. If the argument is negative, an array with all but the last n elements is returned.
`{{last array n}}`	Returns the last value of the array provided. If an argument is specified, an array with the last n elements is returned. If the argument is negative, an array with all but the first n elements is returned.

Helper	Use
`{{itemAt array n}}`	Returns the element of the array provided at the index n. If no argument is specified, the first element is returned. If the argument is negative, the last n-th element is returned.

Examples

Gets the first item of an list or array object

Sample Code

Accessing elements - first

```
"{{first api_service_response.body.stringArray}}" // handlebars statement
"a" // result
```

Returns a specific element of a list/array object

Sample Code

Accessing elements - itemAt

```
"{{itemAt api_service_response.body.stringArray 2}}" // handlebars statement
"c" // result
```

Returns the unique values from an array using the "each" and "eachJoin" helper in combination with the "unique" helper

Sample Code

Working with arrays - unique

```
"{{#each (unique api_service_response.body.stringArray)}}{{.}}{{/each}}" //
handlebars statement
"abcde" // result

"{{#eachJoin (unique api_service_response.body.stringArray)}}
{{.}}.atIndex({{@index}}){{/eachJoin}}" // handlebars statement
"a.atIndex(0),b.atIndex(1),c.atIndex(2),d.atIndex(3),e.atIndex(4)"// result
```

Retrieves property in array of objects as list

Sample Code

Working with arrays - pluck

```
"{{pluck api_service_response.body.results 'Name'}}" // handlebars statement
"Dayum,test,wow" // result

"{{pluck api_service_response.body.results
'to_ProductCategory.MainProductCategory'}}" // handlebars statement
```

```
"Computer Systems,Computer Systems,Computer Systems" // result
```

Using the pluck helper together with the json helper to get an array of sub properties. Without the json helper you get a concatenated string of the elements.

Sample Code

Working with arrays - pluck and json

```
"{{json (pluck api_service_response.body.results
'to_ProductCategory.MainProductCategory')}}" // handlebars statement
"["Computer Systems","Computer Systems","Computer Systems"]" // result
```

Returns a concatenated string of all elements inside of the list or array element. Usage of the *@index* keyword will add the index of the current iterated object.

Sample Code

Working with arrays - eachJoin

```
"{{#eachJoin api_service_response.body.results}}{'title': '{{Name}}',
'subtitle': 'No. {{@index}}'}{{/eachJoin}}" // handlebars statement
"{'title': 'Dayum','subtitle': 'No. 0'},{'title': 'test','subtitle': 'No. 1'},
{'title': 'asdf','subtitle': 'No. 2'}" // result
```

3.3.2.3.1.2 Working with Strings

Helper	Use
`{{append string suffix}}`	Appends the `suffix` to the string.
`{{lowercase string}}`	Transforms all characters in the string to lowercase letters.
`{{uppercase string}}`	Transforms all characters in the string to uppercase letters.
`{{isString value}}`	Returns true if the passed value is a string, false otherwise.
`{{occurrences string substring}}`	Searches for `substring` in `string` and returns the number of occurrences.
`{{remove string substring}}`	Removes all occurrences of `substring` within `string` and returns the result.
`{{replace string substringA substringB}}`	Searches for all occurrences of `substringA` in `string`, replaces them with `substringB` and returns the result.

Helper	Use
`{{split string separator}}`	Splits `string` by using the separator and returns the values as an array. Please note that the array will be returned as a single string. For more information, see Example [page 90].
`{{trim string}}`	Removes any white space characters from the beginning and the end of the string.
`{{length string}}`	Returns the length of the string.
`{{escape variable}}`	Escapes any double-quotes in the variable in order to safely include it as a string within JSON. In case an object or array is passed, it will be converted to an escaped JSON string.
`{{urlEncode string}}`	Encodes all characters within a string that are either illegal or have reserved meanings in a URI, as defined in RFC 3986, using percent-encoding. IBN should be used for navigational links whenever possible.
`{{ padding string targetLength paddingCharacter }}`	Returns the string after left-padding it with the padding Character if the length of the string is less than the target Length. Otherwise the string is returned unchanged.

Example

Sample Code

Split the String

```
"stringABC": "A B C"
"arrayAsAString": "{{split stringABC ' '}}" // handlebars statement
"arrayAsAString": "A,B,C" //result
"arrayABC": "{{json (split stringABC ' ')}}" // handlebars statement
"arrayABC": ["A","B","C"] //result
"{{isArray arrayAsAString}}" // handlebars statement
false //result
"{{isArray arrayABC}}" // handlebars statement
true //result
"{{isArray (split stringABC ' ')}}" // handlebars statement
true //result
"encodedURL": "{{urlEncode "www.sap.com/query=test?§param=42"}}" //
handlebars statement
"encodedURL": "ww.sap.com%2Fquery%3Dtest%3F%C2%A7param%3D42" //result
"dynamicPadding" : " {{ padding "12345" 10 'x' }} "   //handlebars statement
"dynamicPadding" : "xxxxx12345"     //result
```

3.3.2.3.1.3 Working with OData Services

Helper	Use
`{{odataFilter value}}`	Returns a valid OData filter string to be used as `$filter` query parameter. For the `value` parameter, either an object, an array of objects, a string or a combination of those can be passed. When an object or array of objects is passed, each object must have the following attributes: • `service-value`: Value that the OData attribute should be compared to • `service-attribute`: OData attribute that should be filtered by • `filter-operator`: OData operator used for filtering, for example, "eq" when the value of the OData attribute and the value passed as `service-value` should be exactly same Objects with the same `service-attribute` will be combined with an „or“, all others with an „and“. When a string is passed, it needs to be a valid OData filter string and will be appended with `and` to the filter string generated by the helper.

Example Context

Sample Code

```
{
  "conversation": {
    "memory": {
      "supplier": [
        {
          "confidence": 0.99,
          "raw": "samsung",
          "service-value": 2,
          "filter-operator": "eq",
          "value": "samsung",
          "service-attribute": "SupplierID"
        },
        {
          "confidence": 0.99,
          "raw": "apple",
          "service-value": 5,
          "filter-operator": "eq",
          "value": "apple",
          "service-attribute": "SupplierID"
        }
      ],
      "category": [
        {
          "confidence": 0.99,
          "raw": "notebooks",
```

```
          "service-value": 1,
          "filter-operator": "eq",
          "value": "notebooks",
          "service-attribute": "CategoryID"
        }
      ],
      "nlp": {
        "entities": {
          "mass": [
            {
              "confidence": 0.99,
              "raw": "20 lbs",
              "scalar": 20,
              "unit": "lbs",
              "grams": 9071.84
            }
          ]
        }
      }
    }
  }
}
```

Example OData Filter

```
{{odataFilter memory.supplier memory.category 'myuser eq "true"'}}
```

```
myuser eq 'true' and (SupplierID eq 5 or SupplierID eq 2) and CategoryID eq 1
```

Example usage in a URL

https://example.com/odata/?$format=json&$filter=`{{odataFilter memory.supplier memory.category 'myuser eq "true"'}}`

Resulting URL

```
https://example.com/odata/?
$format=json&$filter=myuser%20eq%20%27true%27%20and%20(SupplierID%20eq%205%20or%20S
upplierID%20eq%202)%20and%20CategoryID%20eq%201
```

3.3.2.3.1.4 Comparing Values

The compare helpers are used to construct logical conditions and return values depending on the comparison. The first "..." is a user defined value that will be returned if the comparison resolves as `True` otherwise the second "..." is returned.

In cases where you want to chain multiple compare helpers, you can use the `else` statement with the same or a different compare helper, for example, `{{else if condition}}` or `{{else eq value1 value2}}`.

Helper	Use
`{{#if valueA}} ... {{else}} ... {{/if}}`	Evaluates valueA for existence. If valueA is one of the values listed here (false, undefined, null, "", 0, or []) it will return the second "..."-value otherwise it returns the first "..."-value.

Helper	Use
`{{#unless valueA}} ... {{else}} ... {{/unless}}`	Evaluates valueA for non-existence. It has the inverse functionality of the "if" helper.
`{{#eq valueA valueB}} ... {{else}} ... {{/eq}}`	Evaluates if both values provided are equal (eq).
`{{#gt valueA valueB}} ... {{else}} ... {{/gt}}`	Evaluates if "valueA" greater than "valueB" (gt).
`{{#lt valueA valueB}} ... {{else}} ... {{/lt}}`	Evaluates if "valueA" is lower than "valueB" (lt).
`{{#gte valueA valueB}} ... {{else}} ... {{/gte}}`	Evaluates if "valueA" greater than or equal to "valueB" (gte).
`{{#lte valueA valueB}} ... {{else}} ... {{/lte}}`	Evaluates if "valueA" is lower than or equal to "valueB" (lte).
`{{#compare valueA operator valueB}} ... {{else}} ... {{/compare}}`	Compares "valueA" with "valueB" using a JavaScript operator as "operator" (for example,"'===" or "<=").
`{{default optionalValue defaultValue}}`	Returns the "optionalValue" - if it is defined - otherwise the "default" value is returned.

Examples

Compares two parameters with a JavaScript operator.

Sample Code

Comparing values - compare

```
"{{#compare api_service_response.body.results.0.Name '===' 
api_service_response.body.results.1.Name}}This will not be rendered because 
0.Name and 1.Name are different.{{else}}This will be rendered.{{/
compare}}" // handlebars statement
"This will be rendered." // result
```

Compares two numbers. This checks if valueA is bigger than valueB.

Sample Code

Comparing values - greaterThan (gt)

```
"{{#gt api_service_response.body.results.0.Price data.results.1.Price}}True: 
This will be rendered.{{else}}False: This will be rendered.{{/gt}}" // 
handlebars statement
"False: This will be rendered." // result
```

```
"{{#gt api_service_response.body.results.1.Price data.results.0.Price}}True:
This will be rendered.{{else}}False: This will be rendered.{{/gt}}" //
handlebars statement
"True: This will be rendered." // result
```

3.3.2.3.1.5 Logical Operators

Helper	Use
`{{#and expressionA expressionB}} ... {{/and}}`	Returns the "..."-value if both of the given expressions resolve to True.
`{{#or expressionA expressionB}} ... {{/or}}`	Returns the "..."-value if one of the given expressions is resolved to True.
`{{#not expression}} ... {{/not}}`	Returns the "..."-value if no given expressions resolve to True

Examples

Checks if two expressions are true.

Sample Code

Logical operators - and

```
"{{#and api_service_response.body.results.0.Available
api_service_response.body.results.1.Available}}Both 'True'{{else}}At least
one not 'true'{{/and}}" // handlebars statement
"Both 'True'" // result

"{{#and api_service_response.body.results.0.Available
api_service_response.body.results.2.Available}}Both 'True'{{else}}At least
one not 'true'{{/and}}" // handlebars statement
"At least one not 'true'" // result
```

3.3.2.3.1.6 Working with Numbers

Caution

Using mathematical operations with Big Integers in *Handlebars* may result in a loss of precision due to the lack of native support for large integer arithmetic.

Helper	Use
`{{add valueA valueB}}` / `{{plus valueA valueB}}`	Returns a number after the execution of the arithmetic function. Add
`{{subtract valueA valueB}}` / `{{minus valueA valueB}}`	subtract
`{{multiply valueA valueB}}`	multiply
`{{divide valueA valueB}}`	divide valueB with valueA
`{{modulo valueA valueB}}`	Returns the remainder after division (valueA divided by valueB)
`{{ceil value}}`	Returns the next higher absolute number
`{{floor value}}`	Returns the next lower absolute number
`{{round value}}`	Returns the correct rounded absolute number
`{{formatNumber number decimalLength decimalSeparator thousandsSepatator}}`	Formats the number with the specified number of (rounded) digits after the decimal separator and the specified character is used as decimal separator. In addition, a separator for the group of thousands can be provided. All parameters are optional. Following are some examples: `{{formatNumber value 2}}`- Returns the correct rounded number at two decimal digits `{{formatNumber value decimalSeparator}}`- Returns the number with a specified decimal and thousand separator `{{formatNumber value decimalSeparator thousandSeparator}}`- Returns the number with a specified decimal and thousand separator `{{formatNumber value decimalLength decimalSeparator thousandSeparator}}`- Returns the number with specified decimal digits and separators

3.3.2.3.1.7 Working with Dates

Helper	Use
`{{formatDate date pattern timezone/utc-offset}}`	The date helper returns the provided `date` formatted with the provided `pattern` converted to the provided `timezone` or `utc-offset`. `{{formatDate date pattern timezone/utc-offset}}` The date value can have the following formats: • /Date(1338282808000)/ • 1992-01-01T00:00:00 • 2021-01-01T20:15:00+02:00 • 01-31-2000 (Month/Day/Year) **Note** If the provided date does not contain an offset, it's assumed to be UTC. If no date is provided at all, the current date and time is used. **Note** The *formatDate* helper takes the language/locale setting into account when displaying the month, day, and numerals. The overall date format (for example, order of year/month/day) remains unchanged and is not affected by the language setting. For example, in German instead of **Monday, 1. December**, the date is displayed as **Montag, 1. Dezember**. In Hindi, for example, instead of **2025-04-16**, the date is displayed as २०२५-०४-१६. In the `pattern` parameter, the following tokens can be used and combined as needed (click > to expand the table): Date Formats

Date	Token	Output
Month	M	1 2 ... 11 12
	Mo	1st 2nd ... 11th 12th
	MM	01 02 ... 11 12
	MMM	Jan Feb ... Nov Dec

Helper	Use		
	Date	**Token**	**Output**
		MMMM	January February ... November December
	Quarter	Q	1 2 3 4
		Qo	1st 2nd 3rd 4th
	Day of Month	D	1 2 ... 30 31
		Do	1st 2nd ... 30th 31st
		DD	01 02 ... 30 31
	Day of Year	DDD	1 2 ... 364 365
		DDDo	1st 2nd ... 364th 365th
		DDDD	001 002 ... 364 365
	Day of Week	d	0 1 ... 5 6
		do	0th 1st ... 5th 6th
		dd	Su Mo ... Fr Sa
		ddd	Sun Mon ... Fri Sat
		dddd	Sunday Monday ... Friday Saturday
	Day of Week (Locale)	e	0 1 ... 5 6
	Day of Week (ISO)	E	1 2 ... 6 7
	Week of Year	w	1 2 ... 52 53
		wo	1st 2nd ... 52nd 53rd
		ww	01 02 ... 52 53
	Week of Year (ISO)	W	1 2 ... 52 53
		Wo	1st 2nd ... 52nd 53rd
		WW	01 02 ... 52 53
	Year	YY	70 71 ... 29 30
		YYYY	1970 1971 ... 2029 2030

Helper	Use		
	Date	**Token**	**Output**
		YYYYYY	-001970 -001971 ... +001907 +001971 ⓘ Note Expanded years (Covering the full time value range of approximately 273,790 years forward or backward from 01 January, 1970)
		Y	1970 1971 ... 9999 +10000 +10001 ⓘ Note This complies with the ISO 8601 standard for dates past the year 9999
	Era Year	y	1 2 ... 2020 ...
	Era	N, NN, NNN	BC AD ⓘ Note Abbr era name
		NNNN	Before Christ, Anno Domini ⓘ Note Full era name
		NNNNN	BC AD

Helper	Use		
	Date	**Token**	**Output**
			ⓘ Note Narrow era name
	Week Year	gg	70 71 ... 29 30
		gggg	1970 1971 ... 2029 2030
	Week Year (ISO)	GG	70 71 ... 29 30
		GGGG	1970 1971 ... 2029 2030
	AM/PM	A	AM PM
		a	am pm
	Hour	H	0 1 ... 22 23
		HH	00 01 ... 22 23
		h	1 2 ... 11 12
		hh	01 02 ... 11 12
		k	1 2 ... 23 24
		kk	01 02 ... 23 24
	Minute	m	0 1 ... 58 59
		mm	00 01 ... 58 59
	Second	s	0 1 ... 58 59
		ss	00 01 ... 58 59
	Fractional Second	S	0 1 ... 8 9
		SS	00 01 ... 98 99
		SSS	000 001 ... 998 999
		SSSS ... SSSSSSSSS	000[0..] 001[0..] ... 998[0..] 999[0..]
	Time Zone	z or zz	EST CST ... MST PST
		Z	-07:00 -06:00 ... +06:00 +07:00
		ZZ	-0700 -0600 ... +0600 +0700
	Unix Timestamp	X	1360013296

Helper	Use
	<table><tr><th>Date</th><th>Token</th><th>Output</th></tr><tr><td>**Unix Millisecond Timestamp**</td><td>x</td><td>1360013296123</td></tr></table> **Examples**: • "DD-MM-YYYY" • "MM-YYYYZ" • "DD-MM-YYYY - HH:mm" • "DD.MM.YYYY HH:mm Z" A timezone must be a valid IANA Timezone name like this: • Europe/Berlin • America/Los_Angeles The utcOffset should be formatted like this: • "-02:00" or "+01:00"(for strings) -120 or 60 (for number values as minutes) **Note** Numbers between -16 and 16 will be automatically used as hours, for example, +11 will result in "+11:00".

Helper	Use

`{{convertDate date type}}`
`{{convertDate date "relative" to=date}}`
`{{convertDate date "relative" from=date}}`
	Converts a date into an object or another type.
	
	Note
	If a date is provided, the helper tries to parse it. If no date is given, the **current UTC time** is used as the date.
	
	The following target types are supported:
	
	Type / **Description**
	object — Breaks down the date into its elements and returns them as an object, for example, {"year":2021,"month":11,"day":16,"hour":17,"minute":37,"second":37,"millisecond":78}
	unixms — Returns the milliseconds since 1 January 1970, for example, 1639672873051
	unix — Returns the seconds since 1 January 1970, for example, 1639672873
	iso8601 — Returns the date in ISO-8601 representation, for example, 2021-12-16T16:40:51.402Z
	rfc2822 — Returns the date in RFC-2822 representation, for example, Fri, 17 Dec 2021 14:03:59 GMT
	relative — Has to be used with *either* `to=` *or* `from=` and returns, for example, 5 minutes ago, in 4 days
	
	Note
	`to=` and `from=` for target type "relative" cannot be combined.

Helper	Use

`{{modifyDate addYears/Months/Days/Hours/Minutes/Seconds=number}}`
`{{modifyDate subtractYears/Months/Days/Hours/Minutes/Seconds=number}}`
`{{modifyDate setYear/Month/Day/Week/Hour/Minute/Second=number}}`
`{{modifyDate start=minute/hour/day/month/year}}`
`{modifyDate end=minute/hour/day/month/year}}`
	Modifies an existing date by either adding or subtracting the number of seconds / minutes / hours / days / months or years, or by setting one of these to a provided value.
	
	Note
	If a date is provided, the helper tries to parse it. If no date is given, the **current UTC time** is used as the date.
	
	The following parameters are supported and can be combined:

Parameter	Value	Description
`addDays`	number	Add day(s) to the date
`addHours`	number	Add hour(s) to the date
`addMinutes`	number	Add minute(s) to the date
`addMonths`	number	Add month(s) to the date
`addSeconds`	number	Add second(s) to the date
`addYears`	number	Add year(s) to the date
`end`	year/month/day/hour/minute	Sets the date to the end of the specified element
`setDay`	number (1-31)	Sets the day of the date
`setHour`	number (0-23)	Sets the hour of the time
`setMinute`	number (0-59)	Sets the minute of the time
`setMonth`	number (1-12)	Sets the month of the date
`setSecond`	number (0-59)	Sets the second of the time
`setWeek`	number (1-53)	Sets the week of the date
`setYear`	number	Sets the year of the date

Helper	Use

Parameter	Value	Description
`start`	year/month/day/hour/minute	Sets the date to the start of the specified element
`subtractDays`	number	Subtract day(s) from the date
`subtractHours`	number	Subtract hour(s) from the date
`subtractMinutes`	number	Subtract minute(s) from the date
`subtractMonths`	number	Subtract month(s) from the date
`subtractSeconds`	number	Subtract second(s) from the date
`subtractYears`	number	Subtract year(s) from the date

Examples

Transforms Date to UTC with Offset

Sample Code

Working with Dates - formatDate

```
"{{formatDate "01-31-2000" "DD-MM-YYYYZ" 120}}" // handlebars statement
"31-01-2000+02:00" // result

"{{formatDate "/Date(1544464091000+0000)" "DD-MM-YYYY - HH:mm" "+01:00"}}" //
handlebars statement
"10-12-2018 - 18:48" // result

"{{formatDate "DD-MM-YYYY - HH:mm"}}" // handlebars statement
"01-10-2021 - 15:44" // result will be the current date and time (UTC)

"{{formatDate api_service_response.body.results.0.ReleaseDate "YYYY"}}" //
handlebars statement
"1990" // result

"{{formatDate "2021-09-01T09:00:00+05:00" "DD.MM.YYYY HH:mmZ" "America/
Los_Angeles"}}" // handlebars statement
"31.08.2021 21:00-07:00" // result

"{{formatDate "2021-09-01T09:00:00+05:00" "DD.MM.YYYY HH:mmZ"
client_info.timezone}}" // handlebars statement
```

```
"01.09.2021 06:00 +02:00" // result if conversation timezone detected by the
client is "Europe/Berlin"
```

Converts the Provided or Current Date into an Object or Relative Time Period as Text.

Sample Code

```
{{convertDate "2021/12/12" "object"}} //
{"year":2021,"month":11,"day":12, ...}

{{convertDate "object"}} // uses current date
{"year":2022,"month":1,"day":12, ...}

{{convertDate "relative" to="2021/12/16"}} // uses current date 1 day ago
```

Modifies a Date by, for example, Adding Number of Days, Setting it to The Start or End of a Day or Combining Multiple Such Operations.

Sample Code

```
{{ modifyDate addDays=1 }} // get date of tomorrow

{{ modifyDate "2019/1/1 14:20" addDays=1 setMinute=30 }} // 02.01.2019 at
14:30

{{ modifyDate addHours=1 addMinutes=13 addSeconds=5 }} // add 1h 13m and 5s
to current UTC time

{{ modifyDate "2021/12/12 19:59" start="day" }} // 12.12.2021 00:00:00
{{ modifyDate "2021/12/24 13:37" end="day" }} // 24.12.2021 23:59:59

{{ modifyDate (modifyDate start="hour" subtractMinutes=10)
subtractMinutes=2 }} // chaining
```

3.3.2.3.1.8 Working with Objects

Helper	Use
`{{json object}}`	Returns a json object for "valueA".
`{{#with object}} ... {{/with}}`	Navigates the handlebars value scope into the object. Navigate into an object A with property B allows you to use `{{B}}` syntax to access an element in an array is directly instead of `{{A.B}}`
`{{lookup objectA objectB}}`	Returns the value of the object A accessed with a property value of object B.

Helper	Use

`{{flpAppState object1 object2 ...}}`	Re-formats the object so that it can directly be used in SAP Fiori Launchpad as an `AppStateData` filter.
	
	As parameters, either one or multiple objects, an array of objects, or a combination of those can be passed.
	
	Each object must have the following attributes:
	
	• `service-value`: value that the attribute should be compared to
	• `service-attribute`: attribute that should be filtered by
	• `filter-operator`: operator used for filtering, for example, "eq" is used when the value of the service attribute and the value passed as "service-value" should be exactly same. Supported filter operators are: eq, ne, lt, le, gt, ge.
	
	Objects with the same `service-attribute` will be combined in one `SelectOption` with two ranges.
	
	Input objects for flpAppState:
	
	Sample Code
	
	`{ "raw": "Notebooks", "value": "notebooks", "service-value": "C3712", "service-attribute": "CategoryID", "filter-operator": "EQ" }`
	
	Output object for flpAppState:
	
	Sample Code
	
	`{ "selectionVariant": { "SelectOptions": [ { "PropertyName": "CategoryID", "Ranges": [ { "Sign": "I", "Option": "EQ", "Low": "C3712" } ] } ] } }`

Helper	Use

	`}`
	
	Usage of flpAppState:
	
	Sample Code
	
	`{ "fiori": { "navigate": { "semanticObject": "{{memory.SemanticObject}}", "action": "{{memory.action}}", "params": { "{{memory.key1}}": "{{memory.value1}}" }, "appStateData": {{ flpAppState memory.category }} } } }`
	
	Variations for flpAppState:
	
	• Working with two different properties
	`{{ flpAppState memory.category memory.supplier }}`
	• Working with one property but two filters
	`{{ flpAppState memory.category memory.category2 }}`
	• Working with one property that has two filters and another different property
	`{{ flpAppState memory.category1 memory.category2 memory.supplier }}`

Helper	Use

`{{setValue <objectToModify> <pathToKey> <newValue>}}`	Returns a clone of the original object with the changed new value at the provided key path.
	
	If the key path points to a non-existent key, it will be added.
	
	The path can have the following formats:
	
	1. `path.key`
	2. `path["key"]`
	3. `path.array.[0]`(first element of array)
	4. `path.array.[-1]`(last element of array)
	5. `key`
	
	Example
	
	Sample Code
	
	`{{setValue company "address.street" "Dietmar-Hopp-Allee 16"}}`
	
	`{{setValue company "address['street']" "Dietmar-Hopp-Allee 16"}}`
	
	`{{setValue company "address.[0]" "Dietmar-Hopp-Allee 16"}}`
	
	`{{setValue company "address" "Dietmar-Hopp-Allee 16"}}`
	
	Note
	
	The `newValue` can also be set to the value of an object.
	
	Sample Code
	
	`{{setValue company "address.street" newAddressObject}}`
	
	The newValue can be:
	
	• an object or array
	• a String
	• a number
	• a boolean

Helper	Use

`{{sort <array> <pathToKey> <orderType>}}`	Sorts a list or an array at the given path for the given order.
	
	This helper also supports to sort an array by a given attribute.
	
	The path points to the relative path of an existing key of the list entry. The value at the path of the list entry needs to be present for all elements. The type of the value has to be the same. Only comparisons of values is possible. Comparisons of objects or arrays will lead to errors. If no path is specified, the root element is taken.
	
	The path can have the following formats:
	
	• `path.key`
	• `path["key"]`
	• `path.array.[0]` (first element of array)
	• `path.array.[-1]` (last element of array)
	• `key`
	
	The sort order can be `asc` (ascending) or `desc` (descending). If no sorting order is provided, the default sort order is set to `asc`.
	
	This helper is able to sort the following structures:
	
	• array or list containing objects with arrays or lists by their attributes
	• arrays or lists containing numbers by a given order
	• String arrays and lists by a given order
	• Boolean arrays and lists by a given order
	
	Example:
	
	Sample Code
	
	`Array to sort:`
	`sampleArray1: [{name: "apple", id: 1, price: {value: 1.5, currency: "EUR"}}, {name: "pear", id: 2, price: {value: 0.5, currency: "EUR"}}, {name: "banana", id: 3, price: 0.8}]`
	
	`Helper:`
	`{{sort sampleArray1 "price.value" "desc"}}`
	
	`Result:`
	`[{name: "apple", id: 1, price: {value: 1.5, currency: "EUR"}}, {name: "banana", id: 3, price: {value: 0.8, currency: "EUR"}},`

Helper	Use
	`{name: "pear", id: 2, price: {value: 0.5, currency: "EUR"}]`

`{{lookupPath <targetObject> <pathToKey>}}`	Returns the value of the specified target object at the provided key path.
	
	When the result value is an object, it is returned in json format. Otherwise, the result is returned as a simple string.
	
	If the key path points to a non-existent key, an error will be thrown.
	
	The path can have the following formats:
	
	• `path.key`
	• `path["key"]`
	• `path.array.[0]` (first element of array)
	• `path.array.[-1]` (last element of array)
	• `key`
	
	Example:
	
	Sample Code
	
	`{{lookupPath company "contact.headquarter.email"}} // headquarter@company.com`
	
	`{{lookupPath company "contact.headquarter.['email']"}} // headquarter@company.com`
	
	`{{lookupPath company "contact.headquarter.phoneNumbers.[0]"}} // +49 (0)6227 / 7-47474`
	
	`{{lookupPath company "contact.headquarter"}} // {"email":"headquarter@company.com", "phoneNumbers": ["+49 (0)6227 / 7-47474", "+49 (0)6227 / 7-57575"]}`
	
	If a key in the key path contains a dot, the following format can be used.
	
	Sample Code
	
	`{{lookupPath company "contact.['help.desk'].email"}}`

Examples

Returns the accessed object value and elevates the scope into the access object to print the result.

Sample Code

Working with Objects - with & lookup

```
"{{api_service_response.body.results.0.Name}} with:
{{#with (lookup api_service_response.body.lookup
api_service_response.body.results.0.lookMeUp)}}{{works}}{{/with}}" //
handlebars statement
"Dayum with: works=true" // result
```

Format the number with the formats as common in US and Germany.

Sample Code

Working with Objects - with & lookup

```
{{formatNumber price 2 '.' ','}}
100 -> 100.00
99999.9504 -> 99,999.95


{{formatNumber price 2 ',' '.'}}
100 -> 100,00
99999.9504 -> 99.999,95

{{formatNumber price 2 ','}}
100 -> 100,00
99999.9504 -> 99999,95

{{formatNumber price ',' '.'}}
100 -> 100
99999.9504 -> 99.999,9504
```

3.3.2.3.2 Spring Expression Language

Spring Expression Language (SpEL) Scripting Support

All supported operations are shown as examples without scripting indication.

Depending on the location where these operations are used, the scripting tags <? and ?> will be needed for scripting to work.

Chaining methods

Chaining methods is fully supported within dialog functions.

Sample Code

Chaining example

```
sentence = "This sentence contains words."
sentence.substring(5).contains("words")
```

3.3.2.3.2.1 Native Operations

Native operations consist of default operations that are provided by the SpEL framework. For more information, see Spring Expression Language Documentation

Mathematical Operators

The following mathematical operators are supported:

Name	Operator
Addition	+
Subtraction	-
Multiplication	*
Division	/
Modulus	%
Exponential power	^

In addition the operators follow mathematical precedence.

Usage

Availability	• Dialog functions
Parameters	• Numbers • Mathematical operators
Output	Number

Example	Sample Code
	extract sample
	`2 + 3 * (3 - 1)^2`
	`// =14`

Relational Operators

The following relational operators are supported:

Name	Operator
lt	<
gt	>
le	<=
ge	>=
eq	==
ne	!=

In addition the operators follow logical precedence.

Usage

Availability	• Dialog functions
Parameters	• Numbers • Relational operators
Output	Boolean

Example	Sample Code
	extract sample
	`15 > 11`
	`// =true`

Ternary Operator (If-Then-Else)

A ternary operator consists of a logical expression and a corresponding result depending on whether the expression is true or false.

Usage

Availability	• Dialog functions
Parameters	• Logical expression • True result • False result
Output	Result

Example	Sample Code
	extract sample
	`2 == 2 ? "This is equal." : "Not equal"`
	`// ="This is equal."`

Assignment

Assigning values to variables is possible with the equal sign.

Usage

Availability	• Dialog functions
Parameters	• Variable • Value
Output	None

Example	Sample Code
	extract sample
	`number = 42`
	`// no return value, variable can be used`
	
	`context.number = 42`
	`// creates variable below context node`
	
	`person = { firstName: "John", lastName: "Doe", address: { city: "New York", street: "Fifth Street" } }`
	`// no return value, object assigned to variable`

3.3.2.3.2.2 String Operations

The following string operations are supported where indicated.

String literals and dialog functions are fully supported.

> **Note**
>
> Within message actions, string literals are only supported if they are used within single quotation marks. For example, 'example'.
>
> Use the set variables action as a workaround.

append String.concat

append concatenates the source String with the parameter String without changing the source String.

Usage

Availability	• Dialog functions
Parameters	String
Output	String

Example	Sample Code
	extract sample
	`string = "This is "`
	`string.append("appended!")`
	`// ="This is appended!"`

contains

`contains` checks whether the parameter String exists as a substring within the source String. This check is case sensitive.

Usage

Availability	• Dialog functions
Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Does this contain words?"`
	`string.contains("word")`
	`// =true`
	`string.contains("WORD")`
	`// =false`
	`string.contains("numbers")`
	`// =false`

endsWith

`endsWith` checks whether the source String ends with the parameter substring. This check is case sensitive.

Usage

Availability	• Dialog functions
Parameters	String

Output	Boolean

Example	Sample Code
	extract sample
	`string = "This is the end."`
	`string.endsWith("the end.")`
	`// =true`
	`string.endsWith("the beginning.")`
	`// =false`

equals

`equals` checks for case sensitive String equality.

Usage

Availability	• Dialog functions
Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Does this equal?"`
	`string.equals("Does this equal?")`
	`// =true`
	`string.equals("Does this EQUAL?")`
	`// =true`
	`string.equals("Something else.")`
	`// =false`

equalsIgnoreCase

`equalsIgnoreCase` checks for case insensitive String equality.

Usage

Availability	• Dialog functions

Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Does this equal?"`
	`string.equals("Does this equal?")`
	`// =true`
	`string.equals("Does this EQUAL?")`
	`// =true`
	`string.equals("Something else.")`
	`// =false`

extract

`extract` takes the source String and extracts parts of it based on the provided regular expression.

The provided integer indicates which parts of the regular expressions shall be returned.

Integer	Description
0	The full regular expression match will be returned.
1	The first grouped part in the regular expression will be returned only.
n	The n-th grouped part in the regular expression will be returned only.

Usage	
Availability	• Dialog functions
Parameters	• Regular expression • Integer
Output	String

Example	Sample Code
	extract sample
	`string = "Can you find Waldo in this sentence?"`
	`string.extract("find ([A-Za-z]*) in ([A-Za-z]*) ([A-Za-z]*)", 0) // ="find Waldo in this sentence"`
	`string.extract("find ([A-Za-z]*) in ([A-Za-z]*) ([A-Za-z]*)", 1) // ="Waldo"`
	`string.extract("find ([A-Za-z]*) in ([A-Za-z]*) ([A-Za-z]*)", 2) // ="this"`
	`string.extract("find ([A-Za-z]*) in ([A-Za-z]*) ([A-Za-z]*)", 3) // ="sentence"`

find

`find` checks whether a regular expression matches the source string.

The provided regular expression only needs to match a certain substring and not the full String and therefore differs in behavior from matches.

Usage

Availability	• Dialog functions
Parameters	Regular expression
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Can you find Waldo in this sentence?"`
	`string.find("Waldo") // =true`
	`string.find("find ([A-Za-z]+) in") // =true`
	`string.find("Oswald") // =false`

getMatch

`getMatch` searches for all Strings matching the regular expression. The provided integer is used as an index to return the String at the matching position.

This is different from extract by not referencing the groups provided within the regular expression, but instead the array of matching expressions found within.

Usage

Availability	• Dialog functions
Parameters	• Regular expression • Integer
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Can you find the individual Waldo in this sentence?"`
	`string.getMatch("find the individual Waldo in", 0)`
	`// ="find the individual Waldo in"`
	`string.getMatch("([A-Za-z]*)in([A-Za-z]*)", 0)`
	`// ="find"`
	`string.getMatch("([A-Za-z]*)in([A-Za-z]*)", 1)`
	`// ="individual"`
	`string.getMatch("([A-Za-z]*)in([A-Za-z]*)", 2)`
	`// ="in"`

isEmpty

`isEmpty` checks whether a String is empty.

Usage

Availability	• Dialog functions
Parameters	
Output	Boolean

Example	Sample Code
	extract sample
	`empty = ""`
	`blank = " "`
	`notEmpty = "not empty"`
	
	`empty.isEmpty()`
	`// =true`
	
	`blank.isEmpty()`
	`// =false`
	
	`notEmpty.isEmpty()`
	`// =false`

length

`length` returns the amount of characters within a String.

Usage

Availability	• Dialog functions
Parameters	
Output	Integer

Example	Sample Code
	extract sample
	`string = "four"`
	
	`string.length()`
	`// =4`

matches

`matches` checks whether a regular expression matches the source string.

The provided regular expression needs to match the full String from beginning to end. This differs in behavior from `find`.

Usage

Availability	• Dialog functions

Parameters	Regular expression
Output	Boolean

Example	Sample Code
	extract sample
	`string = "Can you find Waldo in this sentence?"`
	`string.matches("^(.*)in this(.*)$")`
	`// =true`
	`string.matches("(.*)in this(.*)")`
	`// =true`
	`string.matches("find ([A-Za-z]*) in")`
	`// =false`

split

`split` operator splits the text at the provided String separator into an array consisting the String parts excluding the String separator.

Usage

Availability	• Dialog functions
Parameters	String
Output	List of Strings

Example	Sample Code
	extract sample
	`string = "Split at the comma, for we want to test it, even if there are three in the array!"`
	`string.split(", ")`
	`// =[ "Split at the comma", "for we want to test it", "even if there are three in the array!" ]`

startsWith

`startsWith` checks whether the source String starts with the parameter substring. This check is case sensitive.

Usage

Availability	• Dialog functions
Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`string = "The beginning is always somewhere."`
	`string.startsWith("The beginning") // =true`
	`string.startsWith("The end") // =false`

substring

`substring` returns a substring of the source String.

Providing one integer parameter will create a substring starting from the indicated position until the end.

Providing two integer parameters will create a substring starting from the first position until the second position.

Usage

Availability	• Dialog functions
Parameters	• integer • integer (optional)
Output	String

Example	Sample Code
	extract sample
	`string = "From twelve starts this substring"`
	`string.substring(12)`
	`// ="starts this substring"`
	`string.substring(12, 18)`
	`// ="starts"`

toLowerCase

`toLowerCase` returns a lowercase representation of the source String.

Usage

Availability	• Dialog functions
Parameters	
Output	String

Example	Sample Code
	extract sample
	`string = "Put this in all lowercase."`
	`string.toLowerCase()`
	`// ="put this in all lowercase."`

toUpperCase

`toUpperCase` returns an uppercase representation of the source String.

Usage

Availability	• Dialog functions
Parameters	
Output	String

Example	Sample Code
	extract sample
	`string = "Put this in all UPPERCASE."`
	`string.toUpperCase()`
	`// ="PUT THIS IN ALL UPPERCASE."`

trim

`trim` returns a String with removed preceding and succeeding whitespace characters.

Usage

Availability	• Dialog functions
Parameters	
Output	String

Example	Sample Code
	extract sample
	`string = "    Trim this!    "`
	`string.trim()`
	`// ="Trim this!"`

urlEncode

`urlEncode` encodes a String for url consumption. Encodes all characters that are either illegal, or have any reserved meaning, anywhere within a URI, as defined in RFC 3986.

`urlEncode` is only available in dialog functions.

Usage

Availability	Dialog functions
Parameters	
Output	String

Example	Sample Code
	extract sample
	`string = "https://www.google.com/query=something?§param=asdf&something=anotherone"`
	`string.urlEncode()`
	`//="https%3A%2F%2Fwww.google.com%2Fquery%3Dsomething%3F%C2%A7param%3Dasdf%26something%3Danotherone"`

base64Encode

`Base64Encode` encodes a String into its Base64 representation.

`Base64Encode` is only available in dialog functions.

Usage

Availability	Dialog functions
Parameters	
Output	String

Example	Sample Code
	extract sample
	`string = "Encode this into Base64"`
	`string.base64Encode()`
	`//="RW5jb2RlIHRoaXMgaW50byBCYXNlNjQ="`

3.3.2.3.2.3 List Operations

The following operations are supported on lists.

addAll

`addAll` adds a second list to the source list, changing the content of the source list.

Usage

Availability	• Dialog functions
Parameters	List
Output	-

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list2 = [ "entry2", { "objectEntry2": false, "value": 35 }, -21, true ]`
	
	`list.addAll(list2)`
	`// no return value`
	
	`// list after execution:`
	`// list = [ "entry1", { "objectEntry": true, "value": 42 } , 10 , false , "entry2", { "objectEntry2": false, "value" : 35 } , -21 , true ]`

append

append returns the list with the added parameter. This does not change the source list.

Usage

Availability	• Dialog functions
Parameters	List element or list
Output	List

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	
	`list.append(128)`
	`// =[ "entry1", { "objectEntry": true, "value": 42 }, 10, false, 128 ]`
	
	`// list after execution:`
	`// =[ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`

clear

`clear` resets the list to an empty list.

Usage

Availability	• Dialog functions
Parameters	-
Output	-

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]  list.clear()`
	`// no return value`
	
	`// list after execution:`
	`// =[]`

contains

`contains` checks whether an element is present within the list. This does not work on objects.

Usage

Availability	• Dialog functions

Parameters	List element
Output	Boolean

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.contains("entry1") // =true`
	`list.contains(10) // =true`
	`list.contains("10") // =false`
	`list.contains(42) // =false`
	`list.contains(false) // =true`
	`list.contains({ "objectEntry": true, "value": 42 }) // =false`

containsIgnoreCase

`containsIgnoreCase` works similar as contains, except that String equality is case insensitive.

Usage

Availability	• Dialog functions
Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.contains("entry1") // =true`
	`list.contains("ENTRY1") // =true`
	`list.contains(10) // =true`
	`list.contains("10") // =false`
	`list.contains(42) // =false`
	`list.contains(false) // =true`
	`list.contains({ "objectEntry": true, "value": 42 }) // =false`

get

`get` returns the list element at the provided index position.

Usage

Availability	• Dialog functions
Parameters	Integer
Output	List element

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.get(0) // ="entry1"`
	`list.get(1) // ={ "objectEntry": true, "value": 42 }`
	`list.get(42) // no return value`

getRandomItem

`getRandomItem` returns a random list element from the list.

Usage

Availability	• Dialog functions
Parameters	-
Output	List element

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.getRandomItem() // returns random item from list`

indexOf

`indexOf` returns the index within the list for a provided list element.

Usage

Availability	• Dialog functions

Parameters	List element
Output	Integer

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.indexOf("entry1") // =0`
	`list.indexOf({ "objectEntry": true, "value": 42 }) // =1`
	`list.indexOf(10) // =2`
	`list.indexOf(false) // =3`
	`list.indexOf("does not exist) // =-1`

join

`join` transforms the list into a String with the provided parameter as a delimiter.

Usage

Availability	• Dialog functions
Parameters	String
Output	String

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.join(", ") // =entry1, { "objectEntry": true, "value": 42 }, 10, false`

remove

`remove` removes the list element at the provided index position from the list.

Usage

Availability	• Dialog functions
Parameters	Integer
Output	-

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.remove(0)`
	`// no return value`
	`// list after execution:`
	`// =[ { "objectEntry": true, "value": 42 }, 10, false ]`
	
	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.remove(1)`
	`// no return value`
	`// list after execution:`
	`// =[ "entry1", 10, false ]`

removeValue

`removeValue` removes all provided values from the list.

Usage

Availability	• Dialog functions
Parameters	List element
Output	-

Example	Sample Code
	extract sample
	``
	list = ["entry1",
	{ "objectEntry": true, "value":
	42 }, 10, false]
	
	list.removeValue(false)
	// no return value
	
	// list after execution:
	// =["entry1", { "objectEntry":
	true, "value": 42 }, 10]
	``

set

`set` overwrites the entry at the provided position with the new element.

Usage

Availability	• Dialog functions
Parameters	• Integer • List element
Output	-

Example	Sample Code
	extract sample
	``
	list = ["entry1",
	{ "objectEntry": true, "value":
	42 }, 10, false]
	
	list.set(1, "entry2")
	// no return value
	
	// list after execution:
	// =["entry1", "entry2", 10,
	false]
	``

size

`size` returns the number of entries within the list.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`list = [ "entry1", { "objectEntry": true, "value": 42 }, 10, false ]`
	`list.size()`
	`// =4`

3.3.2.3.2.4 Map Operations

The following map operations are supported.

clear

`clear` removes all entries from the map, returning it to an empty map.

Usage

Availability	• Dialog functions
Parameters	-
Output	-

Example	Sample Code
	extract sample
	`map = { "entry1": "value1", "object": { "objectEntry": true, "value": 42 }, "number": 10, "ready": false }`
	`map.clear()`
	`// no return value`
	`// map after execution:`
	`// ={}`

has

`has` checks whether a key exists within the provided map.

Usage

Availability	• Dialog functions
Parameters	String
Output	Boolean

Example	Sample Code
	extract sample
	`map = { "entry1": "value1", "object": { "objectEntry": true, "value": 42 }, "number": 10, "ready": false }`
	`map.has("entry1")`
	`// =true`
	`map.has("value1")`
	`// =false`
	`map.has("object")`
	`// =true`
	`map.has("objectEntry")`
	`// =false`
	`map.has("ready")`
	`// =true`

remove

`remove` deletes a map element with the provided key from the map.

Usage

Availability	• Dialog functions
Parameters	String
Output	

Example

Sample Code

extract sample

```
map = { "entry1": "value1",
"object": { "objectEntry": true,
"value": 42 }, "number": 10,
"ready": false }

map.remove("entry1")
// no return value

// map after execution:
// ={ "object": { "objectEntry":
true, "value": 42 }, "number": 10,
"ready": false }
```

Sample Code

extract sample

```
map = { "entry1": "value1",
"object": { "objectEntry": true,
"value": 42 }, "number": 10,
"ready": false }

map.remove("object")
// no return value

// map after execution:
// ={ "entry1": "value1",
"number": 10, "ready": false }
```

Sample Code

extract sample

```
map = { "entry1": "value1",
"object": { "objectEntry": true,
"value": 42 }, "number": 10,
"ready": false }

map.remove("value1")
// no return value

// map after execution:
// ={ "entry1": "value1",
"object": { "objectEntry": true,
"value": 42 }, "number": 10,
"ready": false }
```

3.3.2.3.2.5 Date Operations

The following date operations are supported:

getYear

`getYear` returns a value that is the result of subtracting 1900 from the year that contains or begins with the instant in time represented by this Date object, as interpreted in the UTC time zone.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getYear()`
	`// =199`

getMonth

`getMonth` returns a number representing the month that contains or begins with the instant in time represented by this Date object, as interpreted in the UTC time zone. The value returned is between 0 and 11, with the value 0 representing January.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getMonth()`
	`// =0`

getDate

`getDate` returns the day of the month represented by this Date object. The value returned is between 1 and 31 representing the day of the month that contains or begins with the instant in time represented by this Date object, as interpreted in the UTC time zone.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getDate()`
	`// =1`

getDay

`getDay` returns the day of the week represented by this date. The returned value (0 = Sunday, 1 = Monday, 2 = Tuesday, 3 = Wednesday, 4 = Thursday, 5 = Friday, 6 = Saturday) represents the day of the week that contains or begins with the instant in time represented by this Date object, as interpreted in the UTC time zone.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getDay()`
	`// =4`

getHours

`getHours` returns the hour represented by this Date object. The returned value is a number (0 through 23) representing the hour within the day that contains or begins with the instant in time represented by this Date object, as interpreted in the UTC time zone.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getHours()`
	`// =14`

getMinutes

`getMinutes` returns the number of minutes past the hour represented by this date, as interpreted in the UTC time zone. The value returned is between 0 and 59.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getMinutes()`
	`// =40`

getSeconds

`getSeconds` returns the number of seconds past the minute represented by this date, as interpreted in the UTC time zone. The value returned is between 0 and 59.

Usage

Availability	• Dialog functions
Parameters	-
Output	Integer

Example	Sample Code
	extract sample
	`future.getSeconds()`
	`// =41`

after

`after` checks whether a date is chronologically after the provided date.

Usage

Availability	• Dialog functions
Parameters	Date
Output	Boolean

Example	Sample Code
	extract sample
	`today.after(past)`
	`// =true`
	`today.after(future)`
	`// =false`
	`today.after(today)`
	`// =false`

before

`before` checks whether a date is chronologically before the provided date.

Usage

Availability	• Dialog functions

Parameters	Date
Output	Boolean

Example	Sample Code
	extract sample
	`today.before(past)`
	`// =false`
	
	`today.before(future)`
	`// =true`
	
	`today.after(today)`
	`// =false`

compareTo

`compareTo` is a method encompassing before, after and equals by providing a corresponding value.

Usage

Availability	• Dialog functions
Parameters	-
Output	Boolean

Example	Sample Code
	extract sample
	`today.compareTo(past)`
	`// >0`
	
	`today.compareTo(future)`
	`// <0`
	
	`today.compareTo(today)`
	`// =0`

getTime

`getTime` returns the milliseconds since the start of the unix epoch time (1st January 1970, 00:00 GMT/UTC).

Usage	
Availability	• Dialog functions
Parameters	-
Output	Long

Example	Sample Code
	extract sample
	`today.getTime()`
	`// equals milliseconds since start of unix epoch time`
	`past.getTime()`
	`// =0`
	`future.getTime()`
	`// =4070961641000`

3.3.2.3.3 Accessing Context Information

With the introduction of Joule Functions, a new approach to access the transient variables is defined. This improves the capability developer experience for transient data access in both GenAI based capabilities

Global Transient Variables accumulate the runtime transient variables such as:

- user
- client
- app_context
- input

in a static variable *$transient*.

Accessing $transient variables

The *$transient* variable is now accessible in the capability in the dialog functions throughout the execution of the capability.

Sample Code

Accessing of *$transient* variable via dialog function:

```
parameters:
  - name: name
action_groups:
  - actions:
      - type: message
        message:
          type: text
          content: "Here are the details for the train station:"
      - type: set-variables
        variables:
          - name: "country_code"
```

```
            value: "<? $transient.app_context["com.trains.appInfo"].country ?
>"
      - type: dialog-function
        name: train/station_lookup
        parameters:
          - name: query
            value: "<? name ?>"
          - name: conutry
            value: "<? country_code ?>"
        result_variable: station_details

result:
  station: <? station_details ?>
```

Implementation Hints

- The *$transient* variable is read-only and cannot be modified by a capability developer.
- Attributes present within the *$transient* variables can be changed per request cycle.
- Complexity of data in transient variables can be reduced by pre-processing the context with scripting before consumption.
- The $transient variable expanded by user utterance.

3.3.2.3.3.1 User Information

If user information is required in dialog functions, it has to be explicitly provided as a parameter to each individual dialog function.

Accessible Information

- email
- uuid
- first_name
- last_name

The fields are accessible with the path `$transient.user.<field_name>`. In case a value is not present, the entire field is absent. The presence of a field depends on the configuration of the connected IAS. When information is not provided, there will be an empty user node.

```
action_groups:
  - actions:
      - type: set-variables
        variables:
          - name: "user_email"
            value: "<? $transient.user.email ?>"
      - type: message
        message:
          type: text
          content: "Email: <? user_email ?>"
result:
  result_variable: <? user_email ?>
```

3.3.2.3.3.2 Client Information

The Joule client provides important information to capabilities, which can be used to improve dialog flow and provide better assistance to users.

Access Client Information

Developers of Joule capabilities can dynamically access client information and use it in dialog functions where Spring Expression Language (SpEL) scripting can be applied. All attributes of client information are transient; therefore, the values have a per-request lifecycle. Client information can be accessed at design time by developers in a field called `client` under the global transient variable.

Sample Code

```
action_groups:
  - actions:
      - type: set-variables
        variables:
          - name: "client_info"
            value: "<? $transient.client.client_type ?>"
      - type: message
        message:
          type: text
          content: "The client type is input: <? client_info ?>"
result:
  result_variable: <? client_info ?>
```

If client information is required for any one of the dialog functions, it must be provided specifically as a parameter for each dialog function.

The fields can be accessed with the path `$transient.client.<field_name>`. The availability of a field depends on the configuration of the connected client. In case a value is not present, the entire field is absent. If none of the information is available, it will be an empty user node.

Time Zones

The `timezone` attributes provide the time zone for a connected client. The following rules apply for the consumption of time zones in Joule:

- The value contains a standard IANA time zone of type string.
- If the time zone values are missing in the client, the request is processed using GMT.
- If an invalid time zone value is passed to Joule from a client, the request is processed using GMT.
- With a valid value, entities associated with date and time process values in accordance with the provided value.

Joule supports the following list of time zones:

Time Zone	Time Zone
Africa/Abidjan	Africa/Accra
Africa/Addis_Ababa	Africa/Algiers
Africa/Asmara	Africa/Asmera
Africa/Bamako	Africa/Bangui
Africa/Banjul	Africa/Bissau
Africa/Blantyre	Africa/Brazzaville
Africa/Bujumbura	Africa/Cairo
Africa/Casablanca	Africa/Ceuta
Africa/Conakry	Africa/Dakar
Africa/Dar_es_Salaam	Africa/Djibouti
Africa/Douala	Africa/El_Aaiun
Africa/Freetown	Africa/Gaborone
Africa/Harare	Africa/Johannesburg
Africa/Juba	Africa/Kampala
Africa/Khartoum	Africa/Kigali
Africa/Kinshasa	Africa/Lagos
Africa/Libreville	Africa/Lome
Africa/Luanda	Africa/Lubumbashi
Africa/Lusaka	Africa/Malabo
Africa/Maputo	Africa/Maseru
Africa/Mbabane	Africa/Mogadishu
Africa/Monrovia	Africa/Nairobi
Africa/Ndjamena	Africa/Niamey
Africa/Nouakchott	Africa/Ouagadougou
Africa/Porto-Novo	Africa/Sao_Tome

Time Zone	Time Zone
Africa/Timbuktu	Africa/Tripoli
Africa/Tunis	Africa/Windhoek
America/Adak	America/Anchorage
America/Anguilla	America/Antigua
America/Araguaina	America/Argentina/Buenos_Aires
America/Argentina/Catamarca	America/Argentina/ComodRivadavia
America/Argentina/Cordoba	America/Argentina/Jujuy
America/Argentina/La_Rioja	America/Argentina/Mendoza
America/Argentina/Rio_Gallegos	America/Argentina/Salta
America/Argentina/San_Juan	America/Argentina/San_Luis
America/Argentina/Tucuman	America/Argentina/Ushuaia
America/Aruba	America/Asuncion
America/Atikokan	America/Atka
America/Atikokan	America/Atka
America/Bahia	America/Bahia_Banderas
America/Barbados	America/Belem
America/Belize	America/Blanc-Sablon
America/Boa_Vista	America/Bogota
America/Boise	America/Buenos_Aires
America/Cambridge_Bay	America/Campo_Grande
America/Cancun	America/Caracas
America/Catamarca	America/Cayenne
America/Cayman	America/Chicago
America/Chihuahua	America/Coral_Harbour
America/Cordoba	America/Costa_Rica
America/Creston	America/Cuiaba

Time Zone	Time Zone
America/Curacao	America/Danmarkshavn
America/Dawson	America/Dawson_Creek
America/Denver	America/Detroit
America/Dominica	America/Edmonton
America/Eirunepe	America/El_Salvador
America/Ensenada	America/Fort_Nelson
America/Fort_Wayne	America/Fortaleza
America/Glace_Bay	America/Godthab
America/Goose_Bay	America/Grand_Turk
America/Grenada	America/Guadeloupe
America/Guatemala	America/Guayaquil
America/Guyana	America/Halifax
America/Havana	America/Hermosillo
America/Indiana/Indianapolis	America/Indiana/Knox
America/Indiana/Marengo	America/Indiana/Petersburg
America/Indiana/Tell_City	America/Indiana/Vevay
America/Indiana/Vincennes	America/Indiana/Winamac
America/Indianapolis	America/Inuvik
America/Iqaluit	America/Jamaica
America/Jujuy	America/Juneau
America/Kentucky/Louisville	America/Kentucky/Monticello
America/Knox_IN	America/Kralendijk
America/La_Paz	America/Lima
America/Los_Angeles	America/Louisville
America/Lower_Princes	America/Maceio
America/Managua	America/Manaus

Time Zone	Time Zone
America/Marigot	America/Martinique
America/Matamoros	America/Mazatlan
America/Mendoza	America/Menominee
America/Merida	America/Metlakatla
America/Mexico_City	America/Miquelon
America/Moncton	America/Monterrey
America/Montevideo	America/Montreal
America/Montserrat	America/Nassau
America/New_York	America/Nipigon
America/Nome	America/Noronha
America/North_Dakota/Beulah	America/North_Dakota/Center
America/North_Dakota/New_Salem	America/Ojinaga
America/Panama	America/Pangnirtung
America/Paramaribo	America/Phoenix
America/Port-au-Prince	America/Port_of_Spain
America/Porto_Acre	America/Porto_Velho
America/Puerto_Rico	America/Rainy_River
America/Rankin_Inlet	America/Recife
America/Regina	America/Resolute
America/Rio_Branco	America/Rosario
America/Santa_Isabel	America/Santarem
America/Santiago	America/Santo_Domingo
America/Sao_Paulo	America/Scoresbysund
America/Shiprock	America/Sitka
America/St_Barthelemy	America/St_Johns
America/St_Kitts	America/St_Lucia

Time Zone	Time Zone
America/St_Thomas	America/St_Vincent
America/Swift_Current	America/Tegucigalpa
America/Thule	America/Thunder_Bay
America/Tijuana	America/Toronto
America/Tortola	America/Vancouver
America/Virgin	America/Whitehorse
America/Winnipeg	America/Yakutat
America/Yellowknife	Antarctica/Casey
Antarctica/Davis	Antarctica/DumontDUrville
Antarctica/Macquarie	Antarctica/Mawson
Antarctica/McMurdo	Antarctica/Palmer
Antarctica/Rothera	Antarctica/South_Pole
Antarctica/Syowa	Antarctica/Troll
Antarctica/Vostok	Arctic/Longyearbyen
ART	AST
Asia/Aden	Asia/Almaty
Asia/Amman	Asia/Anadyr
Asia/Aqtau	Asia/Aqtobe
Asia/Ashgabat	Asia/Ashkhabad
Asia/Baghdad	Asia/Bahrain
Asia/Baku	Asia/Bangkok
Asia/Beirut	Asia/Bishkek
Asia/Brunei	Asia/Calcutta
Asia/Chita	Asia/Choibalsan
Asia/Chongqing	Asia/Chungking
Asia/Colombo	Asia/Dacca

Time Zone	Time Zone
Asia/Damascus	Asia/Dhaka
Asia/Dili	Asia/Dubai
Asia/Dushanbe	Asia/Gaza
Asia/Harbin	Asia/Hebron
Asia/Ho_Chi_Minh	Asia/Hong_Kong
Asia/Hovd	Asia/Irkutsk
Asia/Istanbul	Asia/Jakarta
Asia/Jayapura	Asia/Jerusalem
Asia/Kabul	Asia/Kamchatka
Asia/Karachi	Asia/Kashgar
Asia/Kathmandu	Asia/Katmandu
Asia/Khandyga	Asia/Kolkata
Asia/Krasnoyarsk	Asia/Kuala_Lumpur
Asia/Kuching	Asia/Kuwait
Asia/Macao	Asia/Macau
Asia/Magadan	Asia/Makassar
Asia/Manila	Asia/Muscat
Asia/Nicosia	Asia/Novokuznetsk
Asia/Novosibirsk	Asia/Omsk
Asia/Oral	Asia/Phnom_Penh
Asia/Pontianak	Asia/Pyongyang
Asia/Qatar	Asia/Qyzylorda
Asia/Rangoon	Asia/Riyadh
Asia/Saigon	Asia/Sakhalin
Asia/Samarkand	Asia/Seoul
Asia/Shanghai	Asia/Singapore

Time Zone	Time Zone
Asia/Srednekolymsk	Asia/Taipei
Asia/Tashkent	Asia/Tbilisi
Asia/Tehran	Asia/Tel_Aviv
Asia/Thimbu	Asia/Thimphu
Asia/Tokyo	Asia/Ujung_Pandang
Asia/Ulaanbaatar	Asia/Ulan_Bator
Asia/Urumqi	Asia/Ust-Nera
Asia/Vientiane	Asia/Vladivostok
Asia/Yakutsk	Asia/Yekaterinburg
Asia/Yerevan	Atlantic/Azores
Atlantic/Bermuda	Atlantic/Canary
Atlantic/Cape_Verde	Atlantic/Faeroe
Atlantic/Faroe	Atlantic/Jan_Mayen
Atlantic/Madeira	Atlantic/Reykjavik
Atlantic/South_Georgia	Atlantic/St_Helena
Atlantic/Stanley	Australia/ACT
Australia/Adelaide	Australia/Brisbane
Australia/Broken_Hill	Australia/Canberra
Australia/Currie	Australia/Darwin
Australia/Eucla	Australia/Hobart
Australia/LHI	Australia/Lindeman
Australia/Lord_Howe	Australia/Melbourne
Australia/NSW	Australia/North
Australia/Perth	Australia/Queensland
Australia/South	Australia/Sydney
Australia/Tasmania	Australia/Victoria

Time Zone	Time Zone
Australia/West	Australia/Yancowinna
BET	BST
Brazil/Acre	Brazil/DeNoronha
Brazil/East	Brazil/West
Canada/Atlantic	Canada/Central
Canada/East-Saskatchewan	Canada/Eastern
Canada/Mountain	Canada/Newfoundland
Canada/Pacific	Canada/Saskatchewan
Canada/Yukon	Chile/Continental
CAT	CNT
CET	CST6CDT
Chile/EasterIsland	Cuba
CST	CTT
EAT	ECT
EET	EST5EDT
Egypt	Eire
EST	HST
Etc/GMT	Etc/GMT+0
Etc/GMT+1	Etc/GMT+10
Etc/GMT+2	Etc/GMT+3
Etc/GMT+4	Etc/GMT+5
Etc/GMT+6	Etc/GMT+7
Etc/GMT+8	Etc/GMT+9
Etc/GMT+11	Etc/GMT+12
Etc/GMT-0	Etc/GMT-1
Etc/GMT-3	Etc/GMT-4

Time Zone	Time Zone
Etc/GMT-5	Etc/GMT-6
Etc/GMT-7	Etc/GMT-8
Etc/GMT-9	Etc/GMT0
Etc/GMT-10	Etc/GMT-11
Etc/GMT-12	Etc/GMT-13
Etc/GMT-14	Etc/GMT-2
Etc/Greenwich	Etc/UCT
Etc/UTC	Etc/Universal
Etc/Zulu	Europe/Amsterdam
Europe/Andorra	Europe/Athens
Europe/Belfast	Europe/Belgrade
Europe/Berlin	Europe/Bratislava
Europe/Brussels	Europe/Bucharest
Europe/Budapest	Europe/Busingen
Europe/Chisinau	Europe/Copenhagen
Europe/Dublin	Europe/Gibraltar
Europe/Guernsey	Europe/Helsinki
Europe/Isle_of_Man	Europe/Istanbul
Europe/Jersey	Europe/Kaliningrad
Europe/Kiev	Europe/Lisbon
Europe/Ljubljana	Europe/London
Europe/Luxembourg	Europe/Madrid
Europe/Malta	Europe/Mariehamn
Europe/Minsk	Europe/Monaco
Europe/Moscow	Europe/Nicosia
Europe/Oslo	Europe/Paris

Time Zone	Time Zone
Europe/Podgorica	Europe/Prague
Europe/Riga	Europe/Rome
Europe/Samara	Europe/San_Marino
Europe/Sarajevo	Europe/Simferopol
Europe/Skopje	Europe/Sofia
Europe/Stockholm	Europe/Tallinn
Europe/Tirane	Europe/Tiraspol
Europe/Uzhgorod	Europe/Vaduz
Europe/Vatican	Europe/Vienna
Europe/Vilnius	Europe/Volgograd
Europe/Warsaw	Europe/Zagreb
Europe/Zaporozhye	Europe/Zurich
GB	GB-Eire
GMT	GMT0
Greenwich	Hongkong
Iceland	Indian/Antananarivo
Indian/Chagos	Indian/Christmas
Indian/Cocos	Indian/Comoro
Indian/Kerguelen	Indian/Mahe
Indian/Maldives	Indian/Mauritius
Indian/Mayotte	Indian/Reunion
Iran	Israel
Jamaica	Japan
Kwajalein	Libya
MET	MST7MDT
Mexico/BajaNorte	Mexico/BajaSur

Time Zone	Time Zone
Mexico/General	NZ
NZ-CHAT	Navajo
PRC	PST8PDT
Pacific/Apia	Pacific/Auckland
Pacific/Bougainville	Pacific/Chatham
Pacific/Chuuk	Pacific/Easter
Pacific/Efate	Pacific/Enderbury
Pacific/Fakaofo	Pacific/Fiji
Pacific/Funafuti	Pacific/Galapagos
Pacific/Gambier	Pacific/Guadalcanal
Pacific/Guam	Pacific/Honolulu
Pacific/Johnston	Pacific/Kiritimati
Pacific/Kosrae	Pacific/Kwajalein
Pacific/Majuro	Pacific/Marquesas
Pacific/Midway	Pacific/Nauru
Pacific/Niue	Pacific/Norfolk
Pacific/Noumea	Pacific/Pago_Pago
Pacific/Palau	Pacific/Pitcairn
Pacific/Pohnpei	Pacific/Ponape
Pacific/Port_Moresby	Pacific/Rarotonga
Pacific/Saipan	Pacific/Samoa
Pacific/Tahiti	Pacific/Tarawa
Pacific/Tongatapu	Pacific/Truk
Pacific/Wake	Pacific/Wallis
Pacific/Yap	Poland
Portugal	ROK

Time Zone	Time Zone
Singapore	SystemV/AST4
SystemV/AST4ADT	SystemV/CST6
SystemV/CST6CDT	SystemV/EST5
SystemV/EST5EDT	SystemV/HST10
SystemV/MST7	SystemV/MST7MDT
SystemV/PST8	SystemV/PST8PDT
SystemV/YST9	SystemV/YST9YDT
Turkey	UCT
US/Alaska	US/Aleutian
US/Arizona	US/Central
US/East-Indiana	US/Eastern
US/Hawaii	US/Indiana-Starke
US/Michigan	US/Mountain
US/Pacific	US/Pacific-New
US/Samoa	UTC
Universal	W-SU
WET	Zulu
EST	HST
MST	ACT
AET	AGT
ART	AST
BET	BST
CAT	CNT
CST	CTT
EAT	ECT
IET	IST

Time Zone	Time Zone
JST	MIT
NET	NST
PLT	PNT
PRT	PST
SST	VST

Client Type

The `client_type` attribute differentiates between requests from a native mobile client and requests from a web-based client. This attribute can have one of the following values:

- `webclient`: for a request from a web-based client
- `mobile`: for a request from a native mobile client

Platform

The `platform` attribute specifies the platform from which the request is made. This attribute can have one of the following values:

- `browser`: when the request is made from a browser
- `ios`: when the request is made from a native iOS client
- `android`: when the request is made from a native Android client

If the client does not provide a value for the platform attribute, its value defaults to unknown.

Device Type

The `device_type` attribute specifies from which type of device the request is made. This attribute can have one of the following values:

- `desktop`
- `phone`
- `tablet`

Client Version

The `client_version` attribute is optional and can contain the semantic version of the mobile SDK used at the time of the request, for example, `24.11.01`.

3.3.2.3.3.3 User Input

Information about the user input can be dynamically accessed. The information can be used everywhere in dialog functions where SpEL can be applied.

Sample Code

Example Dialog Function

```
action_groups:
  - actions:
      - type: set-variables
        variables:
          - name: "user_input"
            value: "<? $transient.input.text.raw ?>"
      - type: message
        message:
          type: text
          content: "User input: <? user_input ?>"
result:
  result_variable: <? user_input ?>
```

The user input is available in the scripting context of a dialog function in the transient section under *$transient* with the following structure:

```
"input": {
    "text": {
        "raw": "Hi"
    }
}
```

- **input.text.raw** is available as part of transient variables.
- It can be used by the capability developer to allow advanced use cases and processing. For example, by leveraging an LLM.
- The variable **input** is optional.

3.3.2.4 Messages

Messages can be used in dialog functions to display information to the user. In order to display dynamic information, Spring Expression Language (SpEL) as well as handlebars scripting can be used to generate the message. For more information about supported scripting, see Scripting [page 80].

Example in a dialog function:

Sample Code

```
action_groups:
  - actions:
    - type: message
      message:
        type: text
        content: "Hi There!"
```

Text

1. Type the text message that should be your assistant's response to the user.
2. You can enable Markdown syntax to format your text. For more information, see https://commonmark.org/help/ .
3. You can add a delay of up to 5 seconds between each message in a group of messages.

Sample Code

```
- actions:
    - type: message
      message:
        type: text
        content: Hello User!
        markdown: false
        delay: 3
```

Cards

1. Provide the title, subtitle, and add an image URL (title and subtitle are mandatory).
2. Add an interactive button to:
   1. Postback: Add a postback to send back to the /dialog API or a URI that is to be opened.
   2. Phone Number: Provide a phone number. Tapping or clicking this number triggers a phone call using the default calling application. This feature is available for specific channels.
   3. Navigation: Configure an external navigation. This button type supports intent-based navigation or can be used to open full URLs in a new tab.
3. You can add a delay of up to 5 seconds between each message in a group of messages.

You can also add additional details on the card like description, status text, and detail section.

Sample Code

```
- actions:
    - type: message
      message:
        type: card
        delay: 3
        content:
          title: <title>
```

```
          subtitle: <subtitle>
          imageUrl: <image url>
          imageStyle: <default|avatar>
          enableDetailView: <true|false>
          buttons:
            - type: postback
              title: <button title>
              text: >-
                <optional text that should be displayed in the chat on user
                action>
              value: <postback value sent to the assistant for processing>
            - type: phonenumber
              title: <button title>
              value: <phone number>
            - type: navigation
              title: <button title>
              navigation_target:
                url: <navigation url>
```

You can also add additional details on the card like description, status text, and detail section.

Sample Code

```
- actions:
  - message:
      type: card
      content:
        title: This is the card title
        subtitle: This is the subtitle
        imageUrl: 'sap-icon://product'
        imageStyle: avatar
        description:
          value: >-
            You can now add a **description** to the card for more information
            if needed
          markdown: true
          linesInDetailView: 7
        status: Product
        statusState: success
        enableDetailView: false
        hideHeaderInDetailView: false
        sections:
          - title: Section 1 Title
            attributes:
              - label: Label (Text value)
                value: >-
                  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
                  do eiusmod tempor incididunt ut labore et dolore magna
aliqua.
                  Ut enim ad minim veniam, quis nostrud exercitation ullamco
                  laboris nisi ut aliquip ex ea commodo consequat. Duis aute
                  irure dolor in reprehenderit in voluptate velit esse cillum
                  dolore eu fugiat nulla pariatur. Excepteur sint occaecat
                  cupidatat non proident, sunt in culpa qui officia deserunt
                  mollit anim id est laborum.  Lorem ipsum dolor sit amet,
                  consectetur adipiscing elit, sed do eiusmod tempor
incididunt
                  ut labore et dolore magna aliqua. Ut enim ad minim veniam,
                  quis nostrud exercitation ullamco laboris nisi ut aliquip ex
                  ea commodo consequat. Duis aute irure dolor in reprehenderit
                  in voluptate velit esse cillum dolore eu fugiat nulla
                  pariatur. Excepteur sint occaecat cupidatat non proident,
sunt
                  in culpa qui officia deserunt mollit anim id est laborum.
              - label: Label (Link value)
                value: SAP
```

```
                  url: 'https://www.sap.com'
                  type: link
                - label: Markdown Attribute
                  value: >-
                    **Answer 1:** When transferring within the group, for
example
                    from SAP Germany SE to SAP SE, you need to know the
following:
                    You will receive a new employment contract from the hiring
                    company.  You don't have to submit a workflow through
                    SuccessMap in case of a National Transfer. You have to
submit
                    a signed (wet signature) resignation letter in original via
                    inhouse-mail or regular mail to the HR department (SAP SE,
                    PERS02, Dietmar-Hopp-Allee 16, 69190 Walldorf). Please note
                    that due to legal regulation for Germany a signed
resignation
                    letter in original has to be sent to the employer. The
                    resignation letter should include a comment that it is a
                    national transfer.
                  markdown: true
          buttons:
            - type: postback
              title: Post back button
              text: Post back display text
              value: hi
              imageUrl: 'sap-icon://travel-request'
            - type: clientdata
              title: Button (Client Side Only)
              value: 'https://sap.com'
              client_data:
                ItemClientOnly1: Data for item one
                ItemClientOnly2: data for item 2
            - type: navigation
              title: Display Service Contracts
              navigation_target:
                ibnParams:
                  P_DisplayCurrency: EUR
                ibnTarget:
                  semanticObject: ServiceContract
                  action: displayFactSheet
```

Buttons

1. Type your assistant's message asking the user to choose an appropriate option.
2. Add an interactive button to as described in the Card section.
3. You can add a delay of up to 5 seconds between each message in a group of messages.

Sample Code

```
- actions:
    - type: message
      message:
        - type: buttons
          content: null
          title: <title>
          buttons:
            - type: <postback|clientdata|phonenumber|navigation> # web_url is
deprecated
              title: <reply title>
              text: <display text> # used only when type is postback
```

```
                value: <reply value> # required for postback or phonenumber
                client_data: null # defined only when type is clientdata
                navigation_target: <navigation target> # defined only when type
is navigation
                <key>: <value>
                delay: 3
```

Example with additional button types:

Sample Code

```
- actions:
    - type: message
      message:
        - type: buttons
          delay: 0
          content:
            title: Button List
            buttons:
              - type: clientdata
                title: Button (Client Side Only)
                value: 'https://sap.com'
                client_data:
                  key1: Data for key1
                  key2: data for key2
              - type: postback
                title: "Post back"
                text: "Post back display text"
                value: "Post back value"
                imageUrl: 'sap-icon://learning-assistant'
              - type: phonenumber
                title: Call someone
                value: '1231231234'
              - type: navigation
                title: Open example.com
                navigation_target: null
                url: 'https://example.com'
              - type: navigation
                title: Display Service Contracts (IBN with fallback URL)
                navigation_target:
                  ibnParams:
                    P_DisplayCurrency: EUR
                  ibnTarget:
                    semanticObject: ServiceContract
                    action: displayFactSheet
                  url: <optional fallback URL>
```

Quick Replies

Provide the following information:

1. The assistant's message asking the user to choose an appropriate option (optional).
2. Quick reply text that will be displayed to the user. You can add up to 11 quick replies.
3. Quick reply value that is sent back for further action.

Note

Quick Replies should always be the last message in a response, else the options will not be displayed.

Sample Code

```
actionGroups:
  - actions:
    - type: message
      message:
        type: quickReplies
        content:
          title: <title>
          buttons:
            - title: <reply title>
              value: <reply value>
          delay: 3
```

Carousel

This is a list of basic cards aligned vertically. Follow the steps mentioned for creating a card.

Sample Code

```
- actions:
    - type: message
      message:
        type: carousel
        content:
          enableDetailView: <true|false>
          cards:
            - title: <title>
              subtitle: <subtitle>
              imageUrl: <image url>
              buttons:
                - type: postback
                  title: <button title>
                  text: <optional display text>
                  value: <reply value>
            - title: <title>
              subtitle: <subtitle>
              imageUrl: <image url>
              buttons:
                - type: phonenumber
                  title: <title>
                  value: <phone number>
        delay: 3
```

You can also add additional details on each card like description, status text, and detail section. Supports all button types defined for a button response type. You can use the same format that is used for creating a button.

Sample Code

```
- actions:
    - type: message
      message:
        type: carousel
        delay: 0
        content:
          cards:
            - title: Title 1
```

```
              subtitle: Here is the subtitle
              description: This is a description line here.
              buttons:
                - type: postback
                  title: Post back title
                  text: Post back display text
                  value: post back value
                - type: clientdata
                  title: Button (Client Side Only)
                  value: 'https://sap.com'
                  client_data:
                    key1: Data for key1
                    key2: data for key2
              status: Status
              statusState: success
              imageUrl: >-
                https://www.sap.com/dam/application/shared/logos/sap-logo-
svg.svg.adapt.svg/1493030643828.svg
              sections:
                - title: Section Title 1
                  attributes:
                    - url: 'https://www.sap.com'
                      value: text
                      type: link
                      label: Item Label Link (Client_data)
                    - type: clientdata
                      label: Item Label ClientSideOnly (Client_data)
                      value: 'https://sap.com'
                      client_data:
                        key1: Data for kay1
                        key2: data for key2
            - title: Title 2
              subtitle: Subtitle
              description: >-
                You can now add a description to the card for more
information if
                needed.
              imageUrl: >-
                https://www.sap.com/dam/application/shared/logos/sap-logo-
svg.svg.adapt.svg/1493030643828.svg
              hideHeaderInDetailView: true
              buttons:
                - value: demo 1
                  title: Postback
                  type: postback
```

List

1. Add a title, subtitle, and an image URL (all optional).
2. For each list item, add a title (mandatory), subtitle, description, and an image URL. Only the title is mandatory.
3. Add a button on each list item for the users to make a selection. Supports all button types defined for a button response type. You can use the same format that is used for creating a button.
4. You may add a button for the overall list, for example, to open the full list in your application or website.
5. You can display the list in a collapsible panel container by using the `displayInPanel` field.
6. You can add a delay of up to 5 seconds between each message in a group of messages.

Sample Code

```
- actions:
    - type: message
      message:
        type: list
        content:
          title: <title>
          subtitle: <subtitle>
          enableDetailView: <true|false>
          imageUrl: <image url>
          imageStyle: <default|avatar>
          elements:
            - title: <element title>
              subtitle: <element subtitle>
              imageUrl: <element image url>
              imageStyle: <default|avatar>
              description: <element description text>
              buttons:
                - type: postback
                  title: <reply title>
                  text: <optional display text>
                  value: <reply value>
        delay: 3
```

Example with a detail section:

Sample Code

```
- actions:
    - type: message
      message:
        type: list
        content:
          title: Header Title
          total: 13
          subtitle: Subtitle
          description: This is the description line.
          buttons:
            - type: postback
              title: Post back title
              value: Post back value
          elements:
            - title: Title 1
              subtitle: Subtitle 1
              imageUrl: >-
                https://www.sap.com/dam/application/shared/logos/sap-logo-
svg.svg.adapt.svg/1493030643828.svg
              imageStyle: avatar
              buttons:
                - type: web_url
                  title: Button 1 (link)
                  value: 'https://google.com'
                - type: postback
                  title: Button 2 (Postback)
                  text: Postback display text
                  value: hi
                - type: clientdata
                  title: Button (Client Side Only)
                  client_data:
                    ItemClientOnly1: Data for item one
                    ItemClientOnly2: data for item 2
                - type: phonenumber
                  title: Button 3 (Phone)
                  value: '11231231234'
```

```
                - type: navigation
                  title: Display Service Contracts (IBN without fallback URL)
                  navigation_target:
                    ibnParams:
                      P_DisplayCurrency: EUR
                    ibnTarget:
                      semanticObject: ServiceContract
                      action: displayFactSheet
            - title: Title 2
              subtitle: Subtitle 2
              description: This is a description line (3 buttons)
              buttons:
                - type: web_url
                  title: Button 1 (link)
                  value: 'https://google.com'
            - title: Title 3
              subtitle: Subtitle 3
              buttons:
                - type: postback
                  title: Post back title
                  value: Post back value
              sections:
                - title: Section 1 Title
                  attributes:
                    - label: Label (Text value)
                      value: Display text
                    - url: 'https://www.sap.com'
                      type: link
                      value: text
                      label: Label (Link value)
                    - label: Label 3
                      value: This is text 3
            - title: Title 4
              subtitle: Subtitle 4
              description: Here is the description.
```

Example for a list displayed in a panel.

Sample Code

```
- actions:
      type: message
      message:
        - type: list
          delay: 0
          content:
            displayInPanel: true
            panel:
              collapsed: true
            title: Search results
            enableDetailView: false
            elements:
              - imageUrl: 'sap-icon://question-mark'
                title:
                  value: >-
                    [Create Sales Orders Using Automatic
                    Extraction](https://google.com)
                  markdown: true
                description: >-
                  Create Sales Orders - Automatic extraction App 10. +4920
with
                  this app, you as an internal sales representative can create
                  sales orders from purchase order files in PDF or image which
                  is a built-end feature.
              - title:
                  value: >-
```

```
                    [Create Sales Orders Automatic
                    Extraction](https://google.com)
                  markdown: true
                imageUrl: 'sap-icon://activatesap-icon://learning-assistant'
                description: >-
                  Create Sales Orders - Automatic extraction App 10. +4920
with
                  this app, you as an internal sales representative can create
                  sales orders from purchase order files in PDF or image which
                  is a built-end feature.
              - title:
                  value: >-
                    [Create Sales Orders with PDF Automatic
                    Extraction](https://google.com)
                  markdown: true
                imageUrl: 'sap-icon://customer'
                description: >-
                  Create Sales Orders - Automatic extraction App 10. +4920
with
                  this app, you as an internal sales representative can create
                  sales orders from purchase order files in PDF or image which
                  is a built-end feature.
```

Media

Provide a URL of an image or video that should be displayed to the user. You can also add a delay of up to 5 seconds between each message in a group of messages.

Limitations:

- URLs for image or video should be public.
- Supported URLs:
  - The supported video formats are - *MP4*, *WEBM*, *OGV*, and *3GP*.
  - The video URLs that are currently supported are - *YouTube*, *Vimeo*, *Daily Motion*, *SAP Video*, and *SAP TV Video*.

Sample Code

```
- actions:
    - type: message
      message:
        type: media
        content: <media url>
        delay: 3
```

Client Data

The content of client data messages is not displayed directly on the screen. This message type allows you to define a JSON key-value pair that is returned to the web client.

Sample Code

```
- actions:
    - type: message
      message:
        type: client_data
        content:
          elements:
            client_action:
              fiori:
                navigate:
                  semanticObject: <semanticObject>
                  action: <action>
                  params:
                    key1: value1
                  appStateData: {}
```

UI Integration Card

Add a UI integration card `manifest` to the content to display a non-actionable UI5 integration card.

Sample Code

```
- actions:
    - type: message
      message:
        type: ui5integrationCard
        content:
          manifest: <UI Integration card manifest>
          submit_message: >-
            <Optional submit message to be sent in the chat on Submit of an
            interactive integration card>
        delay: 3
```

Illustrated Message

Provide a URL of an image or video that should be displayed to the user. You can also add **Define an illustrated message** in scenarios where you would like to show a success message or an error message as a result of a transaction. You can also use this message to display a no-data state to your users with a nuanced actions to guide the user to the next step in the conversation. Navigation can be added via call to action buttons to help user keep track of important actions they can take outside of the assistant or within the surrounding application where your assistant is integrated.

Illustrated messages are informative cards consisting of illustrations, a header title, a subtitle and up to two call to action buttons.

Sample Code

```
- actions:
    - type: message
      message:
```

```
        type: illustrated_message
        content:
          illustration_name: tntSuccess
          title: This is the illustrated_message title
          subtitle: This is the illustrated_message subtitle
          buttons:
            - title: <button title>
              type: clientdata
              client_data:
                <key>: <value>
            - title: <button title>
              type: postback
              text: <display text>
              value: <postback value>
```

Likert Scale

Define the Likert Scale message for a Likert Slot Filling option in the scenario:

Sample Code

scenarios/joule_satisfaction.yaml

```
description: This scenario captures the user's satisfaction level with the
Joule platform.
slots:
  - name: joule_fit_for_purpose
    description: The user's perception of whether the Joule platform does
what they need it to do, rated on a scale from 1 (Very Poor) to 5 (Very Good).
target:
  type: function
  name: scenario_joule_survey
```

The likert_scale message type enables Joule Content Developers to configure pointer scales, labels, and values at design time. It aims to ensure proper rendering of Likert scales in the Joule Web Client.

The supported range is from 1 to 7. The compiler will not enforce validation for contiguous values (i.e., allowing gaps between Likert option values) or prevent value duplication (i.e., permitting multiple options with the same value). This approach provides survey designers with greater flexibility to configure their Likert scales according to specific requirements.

Sample Code

functions/scenario_joule_survey.yaml

```
type: message
message:
    type: likert_scale
    content: # (Required) Main configuration block for the Likert scale
element
        prompt: 'Joule does what I need it to do'
        options: # Required; array of scale options. Minimum: 1, Maximum: 7
items.
            - label: 'Strongly disagree' # (Required) Text shown on the
button for this option.
              value: 1
            - label: Disagree
              value: 2
```

```
            - label: Neutral
              value: 3
            - label: Agree
              value: 4
            - label: 'Strongly agree'
              value: 5
```

If the user select 4, then the slot *joule_fit_for_purpose* should now contain the value 4 and be usable in downstream parts of the dialog function, i.e:

Sample Code

Usage Example of the Likert Slot Filling Parameter

```
- condition: joule_fit_for_purpose != null
  actions:
    - type: api-request
      method: POST
      system_alias: JouleFeedbackService
      path: api/joule-feedback
      headers:
        Content-Type: application/json
      body: { "fit_for_purpose": "{{joule_fit_for_purpose}}" }
    - type: message
      scripting_type: handlebars
      message:
        type: text
        content: "Thank you for your feedback on Joule! You rated its fit for
purpose as {{joule_fit_for_purpose}}."
```

3.3.2.4.1 UI Integration Cards

You can define a UI integration card manifest as a response in your assistant.

A UI integration card can be used to respond to the user with a non-actionable card that presents data in a rich UI format, using different card types.

For more information about the available card types, see UI Integration Card.

Data Binding

You can use scripting to bind context data in your assistant to the UI Integration card. For more information, see Scripting [page 80]. You can define the data binding in the following ways:

- Bind the context data to the `json` property of UI5 integration card and use the UI Integration card binding syntax for the individual UI elements defined in the manifest.
- Use the DA binding syntax to bind context directly to individual UI elements defined in the manifest.

Note

Dynamic data binding to the card using destination configurations inside the manifest is not supported. Use Dialog Functions [page 58] to make remote API calls and bind the data as defined above.

- Sample Code

```
type: ui5integrationCard
content:
  submit_message: >-
    Request product update of <? $mock_data.product.ProductName ?> to
    {submitted_data.ProductName}
  manifest: |
    {
                  "_version": "1.17.0",
                  "sap.app": {
                    "type": "card",
                    "id": "card.explorer.adaptive.extensionSample"
                  },
                  "sap.card": {
                    "data": {
                      "json": "<? $product_data ?>"
                    },
                    "type": "AdaptiveCard",
                    "header": {
                      "title": "Edit Product"
                    },
                    "content": {
                      "schema": "http://adaptivecards.io/schemas/adaptive-
card.json",
                      "type": "AdaptiveCard",
                      "version": "1.0",
                      "body": [
                        {
                          "type": "Input.Text",
                          "label": "Product Name",
                          "value": "${product.ProductName}",
                          "style": "text",
                          "id": "ProductName"
                        },
                        {
                          "type": "Input.ChoiceSet",
                          "label": "Product Category",
                          "id": "CategoryID",
                          "style": "compact",
                          "isMultiSelect": false,
                          "value": "${product.CategoryID}",
                          "choices": [
                            {
                              "data": "${categories}",
                              "title": "${CategoryName}",
                              "value": "${CategoryID}"
                            }
                          ]
                        },
                        {
                          "type": "Input.Text",
                          "label": "Supplier Name",
                          "value": "<?
$mock_data.product.supplier.CompanyName ?>",
                          "style": "text",
                          "id": "CompanyName"
                        }
                      ],
                      "actions": [
                        {
                          "type": "Action.Submit",
                          "title": "Submit"
                        }
                      ]
                    }
```

```
                    }
```

Card Actions

For information about card action types, see Card Actions.

> Note
>
> - The **navigation action type** (`Navigation`) supports only absolute URLs that open in a new tab.
> - The custom action type (`Custom`) is not supported.

Restrictions

- If you are using the `data` property of the integration card to bind data to the card, only the static data binding json is supported.
- Action Handlers inside the manifest are not supported.
- Component card types cannot be defined as a response in your assistant.
- Filters and user defined manifest parameters are not supported.
- Dynamic data fetching on the front end via destination declarations in the manifest are not supported.

3.3.2.4.2 Configure Cross-Product Navigation

You can configure navigation to business applications or external application in Joule responses. This helps in designing conversational scenarios that allows end users to navigate to specific applications that provide additional context to the ongoing conversation or complete complex transactions inside the application that is launched.

A button type called `navigation` is supported to configure such interactions. This button type is supported within the following UI element responses within Joule.

- Buttons
- Cards
- List
- Carousel
- UI Integration Cards support Navigation action out of the box. See format.

Setup for Cross Product Navigation

Joule platform uses the Navigation service to resolve the IBN targets defined in capabilities.

> **Note**
>
> As of DTA Specification version 3.20, the `url` property is no longer supported, and the resolution of navigation targets via direct calls to the Navigation Service within dialog functions in capabilities is no longer allowed. If the `url` property is used in a capability, the Joule platform will not resolve the target, but instead navigate to the URL. For Joule on mobile, targets are resolved for the mobile device type.

Definition of Navigation Button Type

Attribute	Type	Description	Required	Allowed Values
type	string	type of the button	true	navigation
navigation_target	navigation_target (see Navigation Target) [page 173].	Represents the navigation target that should be launched when the button is clicked. This case can be used to perform either an intent based navigation or navigation to a URL in a new tab.	true	

Navigation Target

Use this to define a configuration for the navigation action and perform either an intent based navigation or navigation to a URL in a new tab.

- **Define an intent based navigation**
  The intent-based navigation mechanism allows launching applications in-place using semantic objects and actions. For this to work, you must maintain the exact parameters, and ensure that the host application where Joule is embedded supports in-place navigation to intents.

  > **Note**
  >
  > In-place intent based navigation is supported by Joule in SAP S/4HANA (Fiori Launchpad). For other business applications, check the documentation for onNavigation callback in Web Client bridge. If the callback is not implemented by the host application, Joule will always open the resolved url in a new tab.

Attribute	Type	Description	Required	Allowed Values
ibnTarget	object	Intent-based navigation target	true	navigation
ibnParams	object	Map of parameters for intent-based navigation.	false	
url **Note** Not supported as of DTA Specification version 3.20.	string	Optional fallback URL which Joule uses when the intent cannot be resolved by the framework.	false	https URL or relative URL

Properties for intent based navigation targets

Attribute	Type	Description	Required
semanticObject	string	Semantic object for the navigation target.	true
action	string	Action configuration of the navigation target.	true

- **Define URL navigation**

Attribute	Type	Description	Required	Allowed Values
url	string	URL must be launched in a new tab.	true	https URL or relative URL

Additional Information

Custom handling of navigation

If you want to integrate Joule with an SAP business application, you have the option to handle navigation yourself using Web Client bridge APIs.

You can implement platform dependent navigation logic, either using the URL (if resolved) or the IBN targets in the `onNavigation` (payload) API of your Web Client bridge. If such an implementation is provided in your bridge, Web Client will call this method on user action, instead of executing the default logic..

Examples

Intent based navigation

Sample Code

```
- type: navigation
  title: Display Service Contracts
  navigation_target:
    ibnParams:
      P_DisplayCurrency : EUR
    ibnTarget:
      semanticObject: ServiceContract
      action: displayFactSheet
```

Intent-based navigation for UI Integration Cards

To use intent-based navigation (IBN) in UI integration cards, capability developers must ensure that only Joule data binding syntax is used in the manifest definition. This ensures that the manifest is fully resolved by Joule platform to determine IBN target definitions and their values.

Note

UI5 binding syntax along with IBN is not supported in UI integration cards.

URL navigation

Sample Code

```
- type: navigation
  title: Go to Example.com
  navigation_target:
    url: https://example.comx
```

3.3.2.4.3 UI Constraints and Limits

The following table lists the various components of UI Constraints and Limits and details of each component.

Text Length Limits

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
Text	content (markdown: false)	MAX_SAFE_INTEGER		Limit to the javascript MAX_SAFE_INTEGER value
	content (markdown: true)	MAX_SAFE_INTEGER		
Card	title (markdown: false)	Displays up to 3 lines of available space. Maximum 640 characters		

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
	title (markdown: true)	Approximately 150 characters	Approximately 240 characters	Excluding the markdown symbols
	subtitle (markdown: false)	Displays up to 2 lines of available space. Maximum 640 characters		
	subtitle (markdown: true)	Approximately 100 characters	Approximately 160 characters	Excluding the markdown symbols
	description (markdown: false)	Displays up to 5 lines of available space. Maximum 640 characters		
	description (markdown: true)	Approximately 250 characters	Approximately 400 characters	Excluding the markdown symbols
Buttons (Message)	title	80 characters		
Button (individual button displayed in Buttons, List, Carousel and Card)	title	640 characters		
	text	80 characters		
Quick Reply	Message title (markdown: false)	640 characters		
	Message title (markdown: true)	Approximately 640 characters		
	text	32 characters		
List	Header title (markdown: false)	Displays up to 3 lines of available space. Maximum 640 characters		

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
	Header title (markdown: true)	• 150 characters without image and row count • 132 characters with image • 90 characters with row count • 72 characters with image and row count	• 240 characters without image and row count • 220 characters with image • 180 characters with row count • 160 characters with image and row count	
	Header subtitle (markdown: false)	Displays up to 2 lines of available space. Maximum 640 characters		
	Header subtitle (markdown: true)	• 100 characters without image • 88 characters with image	• 160 characters without image • 148 characters with image	
	Header description (markdown: false)	Displays up to 5 lines of available space. Maximum 640 characters		
	Header description (markdown: true)	• 250 characters without image • 220 characters with image	• 400 characters without image • 370 characters with image	
	List element title (markdown: false)	Displays up to 3 lines of available space. Maximum 640 characters		

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
	List element title (markdown: true)	• 150 characters without image and status or button • 132 characters with image • 90 characters with status or button • 72 characters with image and status or button	• 240 characters without image and status or button • 220 characters with image • 180 characters with status or button • 160 characters with image and status or button	
	List element subtitle (markdown: false)	Displays up to 2 lines of available space. Maximum 640 characters		
	List element subtitle (markdown: true)	• 100 characters without image and button • 88 characters with image • 60 characters with button • 48 characters with image and button	• 160 characters without image and button • 148 characters with image • 120 characters with button • 108 characters with image and button	
	List element description (markdown: false)	Displays up to 5 lines of available space. Maximum 640 characters		
	List element description (markdown: true)	• 250 characters without image and button • 220 characters with image • 150 characters with button • 120 characters with image and button	• 400 characters without image and button • 370 characters with image • 300 characters with button • 270 characters with image and button	

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
	List element subtitle (markdown : true)	~ 100 without image and button ~ 88 with image ~60 with button ~ 48 with image and button	~ 160 without image and button ~ 148 with image ~120 with button ~ 108 with image and button	
Carousel	Carousel contains a set of cards displayed horizontally. See Card limits			
Detail View (displayed in Card, List and Carousel)	Detail View title (markdown: false)	Displays up to line(s) defined in `linesInDetailView` (Defaults 3) of available space. Maximum 640 characters		Values taken over from definition in the individual element of the composite control (that is. Card, List element, Carousel card). Can be controlled using the attribute `linesInDetailView`. If `linesInDetailView` is not defined, will use the default card limits For Carousel, the text lengths will ignores the image
	Detail View title (markdown: true)	~ 150 without image and status ~ 132 with image ~ 90 with status ~ 72 with image and status if `linesInDetailView` is defined, see the note below	~ 240 without image and status ~ 220 with image ~180 with status ~ 160 with image and status if `linesInDetailView` is defined, see the note below	
	Detail View subtitle (markdown: false)	Displays up to line(s) defined in `linesInDetailView` (Defaults 2) of available space. Maximum 640 characters		
	Detail View subtitle (markdown: true)	~ 100 without image ~ 88 with image if `linesInDetailView` is defined, see the note below	~ 160 without image ~ 148 with image if `linesInDetailView` is defined, see the note below	

Component	Attribute	Limit in Web Client (Default width)	Limit in Web Client (Full Screen)	Comments
	Detail View description (markdown: false)	Displays up to line(s) defined in `linesInDetailView` (Defaults 5) of available space. Maximum 640 characters		
	Detail View description (markdown: true)	~ 250 without image ~ 220 with image if `linesInDetailView` is defined, see the note below	~ 400 without image ~ 370 with image if `linesInDetailView` is defined, see the note below	
	Section header title	640 characters		
	Section attribute label	Displays up to 1 line of available space		
	Section attribute value (markdown: true)	Displays up to 4 lines of available space		
	Section attribute value (markdown: false)	Approximately 200 characters	Approximately 320 characters	

> **Note**
>
> If the `linesInDetailView` is defined for markdown text, the following logic is used:
>
> maxLength = linesInDetailView * (defaultLength - imageLength - statusLength)
>
> Standard: defaultLength = 50, imageLength = 6, statusLength = 20
>
> Full Screen: defaultLength = 80, imageLength = 6, statusLength = 20
>
> The imageLength and or statusLength will be 0 if no image or status is defined.

UI Elements Limit

Component	Elements	Minimum	Maximum	Comments
Card	button	1	3	
	section	0	1	
	content or attribute pairs	8		
List	button	0	1	

Component	Elements	Minimum	Maximum	Comments
	elements	1	30	6 elements are shown initially. *View More* allows the user to see additional elements upto 30 elements. Following attributes can be used to control: • initial page size (initialPageSize) • incremental page size (incrementPageSize) • Maximum elements in a list should be less than 30 (maxNumberOfRowsInList)
	element buttons	0	7	
Buttons	button	1	15	5 buttons are shown initially. *View More* allows the user to see additional buttons up to 15.
Quick Replies	button		1	
Carousel	card	1	12	After the pages reach the count of 8, the dots are replaced by numeral representation (valid for all devices).

Markdown Limitations

Joule supports extended markdown syntax in responses except for the following limitations:

- Heading IDs - Not Supported, for more information, see Heading IDs .
- Definition Lists - Not Supported, for more information, see Definition Lists .
- Emoji Shortcodes - Not Supported, for more information, see Using Emoji Shortcodes .
- Word Highlight - Not Supported, for more information, see Highlight .

3.3.2.5 Internationalization (i18n)

3.3.2.5.1 Language Setting Propagation

To facilitate multi-language support in Joule and your capabilities, it is essential to propagate your application's language settings both to Joule and to any system aliases or backend destinations that have language-dependent endpoints utilized in your capability. This propagation is critical if you aim to provide a consistent multi-language experience to your users in Joule. Without this, users might encounter a mix of languages across different features, such as receiving a welcome message in English while the rest of the conversation occurs in another language, like German.

From Your Capability to Your Remote Services

If you have some language dependent remote endpoints, meaning returning some sentences, then you will have to propagate the `$transient.processing_locale.raw` Joule conversation locale attribute to your remote service. This will allow your remote service to return the sentences with your own internalization or translation in the end user language. This may necessitate modifying parameters within your remote endpoints and implementing a translation or internalization process within that remote service to handle such textual content accurately.

The following example demonstrates the method used to propagate the Joule conversation locale attribute to a remote endpoint:

Dialog Function example

```
parameters:
  - name: valueExample
action_groups:
  - actions:
      - type: api-request
        method: GET
        system_alias: SFSF
        path: /odatav4/workforce/Workforce.svc/v1/WorkProfile?language=<?
$transient.processing_locale.raw ?>
        result_variable: workprofile_result
```

Other possible $transient.processing_locale values

```
{
    "transient": {
      "processing_locale": {
        "raw": "en-EN",
        "language": "en",
        "country": "EN"   // null if not specified in the locale
      }
    }
}
```

Recommendations

Character Encoding

Given that certain language character sets include non-ASCII characters, you may need to make adjustments when making backend or destination API calls (for example in api-request actions) to accommodate user inputs in an encoded format.

When incorporating non-ASCII characters in URLs, especially in scenarios where these URLs are transmitted to remote endpoints, it is imperative to ensure their correct encoding to maintain compatibility and proper functionality. Initially introduced in RFC 1630, percent-encoding was recommended for dealing with "unsafe" characters, and this approach has been refined and specified in subsequent RFCs, such as RFC 3986, which advocates for the use of UTF-8 encoding.

Before implementing any encoding, it is crucial to first verify how the remote endpoint handles non-ASCII characters. Different systems and platforms may have varying capabilities and requirements for URL processing, and understanding these specifications will guide you in choosing the most effective approach.

Once you have confirmed that the remote endpoint can appropriately process percent-encoded URLs, we recommend utilizing the UTF-8 character encoding and ensuring that each UTF-8 octet within the URL is percent-encoded.

For URLs containing data in the form of a query string, note that the application/x-www-form-urlencoded format, which is often used in HTML forms, dictates replacing spaces with a plus sign (+) and encoding line breaks as CRLF (%0D%0A). According to the HTML5 specification, while various ASCII-compatible character sets can be used, all non-ASCII characters must still be percent-encoded.

By adhering to these standards and conducting a preliminary check on the endpoint's compatibility, you can utilize the `urlEncode` method effectively. This proactive approach ensures that your URLs are consistently interpreted and handled correctly across diverse systems and software, enhancing the reliability and efficiency of your data transmissions to remote endpoints.

Dialog Function example:

```
parameters:
  - name: valueExample
action_groups:
  - actions:
      - type: api-request
        method: GET
        system_alias: SFSF
        path: /odatav4/workforce/Workforce.svc/v1/WorkProfile?find=<?
valueExample.urlEncode() ?>
        result_variable: workprofile_result
```

3.3.2.5.2 Handbook for Internationalizing Capabilities

Initial Setup

To achieve multi-language support in your digital assistant, for translation purposes, you need to create one default `messages.properties` file in the `i18n` folder in each capability directory and use the Spring Expression Language (SpEL) or Handlebar scripts inside Joule functions to access the translated content.

Folder Structure

```
sample_capability
├── i18n
│   ├── messages.properties
│   └── messages_en.properties
└── capability.sapdas.yaml
```

Naming Rules for the .properties File

To ensure that the Joule runtime consistently resolves the correct translation file for all LoBs, it's essential to follow a consistent naming convention for the .properties files.

- The name of the .properties files must follow a format that is consistent with IETF BCP 47 (which is a combination of ISO 639 (language codes), ISO 15924 (script codes), and ISO 3166-1 (region codes)) with the following exceptions:
  - Instead of a hyphen, underscore (_) must be used as a delimiter.
  - The file name must consist of the following parts: resource bundle (`messages`), language, country, and variant

  Examples
  - `messages_en.properties`
  - `messages_en_GB.properties`
  - `messages_de.properties`
  - `messages_de_CH.properties`
- All i18n keys are present in all .properties files and have a 1:1 relation with the keys in the `messages.properties` file. In case certain text cannot be translated,locale sub-tags (such as country, script, or variant) are iteratively removed to find a more general match. The fallback process aims to provide the closest available alternative while maintaining the most relevant linguistic context. Following the Oracle documentation Java look-up sequence, based on the string ID, the string resolution should be:
  - `baseName + "_" + language + "_" + country.properties //es_CODE`
  - `baseName + "_" + language.properties //es`
  - `baseName.properties //default`
- The resource bundle name must be `messages` for all .properties files.
- In a scenario where the string ID cannot be found in any file, despite the various checks performed by the compiler at design time, the response should display the string ID.
  For more information about this approach, see RFC 4647 .
- No folders are allowed within the `i18n` folder.
- Multiple files cannot exist for the same IETF BCP 47 language tag, for example, `messages_fr_CA.properties`, `messages_fr.properties`. Each language must have its own file.

Internationalization for Joule Functions

The adaptation process of Joule functions to multi-language support consists of the following tasks:

- All translation-relevant strings must be moved from capability function files to `messages.properties` files. Each string should be assigned to unique key.
- All initial strings in source files must be replaced by pointers to created keys.

You can achieve this using two different syntax: Handlebar or SpEL. Below you can find examples of replacing strings with and without variables using both syntaxes.

Handlebars

In Handlebars, the `var-args` method can be indirectly supported by passing an array as a parameter, for example, given the `var-args` expression example below:

```
{{i18n 'MESSAGE_ID' <var-arg param1> <var-arg param2> ... <var-arg paramN>}}
```

Sample Code

Weather, Joule Function without i18n keys

```
- condition: is_mine == true
  actions:
    - type: message
      scripting_type: handlebars
      message:
        {
          "type": "text",
          "content": "Here's the link to view the profile of your manager."
        }
- condition: is_mine == false
  actions:
    - type: message
      scripting_type: handlebars
      message:
        {
          "type": "text",
          "content": "Here's the link to view the profile of
{{selected_person.displayName}}'s manager."
        }
```

Sample Code

Weather, Joule Function with i18n keys

```
- condition: is_mine == true
  actions:
    - type: message
      scripting_type: handlebars
      message:
        {
          "type": "text",
          "content": "{{i18n 'LINK_TO_MANAGER_PROFILE'}}"
        }
- condition: is_mine == false
  actions:
    - type: message
      scripting_type: handlebars
      message:
        {
```

```
          "type": "text",
          "content": "{{i18n 'LINK_TO_PERSONS_MANAGER_PROFILE'
selected_person.displayName}}"
        }
```

```
#XMSG
LINK_TO_MANAGER_PROFILE=Here's the link to view the profile of your manager.

#XMSG, {0} is a person name
LINK_TO_PERSONS_MANAGER_PROFILE=Here's the link to view the profile of {0}'s
manager.
```

SpEL

In SpEL, the `var-args` method is natively supported:

```
new i18n('MESSAGE_ID', <var-arg param1>, <var-arg param2> ... <var-arg paramN>)
```

Sample Code

Weather, Joule Function without i18n keys

```
- condition: weather_result.status_code == 200
   actions:
     - type: message
       message:
         type: text
         content: "In <? weather_location.city ?>, on <?
weather_result.body.dayOfWeek[0] ?> temperature will be between
<? weather_result.body.calendarDayTemperatureMin[0] ?> °C and <?
weather_result.body.calendarDayTemperatureMax[0] ?> °C. Details: <?
weather_result.body.narrative[0] ?>"
- condition: weather_result.status_code != 200
   actions:
     - type: message
       message:
         type: text
         content: "An error occurred when fetching the weather information.
The result was: <? weather_result ?>"
```

Sample Code

Weather, Joule Function with i18n keys

```
- condition: weather_result.status_code == 200
    actions:
      - type: message
        message:
          type: text
          content: "<? new i18n('WEATHER_RESULTS',
weather_location.city, weather_result.body.dayOfWeek[0],
weather_result.body.calendarDayTemperatureMin[0],
weather_result.body.calendarDayTemperatureMax[0],
weather_result.body.narrative[0]) ?>"
  - condition: weather_result.status_code != 200
    actions:
      - type: message
        message:
```

```
            type: text
            content: "<? new i18n('FETCHING_WEATHER_ERROR', weather_result) ?>"
```

```
# XMSG, {0} is the city name, {1} is the day of the week, {2} is the minimum
temperature, {3} is the maximum temperature, {4} is the weather description
WEATHER_RESULTS=In {0}, on {1} temperature will be between {2} °C and {3} °C.
Details: {4}

# XMSG, {0} is the error message
FETCHING_WEATHER_ERROR=An error occurred when fetching the weather information.
The result was: {0}
```

Merging an Internationalized String

In the case that the i18n helper is called inside an existing string which needs to be concatenated or merged with, then the `toString` method will have to be called. For example:

```
 - type: message
        message:
          type: text
          content: "ID_SAMPLE value in SPEL is <? new
i18n('ID_SAMPLE').toString() ?>"
```

Otherwise an error message will be returned in the "Request Logs" of the Web Client Debug Window:

```
{
  "level": "warning",
  "code": "ACTION_FAILED",
  "message": "Message action skipped due to invalid scripting expression in
message",
  "data": {
    "warnings": [
      {
        "invalid_expression": "ID_SAMPLE value in SPEL is <? new
i18n('ID_SAMPLE') ?>",
        "scripting_error": "EL1001E: Type conversion problem, cannot convert
from Translation to java.lang.String"
      }
    ]
  }
}
```

Best Practices for Quotes in i18n.properties Files

- SpEL
  - No extra escaping for quotes is required. The number of quotes (both single (') and double (")) will be rendered the same as specified in the .properties file.
  - SpEL internally aligns with Handlebars behavior, so there is no need for double single quotes.
  - There aren't any restrictions for curly quotes (‘’, “”) .
- Handlebars
  - If i18n strings are inserted into plain text, no escaping is required.
  - The number of quotes will be rendered the same as specified in the .properties file.
  - If i18n strings are inserted into a JSON structure, always use escape helper which serializes the value. This prevents JSON from breaking when the string contains quotes.

```
"{{ escape (i18n 'EXAMPLE') }}"
```

- messages.properties

```
EXAMPLE=example with " (double quotes)
```

- `escape_example` using YAML

```
action_groups:
  - actions:
      - type: message
        scripting_type: handlebars
        message: >
          {
            "type": "list",
            "content": {
              "total": "15",
              "elements": [
                {
                  "title": {
                    "value": "{{ escape (i18n 'EXAMPLE') }}"
                  }
                }
              ],
              "initialPageSize": 15,
              "enableDetailView": false,
              "title": "Available cars"
            },
            "markdown": false,
            "delay": 0
          }
```

> **Note**
>
> Translators typically follow the pattern of quotes in the source .properties file (for example, if the source has one quote, they return one; if two, they return two). It might be necessary to align with your translation team to ensure that quotes are rendered correctly in all languages.

Edge Cases

- All special characters (for example, emoji) in strings should be replaced by unicode symbols (for example, \n, \uxxxx).
- Try to avoid words which can't be translated adequately during capability creation (for example, are, is). For more information, see Working with Composite Messages .
- Avoid using conditions within strings when creating capabilities, as it results in complex constructions in the `messages.properties` file that are difficult to translate and are not supported. Only simple parameter syntax, such as `{0}, {...}`, is supported.

> **Sample Code**
>
> Joule Function without i18n keys (string contains condition)
>
> ```
> - condition: only_one_user
>  actions:
>  - type: message
>  scripting_type: handlebars
>  message: >
> ```

```
        {
          "type": "text",
          "content": {{#if selected_person}}"I found one
employee, {{selected_person.displayName}}{{#if selected_person.email}}
({{selected_person.email}}){{/if}}, matching the name you
provided."{{else}}"I found one employee matching the name you provided."{{/
if}}
        }
```

Sample Code

Joule Function with i18n keys (string contains condition)

```
- condition: only_one_user
  actions:
    - type: message
      scripting_type: spel
      message: >
        {
          "type": "text",
          "content": "<? new i18n('FOUND_EMPLOYEE_WITHOUT_NAME',
selected_person.displayName, selected_person.email) ?>"
        }
```

```
#XMSG, {1} is a person name, {2} is a person email
FOUND_EMPLOYEE_WITHOUT_NAME=I found one employee {0,choice,0#matching the name
you provided.|1#, {1}{2,choice,0#|1# ({2})} matching the name you provided.}
```

For any scenario not covered in the documentation above, please refer to the additional examples provided in the following links to learn more about non-standard cases. Note that these cases are not supported in Joule:

- Message Format (Java 2 Platform SE 5.0)
- Java Message Format Converter

3.3.3 Agent Integration

SAP provides content-based agent building capabilities that enable developers to create custom agents to automate complex, non-deterministic business processes. For more information, see Content-Based Agents [page 190].

Furthermore, the integration of code-based agents, that is, agents built on third-party agent frameworks, is supported. For more information, see Code-Based Agents (Bring Your Own Agent) [page 204].

Note

To build and run agents, you might require a special entitlement. You can find the current agreement in Joule Admin Center.

In Joule Admin Center, you can enable agent-building capabilities in the Joule Studio CLI and Joule Studio code editor tools and, if necessary, accept the *Terms & Conditions*. For information about accessing Joule Admin Center, see Joule Admin Center.

3.3.3.1 Content-Based Agents

SAP provides content-based agent building capabilities that enables developers to create custom agents to automate complex, non-deterministic business processes.

Integration and definition of agents within Joule content are easily achievable. When deploying an assistant, agents specified in a capability are automatically created in an agent runtime. During scenario processing, agents can receive messages from and send status updates and messages back to Joule. Agents may request user input if needed to complete tasks or seek human approval. Upon deletion of an assistant, the associated agent is also removed from the agent runtime again.

- Capabilities for building content-based agents are supported in the following regions: JP10, EU11, EU30, US30, AP10, US10, EU10, EU20.
- Multi-lingual support is available for Joule content-based agents.

> **Note**
>
> Agent testing/debugging features are not supported. Extensibility developers can build, deploy, and consume agents from Joule; however, the testing/debugging feature to check agent execution traces/logs is not available.

Agent Definition

Under the folder **agents**, extensibility developers can create a new agent YAML file that contains the agent configuration.

```
└── my_capability
├── agents
│ ├── crm_agents # nested folder
│ ├── sales_agents # nested folder
│ ├── sales_order_agent.yaml
│ └── service_request_agent.yaml
│ ├── sales_order_agent.schema.json # for developer to define JSON schema for
agent
│ └── service_request_agent.schema.json
```

Each agent file contains specific instructions and properties that tailor the agents behavior during runtime. The compiler impose resource level limits via the metadata configuration. The following limits apply:

For agent files:

- Maximum 3 folder nesting levels
- Maximum 50 agents
- Maximum 100 KB per agent YAML file

For schema files:

- Maximum file size: 100 KB
- Maximum number of schema files: 50

In addition, the total DTA size will also be increased by 5 MB (35 MB -> 40 MB) to accommodate the agent files.

Here is an example of an agent that is capable of resolving dispute cases.

```
agent: "dispute_resolution_agent"
expert_in: >
  You are an expert in financial dispute case resolution with a focus on
reconciling invoiced amounts against contract terms.
  Your responsibilities include:
    – Retrieving and reviewing dispute cases along with associated invoiced
amounts and contract attachments.
    – Recalculating invoice amounts based on the contractual terms.
    – Identifying valid discrepancies by comparing the recalculated amounts to
the actual invoices.
    – Creating and processing credit memos through established tools.
iterations: 100
preprocessing_enabled: true
postprocessing_enabled: true
missing_input_tool_enabled: false
base_model: gpt-4o
advanced_model: gpt-4o
backup_base_model: gpt-4o
backup_advanced_model: gpt-4o
task: >
  ──────────────────────────────  Step-by-Step Instructions:
──────────────────────────────

  1. Data Retrieval
    - Ensure that the dispute case ID (`disputeCaseId`) is provided.
    - Retrieve all necessary information about the dispute case using the
\"fetch dispute case data\" tool.
    - If the type of the dispute case (`disputeType`) is not \"A001\"Inform
that only dispute case of type \"A001 (Incorrect Invoice)\" can be process and
proceed to Step 8
    - Verify that all required details related to the dispute case are
present (i.e., `disputedAmount`, `disputedCaseCurrency`, `invoiceLineItems`,
`disputeStatus`, `disputeNotes`) and the contract attachment.
    - Ensure that `invoiceLineItems` contains `documentDueDate`, `contractId`,
`invoiceAmount`, `invoiceCurrency`, `taxRateInPercent`.
    - If any of the required information is missing Inform that the dispute case
cannot be handled due to incomplete or invalid information and proceed to Step 8
    - If `disputeNotes` contains a note that there was already a credit memo
created for the dispute case Inform that the dispute case cannot be handled due
to a credit memo already created and proceed to Step 8

  2. Determine the Adjustment Period
    1) Identify the `documentDueDate` for which the invoice should be
recalculated, and the `contractStartDate` from the contract attachment.
    2) The `adjustmentRate` is the rate at which the base fee is adjusted over
time, and the `adjustmentPeriodUnit` and `adjustmentPeriodLength` are used to
determine the frequency of adjustments.
    3) Furthermore, retrieve the information from the contract whether the
adjustment sets in after a certain period has passed (i.e. `periodCriteria` =
\"length\", e.g. after one year (365 days) has passed) or if the adjustment is
applied at the start of a new calendar period (i.e. `periodCriteria` = \"date\",
e.g. beginning of a new year (1st of January)).
    4) If the `periodCriteria` is \"date\", the `contractStartDate` is
rounded down to the beginning of the `adjustmentPeriodUnit` (e.g. if the
`adjustmentPeriodUnit` is \"months\", the `contractStartDate` is rounded down to
the first day of the month). Do not use the calculator tool for that. This new
value is overwriting the existing `contractStartDate` value.
    5) The adjustment period is determined with the following steps:
      - Determine the days that lie between the `contractStartDate` and
`documentDueDate` to UNIX timestamps using the date conversion tool.
      - Calculate the difference between the two dates and afterwards convert
it to the unit of the `adjustmentPeriodUnit` (e.g., days, months, years). This
is done by dividing the difference in seconds by the number of seconds in the
respective unit (e.g., 60 seconds * 60 minutes * 24 hours * 30 days for months).
The result is the `dateDiff`.
```

```
      - Calculate the `adjustmentPeriod`as follows: floor(dateDiff /
adjustmentPeriodLength).
      - If `adjustmentPeriod` < 0 -> `adjustmentPeriod` = 0
  3. Invoice Recalculation
      1) Identify the `baseFee`, the `adjustmentRate`, and whether taxes are
included in the base fee (i.e. `taxIsIncluded` = \"true\" or \"false\") from the
contract attachment.
      2) If the `baseFee` is including taxes based on the contract terms, ensure
that it is converted to a tax-free amount by excluding taxes, i.e. baseFee =
baseFee / (1 + taxRateInPercent / 100).
      3) Recalculate the invoice amount (`recalculatedInvoiceAmount`) strictly
based on the contract terms specified in the contract attachment for the
given `documentDueDate`. Be careful, to consider the `contractStartDate` as
well as the `documentDueDate`, i.e. recalculatedInvoiceAmount = baseFee * (1 +
adjustmentRate) ^ max(0, adjustmentPeriod).

  4. Invoice Amount Comparison
    - Calculate the `discrepancy` between the `recalculatedInvoiceAmount`
and the original invoice amount (`invoiceAmount`) (i.e., `discrepancy =
invoiceAmount - recalculatedInvoiceAmount`).
    - If the `discrepancy` (considering currency and rounding) is unequal to
zero: Proceed to Step 5. Else if the `discrepancy` is zero (no difference):
Inform that there is no difference found in invoice and the corresponding
contract, question the validity of the dispute case, and proceed to Step 8.

  5. Compare Discrepancy to Disputed Amount
    - Compare the `discrepancy` to the `disputedAmount`.
    - If the `discrepancy` exactly matches the `disputedAmount` (considering
currency and rounding): Proceed to Step 6. Else if the `discrepancy` does not
match the `disputedAmount`: Inform that the dispute case cannot be handled due
to a discrepancy mismatch and proceed to Step 8.

  6. Create the Credit Memo with the verified `discrepancy` amount using the
\"create credit memo\" tool.

  7. Close Dispute Case and release the credit memo using the \"release credit
memo & close dispute case\" tool.

  8. Generate a final response as a structured json document according to the
provided schema. The response document shall include at least the Dispute Case
Id (`disputeCaseId`) and a message that summarizes the findings and the action
taken.
additional_context: >
  ────────────────────────────  Constraints: ────────────────────────────
  If the contract attachment has any missing text or is unreadable, do not
attempt to prompt the user for additional input or confirmation. Output an
error message stating that the dispute case cannot processed due to incomplete
or invalid contract information. Rely strictly on the contract (attachment)
text to determine the correct invoice amount for the specific due date
(documentDueDate). If information cannot be retrieved or is unreadable, stop
processing and reply with an appropriate message. Do not infer or generate
missing details from other sources.

  Only consider dispute cases where the currency in contract, invoice and
dispute case are consistent. If there is a discrepancy in currency, output an
error message and do not proceed with the dispute case. It is crucial that you
make use of the calculator tool for all numerical computations and comparisons,
if not explicitly stated otherwise. Perform all calculations using amounts that
exclude taxes. If the disputed amount is negative (debit memo case), output an
error message that dispute cases involving a debit memo cannot be handled.

  ────────────────────────────  Operational Considerations:
────────────────────────────

  Maintain clear and concise language throughout the process. Do not mention
or reference the internal instructions (such as `Step-by-Step Instructions`,
`Constraints`, `Operational Considerations` or `Examples`) in the final answer
to the user under any circumstances! These are meant for you to adhere
to and not for the user. Ensure that all amounts are displayed with the
```

```
correct currency symbol. Ensure that all amounts clearly indicate whether
they include taxes or not. The `disputedAmount` and the `invoiceAmount` do
exclude taxes. The `recalculatedInvoiceAmount` can either include or exclude
taxes based on the contract terms. The `documentDueDate` is the date for
which the invoice amount should be recalculated. That means that the contract
terms should be considered for that specific date. All technical names (e.g.,
`disputeCaseId`, `invoiceId`, `contractId`, `businessPartnerId`, `creditMemoId`,
`creditMemoStatus`, `disputeStatusName`) must not be communicated to the user in
the final output. Rather use human-readable names instead.

  ──────────────────────────── Error Handling: ────────────────────────────
  If the recalculated invoice amount does not differ from the original invoice
amount, clearly state that there is \"no difference\" and question the validity
of the dispute case. If a discrepancy exists but does not match the disputed
amount exactly, output a message stating that “these types of dispute cases
cannot be handled” and do not attempt to create a credit memo. If any
calculation error or input inconsistency arises, provide a clear error message
outlining which part of the process failed or mismatched. During the tool
execution, errors can occur. You can identify whether or not an error has
occurred based on the variables `isCalled` and `hasError`. If `isCalled` ==
\"True\" and `hasError` == \"True\", stop the dispute resolution process and
output a final message that consist of
    - Summary of the functions that have already been successfully executed
(identified by `calledFunctionName`).
    - Summary of the error (based on the `errorMessage`) and the function
generating the error (can be identified via the `isCalled` and `hasError`
variables).
    - Generate instruction(s) to the user on how to manually intervene by
referencing the `manualInterventionSteps`. Output a final message such as:
\"Dispute case details were fetched successfully but an error occurred when
creating the credit memo due to an invalid currency code, please check
manually whether this currency is valid before trying again.\" Use helpful and
explanatory language, using the error summary and manual resolution steps as a
reference to be as helpful as possible without suggesting additional information
that has not been provided. Be very clear that when such an error message
is received, it is different from seeking human approval. Rather, subsequent
instructions need not be attempted after providing this final error 'message'.
  ──────────────────────────── Examples: ────────────────────────────
    Example 1 (Valid Dispute Case):
    - contract language: “The monthly amount is €200, excluding tax. The monthly
fee is increased at the beginning of each year by 2%”
    - baseFee: \"€200.00\"
    - invoiceAmount: “€210.00”
    - adjustmentRate: \"2%\"
    - contractStartDate: \"2023-08-01\" -> rounded down to \"2023-01-01\" due to
periodCriteria == \"date\"
    - documentDueDate: “2024-02-01”
    - adjustmentPeriodUnit: \"years\"
    - adjustmentPeriodLength: \"1\"
    - periodCriteria: \"date\"
    - adjustmentPeriod: \"1\"
    - recalculatedInvoiceAmount: “€204.00”
    - discrepancy: “€6”
    - disputedAmount: “€6”
    - result: Since the discrepancy of €6 exactly matches the disputed amount,
the dispute case is valid.
    ####################################
    Example 2 (Invalid Dispute Case – No Discrepancy):
    - contract language: “The monthly amount is €150, excluding tax. The monthly
fee is increased yearly by 2%”
    - baseFee: \"€150.00\"
    - invoiceAmount: “€150.00”
    - adjustmentRate: \"2%\"
    - contractStartDate: \"2023-01-01\"
    - documentDueDate: “2023-10-01”
    - adjustmentPeriodUnit: \"years\"
    - adjustmentPeriodLength: \"0\"
    - periodCriteria: \"length\"
```

```
    - recalculatedInvoiceAmount: “€150.00”
    - discrepancy: “€0.00”
    - result: The recalculated invoice amount is €150 since the first period is
not yet over. Furthermore, there is no discrepancy, the dispute case is invalid.
    ####################################
    Example 3 (Invalid Dispute Case – Discrepancy Mismatch):
    - contract language: “The monthly amount is €490, excluding tax. The monthly
fee is increased yearly by 2%”
    - baseFee: \"€490.00\"
    - invoiceAmount: “€500.00”
    - adjustmentRate: \"2%\"
    - contractStartDate: \"2023-01-01\"
    - documentDueDate: “2023-10-01”
    - adjustmentPeriodUnit: \"years\"
    - adjustmentPeriodLength: \"0\"
    - periodCriteria: \"length\"
    - recalculatedInvoiceAmount: “€490.00”
    - discrepancy: “€10.00”
    - disputedAmount: “€11.00”
    - result: Since the discrepancy of €10 does not match the disputed amount of
€11, the dispute case is invalid.
    ####################################
    Example 4 (Invalid Dispute Case - Debit Memo):
    - contract language: “The monthly amount is €200, excluding tax. The monthly
fee is increased yearly by 5%”
    - baseFee: \"€200.00\"
    - invoiceAmount: “€205.00”
    - adjustmentRate: \"5%\"
    - contractStartDate: \"2023-01-01\"
    - documentDueDate: “2024-10-01”
    - adjustmentPeriodUnit: \"years\"
    - adjustmentPeriodLength: \"1\"
    - periodCriteria: \"length\"
    - recalculatedInvoiceAmount: “€210.00”
    - discrepancy: “- €5.00”
    - disputedAmount: “€5.00”
    - result: Since the discrepancy is negative (debit memo case), the dispute
case cannot be handled.
    ####################################
    Example 5 (Valid Dispute Case):
    - contract language: “The monthly amount is €200, excluding tax. The monthly
fee is increased monthly by 2%”
    - baseFee: \"€200.00\"
    - invoiceAmount: “€210.00”
    - adjustmentRate: \"2%\"
    - contractStartDate: \"2023-08-01\"
    - documentDueDate: “2024-02-01”
    - adjustmentPeriodUnit: \"month\"
    - adjustmentPeriodLength: \"1\"
    - periodCriteria: \"length\"
    - adjustmentPeriod: \"6\"
    - recalculatedInvoiceAmount: “€225.23”
    - discrepancy: “€15.23”
    - disputedAmount: “€15.23”
    - result: Since the discrepancy of €15.23 exactly matches the disputed
amount, the dispute case is valid.
default_output_format: json
default_output_format_options: agent.schema.json
toolkits:
  - type: calculator
    name: "Calculator Tool"
  - type: joule
    name: "Fetch Dispute Case Data"
    config:
      target:
        type: function
        name: agent/agentTools/fetch_dispute_case_data
```

```
        description: Tool for fetching dispute data, contract data, and invoice
data
      specification:
        description: Parameters for fetching dispute case data
        parameters:
          - name: disputeCaseId
            description: The Dispute Case ID
            type: string
            optional: false
  - type: joule
    name: "Create Credit Memo"
    config:
      target:
        type: function
        name: agent/agentTools/create_credit_memo_process_dc
        description: Tool for creating a credit memo, setting its status,
updating the dispute case status, and assigning a note to the dispute case with
a link to the billing request
      specification:
        description: Parameters for creating a credit memo
        parameters:
          - name: disputeCaseId
            description: The Dispute Case ID
            type: string
            optional: false
          - name: invoiceId
            description: The invoice ID
            type: string
            optional: false
          - name: description
            description: The description of the credit memo (maximum length 60
characters)
            type: string
            optional: false
          - name: amount
            description: The amount of the credit memo
            type: string
            optional: false
          - name: currencyCode
            description: The The alphabetic currency code of the credit memo
following ISO 4217 standard (e.g. USD, EUR)
            type: string
            optional: false
      human_approval_required: true
      human_approval_prompt: >
        The agent should provide a clear summary of the details identified in
the dispute case, the steps already taken, and the recommended actions that the
user is asked to confirm.
        The agent shall respond with a json object and provide the summary in
property 'message'.
        The message should be delivered with clear sections similar to
the following format. In the overview section, the agent should provide a
comprehensive summary of the details identified so far in the dispute resolution
process, and the steps already taken.
        ---------------------------------------------------
        ### Dispute Case: <disputeCaseId>
        #### Overview
        Dispute <disputeCaseId> involves a discrepancy between the recalculated
invoice amount and the original invoice amount based on the contract
(attachment). A discrepancy of <discrepancy> was identified on DocumentDate
<documentDueDate>.
        #### Discrepancy Identified:
        - Recalculated Invoice Amount (excl. tax): <recalculatedInvoiceAmount>
        - Original Invoice Amount (excl. tax): <invoiceAmount>
        - Disputed Amount (excl. tax): <disputedAmount>
        #### Finding:
        The discrepancy of <discrepancy> has been confirmed to match the
disputed amount.
```

```
        #### Recommendation:
        - Create a credit memo for <disputedAmount>.
        Would you like to proceed with these actions?
  - type: joule
    name: "Release Credit Memo & Close Dispute Case"
    config:
      target:
        type: function
        name: agent/agentTools/release_credit_memo_close_dc
        description: Tool for releasing the credit memo and closing the dispute
case
      specification:
        description: Parameters for releasing credit memo and closing dispute
case
        parameters:
          - name: disputeCaseId
            description: The Dispute Case ID
            type: string
            optional: false
          - name: creditMemoId
            description: The Credit Memo ID
            type: string
            optional: false
      human_approval_required: true
      human_approval_prompt: >
        The agent should provide a clear summary of the recently taken steps and
the recommended actions that the user is asked to confirm.
        The agent shall respond with a json object and provide the summary in
property 'message'.
        The message should be delivered with clear sections similar to the
following format:
        I created a credit memo <creditMemoId> over <discrepancy> (excl. tax)
and added a note about it to the dispute case.
        Do you want to go ahead with releasing the credit memo and closing the
dispute case?
  - type: joule
    name: "Date Conversion"
    config:
      target:
        type: function
        name: agent/agentTools/date_conversion
        description: Tool for converting date formats to unix timestamp
      specification:
        description: Parameters for date conversion
        parameters:
          - name: contractStartDate
            description: The contract start date in YYYY-MM-DD format
            type: string
            optional: false
          - name: documentDueDate
            description: The document due date in YYYY-MM-DD format
            type: string
            optional: false
```

Initiating an Agent Request

Agent requests are defined as part of a dialog function using an agent request action. Once an agent request is initiated, execution of the dialog function is paused for up to 5 minutes. If the agent responds within this 5-minute window, processing of the dialog function continues and the rest of the defined actions in the dialog functions are processed. Otherwise, a timeout occurs and the execution of the dialog function is aborted.

```
parameters:
```

```
  - name: query
    optional: true
  - name: disputeCaseId
    optional: true
action_groups:
  - condition: query == null
    actions:
      - type: set-variables
        variables:
          - name: query
            value: "<? $transient.input.text.raw ?>"
  - actions:
      - type: agent-request
        name: dispute_resolution_agent
        body: >
          {
            "message": "<? query ?>"
          }
        result_variable: "apiResponse"
  - condition: apiResponse != null && apiResponse.body != null &&
apiResponse.body.type != null && apiResponse.body.type == "finalAnswer"
    actions:
      #case: "finalAnswer" response
      - type: set-variables
        variables:
        - name: agentMessage
          value: "<? apiResponse.body.message ?: 'Error: missing <message> in
agent response' ?>" #requested outputFormat JSON, message contains json
      - type: set-variables
        variables:
        - name: disputeCaseId
          value: "<? agentMessage.disputeCaseId ?: 'Error: missing
<disputeCaseId> in agent response' ?>"
      - type: dialog-function
        name: agent/helper/outputDisputeCaseProcessing
        parameters:
          - name: agentMessage
            value: "<? agentMessage ?>"
          - name: disputeCaseId
            value: "<? disputeCaseId ?>"
  - condition: apiResponse != null && apiResponse.body != null &&
apiResponse.body.type != null && (apiResponse.body.type == "missingInput" ||
apiResponse.body.type == "permissionRequest")
    actions:
      #case: "MissingInput" or "approval" response
      - type: dialog-function
        name: agent/helper/outputQuestionForUser
        parameters:
          - name: agentQuestionResponseBody
            value: "<? apiResponse.body ?>"
          - name: disputeCaseId
            value: "<? disputeCaseId ?>"
        result_variable: outputQuestionForUser_result
      - type: set-variables
        variables:
        - name: disputeCaseId
          value: "<? outputQuestionForUser_result.disputeCaseId ?:
disputeCaseId ?>"
result: # functions return a map with the defined variables
  msg: <? apiResponse.body.message ?> # response "text" from agent / json
  disputeCaseId: <? disputeCaseId ?> # response "disputeCaseId" from agent / json
```

Initiating an Agent Request with Immediate Response

As part of triggering conversational agents from Joule, though it is expected to complete the agent execution within 5-minute window, sometimes it might take beyond 5 minutes which results in timeout issues. As a workaround for this issue, an optional boolean field named `awaitReply` can be defined within the body of the agent request action which will determine whether an immediate response is expected or not. The default value for this field will be true, in which case the agent response will be send back after the agent execution. However, for long running agent requests, this field can be defined as false, in which case a static success response will be send back once the agent has been successfully triggered.

```
- actions:
    - type: agent-request
      name: dispute_resolution_agent
      body: >
        {
          "message": "<? query ?>",
          "awaitReply": false
        }
      result_variable: "apiResponse"
```

Invoking a Dialog Function As a tool for an Agent

Agents can utilize dialog functions as tools to perform complex operations or retrieve back end information through Joule. These tools are set up within the agent configuration detailed earlier in this document. Unlike standard dialog functions, dialog functions used as tools do not have access to conversational context information.

During the planning and reasoning phase, the agent autonomously identifies the necessary tools required to complete the task and invokes them during the execution phase. No additional configuration is required beyond specifying the tool in the agent configuration.

Human in the Loop

An agent may request user intervention during processing for missing information or if approval is required to proceed with certain steps, such as accessing sensitive user information.

To enable the agent to invoke a Joule function tool with these capabilities, you need to perform the following steps:

1. Set the `human_approval_required` flag to `true` in the tool definition.
2. Add the `human_approval_prompt` parameter to the tool definition, which includes a prompt for a dedicated LLM call to compose the message to the user requesting approval.

This setup ensures that the agent can seek user consent effectively when necessary.

```
  - type: joule
    name: "Release Credit Memo & Close Dispute Case"
    config:
      target:
        type: function
```

```
        name: agent/agentTools/release_credit_memo_close_dc
        description: Tool for releasing the credit memo and closing the dispute
case
      specification:
        description: Parameters for releasing credit memo and closing dispute
case
        parameters:
          - name: disputeCaseId
            description: The Dispute Case ID
            type: string
            optional: false
          - name: creditMemoId
            description: The Credit Memo ID
            type: string
            optional: false
      human_approval_required: true
      human_approval_prompt: >
        The agent should provide a clear summary of the recently taken steps and
the recommended actions that the user is asked to confirm.
        The agent shall respond with a json object and provide the summary in
property 'message'.
        The message should be delivered with clear sections similar to the
following format:
        I created a credit memo <creditMemoId> over <discrepancy> (excl. tax)
and added a note about it to the dispute case.
        Do you want to go ahead with releasing the credit memo and closing the
dispute case?
```

Missing Input or Approval Responses from an Agent

Responses from the agent that are tagged as `missingInput` or `permissionRequest` can be handled by defining a condition, after the agent-request response is returned in the dialog function.

```
  - condition: apiResponse != null && apiResponse.body != null &&
apiResponse.body.type != null && (apiResponse.body.type == "missingInput" ||
apiResponse.body.type == "permissionRequest")
    actions:
      #case: "missingInput" or "permissionRequest" response
      - type: dialog-function
        name: ...outputQuestionForUser...
        parameters:
          - name: agentQuestionResponseBody
            value: "<? apiResponse.body ?>"
          - name: disputeCaseId
            value: "<? disputeCaseId ?>"
        result_variable: outputQuestionForUser_result
      - type: set-variables
        variables:
        - name: disputeCaseId
          value: "<? outputQuestionForUser_result.disputeCaseId ?:
disputeCaseId ?>"
```

Final Agent Response

After an agent finishes processing, a final message is sent to Joule to present the results to the end user. Such responses can be handled by defining a condition, after the agent-request response is returned in the dialog function.

```
- condition: apiResponse != null && apiResponse.body != null &&
apiResponse.body.type != null && apiResponse.body.type == "finalAnswer"
    actions:
      #case: "finalAnswer" response
      - type: set-variables
        variables:
        - name: agentMessage
          value: "<? apiResponse.body.message ?: 'Error: missing <message> in
agent response' ?>" #requested outputFormat JSON, message contains json
      - type: set-variables
        variables:
        - name: disputeCaseId
          value: "<? agentMessage.disputeCaseId ?: 'Error: missing
<disputeCaseId> in agent response' ?>"
      - type: dialog-function
        name: ...outputDisputeCaseProcessing...
        parameters:
          - name: agentMessage
            value: "<? agentMessage ?>"
          - name: disputeCaseId
            value: "<? disputeCaseId ?>"
```

Agent Output Formatting

Agent configuration in Joule allows you to specify an output format for the agent response like this.

```
default_output_format: json
default_output_format_options: agent.schema.json
```

Along with the default_output_format, here json, you need to specify an output_format_option schema that needs to be added under the agents folder. Here is an example for such a schema, used by the Dispute Case Agent.

```
{
  "$id": "https://sap.com/dispute_case_agent_response.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SAP FI-CA Dispute Case Agent Response",
  "description": "This JSON schema defines a structure for representing a
dispute case and it's associated information.",
  "type": "object",
  "properties": {
    "message": {
      "type": "string",
      "description": "A message displayed to the user. (e.g., Summary of the
successful resolution process, error message, manual recovery steps, completed
steps, etc.)"
    },
    "disputeCaseId": {
      "type": "string",
      "description": "Id of the Dispute Case"
    },
    "invoiceId": {
```

```
      "type": "string",
      "description": "Id of Invoice"
    },
    "contractId": {
      "type": "string",
      "description": "Id of Contract"
    },
    "businessPartnerId": {
      "type": "string",
      "description": "Id of Business Partner"
    },
    "creditMemoId": {
      "type": "string",
      "description": "Id of the Credit memo"
    },
    "creditMemoStatus": {
      "type": "string",
      "description": "Status of the Credit memo"
    },
    "disputeStatusName": {
      "type": "string",
      "description": "Status of the Dispute Case"
    }
  },
  "required": [
    "disputeCaseId",
    "message"
  ],
  "additionalProperties": false
}
```

MCP Tool Support for Content-Based Agents (Version 3.23.0 and Later)

The toolkit type **MCP** is supported. The following example shows a sample **MCP** toolkit definition:

```
toolkits:
  - type: mcp
   name: com.sap.MCP_1           # MCP server name
   config:
      description: this mcp server serves some purpose            # description
of MCP server
     namespace: com.sap.mcp                                        # namespace
for MCP server
     path: /path/to/                                               # path to
call the server
     destination: MCP_1                           # destination on Joule account
to MCP server
      timeout: 20000                              #optional, in seconds
```

Multi-Agent Support (Agent As Tool) for Content-Based Agents (Version 3.25.0 and Later)

Through multi-agent support, you can define multiple agents and add them as children to a parent agent.

The `agents/` folder can hold multiple agent definitions:

```
|___agents/
```

```
|___agent_a.yaml
|___agent_b.yaml
|___agent_c.yaml
|___agent_d.yaml
|___agent_e.yaml
```

The agent definition should extend and introduce a section to add sub agents(children):

```
agent: agent_a
iterations: 20
preprocessing_enabled: true
postprocessing_enabled: true
base_model: gpt-4o-mini
advanced_model: gpt-4o
default_output_format: markdown
missing_input_tool_enabled: true
subagents:
- agent_b
- agent_c
toolkits:
- name: calc
type: calculator
```

Tool Caching (Tool Configurations for Content-Based Agents, Version 3.27.0 and Later)

The config parameter for the toolkits for content-based agents allows extensibility developers to take advantage of memory concepts that include a tool cache. This optional tool config parameter is used in document grounding.

The data type is number and corresponds to milliseconds. The default value is be 0. value in seconds from 0 to 86400 (24hours in seconds) If “cache_ttl” is set, the tool cache is enabled in the Joule framework and the retention time for the tool output is set to the value specified in the 'cache time to live', that is, cache_ttl.

```
toolkits:
- type: joule
name: "Fetch Dispute Case Data"
config:
target:
type: function
name: agent/agentTools/fetch_dispute_case_data
description: Tool for fetching dispute data, contract data, and invoice data
cache_ttl: 60 # 60 seconds
specification:
description: Parameters for fetching dispute case data
parameters:
- name: disputeCaseId
description: The Dispute Case ID
```

Agent Configuration Properties

For information about the configuration values limits that are validated during design time, see Agent Configuration Properties [page 203].

3.3.3.1.1 Agent Configuration Properties

The following configuration values and limitations are validated during design time.

Configuration Properties

Property	Type	Validations
`agent`	string	`maxLength: 250`, **required**
`expert_in`	string	`maxLength: 10000` If you exceed the character limit, you can make further use of the additional_context field
`iterations`	integer	`minimum: 1`, `maximum: 100`
`preprocessing_enabled`	boolean	—
`postprocessing_enabled`	boolean	—
`missing_input_tool_enabled`	boolean	—
`base_model`	string	enum: `gpt-4o`, `gpt-4o-mini`, `mistralai--mistral-large-instruct`, `anthropic--claude-3-sonnet`, etc.; **required**
`backup_base_model`	string	enum: `gpt-4o`, `gpt-4o-mini`, `mistralai--mistral-large-instruct`, `anthropic--claude-3-sonnet`, etc.;
`advanced_model`	string	enum: `gpt-4o`, `gpt-4o-mini`, `mistralai--mistral-large-instruct`, `anthropic--claude-3-sonnet`, etc.; **required**
`backup_advanced_model`	string	enum: `gpt-4o`, `gpt-4o-mini`, `mistralai--mistral-large-instruct`, `anthropic--claude-3-sonnet`, etc.;
`task`	string	`maxLength: 10000`
`additional_context`	string	`maxLength: 25000`
`default_output_format`	string	enum: `text`, `markdown`, `json`; `default: markdown`
`default_output_format_options`	string	`maxLength: 5000`
`toolkits`	array	
— `toolkits[].name`	string	**required**

Property	Type	Validations
— `toolkits[].type`	string	enum: `documentGrounding`, `calculator`, `joule`; **required**
— `toolkits[].config` (if Joule)	object	`target` (object, **required**), `specification` (object, **required**)
— `toolkits[].config.target`	object	`type` (enum: `function`, **required**), `name` (**required**)
— `toolkits[].config.specification`	object	`description` (string, **required**), `parameters` (array, **required**)
— `toolkits[].config.specification.parameters[].name`	string	**required**
— `toolkits[].config.specification.parameters[].type`	string	enum: `string`, `number`, `boolean`, `array`, `object`, `null`; **required**
— `toolkits[].config.specification.parameters[].description`	string	`maxLength: 512`, **required**
— `toolkits[].config.specification.parameters[].optional`	boolean	`default: false`

3.3.3.2 Code-Based Agents (Bring Your Own Agent)

In alignment with the "Bring Your Own Agent" (BYOA) approach, code-based agents refer to AI agents developed using third-party frameworks rather than the proprietary Joule framework.

> ⓘ Note
>
> Code-based agents are supported as of DTA schema version 3.28.0.

Joule prepares an A2A message request for a `"message/send"` method with a user utterance in accordance with the open-standard Agent2Agent (A2A) Protocol, version 0.3.0 (JSON-RPC). For information, refer to the official documentation for the A2A Protocol at `https://a2a-protocol.org/v0.3.0/specification/`.

Action Definition for A2A Requests in Dialog Functions (Synchronous Communication)

Joule constructs the A2A request payload internally using the `"message/send"` method with a user utterance (text message) and calls the agent (only the `"text"` message type is supported).

The handling of the agent's response is delegated to capability development or scripting within dialog functions, similar to standard HTTP request handling. Joule does not perform validation on the response object.

> **Note**
>
> For synchronous communication, Joule expects the response from agent server within 60 seconds.

> **Sample Code**
>
> Action Definition for A2A Requests in Dialog Functions (Synchronous Communication)

```
-actions:
  - type: agent-request
    system_alias: <agent_api> # mandatory for remote agents (BYOA)
    agent_type: remote | local # defaults to local as for BAF/local agents,
file exists in Joule capability
    body: //optional
      {
        Message: <? Message_value ?>  # optional – enables to send custom
message, if not filled – user utterance is sent by default from Joule for
calling agent
      }
    name: invoices/my_invoice_agent # mandatory for local agents
    path: /travel_assistant # optional for remote agents to refer to specific
agent end point or resource
    result_variable: "apiResponse" # A2A Response
```

Action Definition for A2A Requests in Dialog Functions (Asynchronous Communication)

For long-running tasks (for example, for tasks that take minutes) or when clients are unable to or prefer not to maintain persistent connections (such as mobile clients or when serverless functions are used), Joule supports asynchronous updates using push notifications. This allows the A2A server to actively notify a client-provided webhook when a significant task update occurs. In this case, the client is Joule.

You can define the payload accordingly in the dialog functions for calling agents. Joule takes care of configuring the Joule webhook (as the client) to get the server updates.

> **Sample Code**
>
> Action Definition for A2A Requests in Dialog Functions (Asynchronous Communication)

```
-actions:
  - type: agent-request
    system_alias: <agent_api> # mandatory for remote agents (BYOA)
    agent_type: remote | local # defaults to local as for BAF/local agents,
file exists in Joule capability
    body: //optional
      {
        Message: <? Message_value ?>  # optional – enables to send custom
message, if not filled – user utterance is sent by default from Joule for
calling agent
        contextId: <? context_ID ?> # optional - enables propagation of
context ID to agent server for runtime context handling
        taskId: <? task_ID ?> # optional - enables propagation of task ID to
agent server
```

```
        configurations:
        {           # optional – if not specified, synchronous mode will be
considered as default for agent call
         pushNotifications: true/false
        }
      }
    name: invoices/my_invoice_agent # mandatory for local agents
    path: /travel_assistant # optional for remote agents to refer to specific
agent end point or resource
    result_variable: "apiResponse" # A2A Response
```

Using this feature, the server can send task updates to the Joule client, including status notifications and final responses/completed statuses.

> **Note**
>
> To ensure secure inbound communication and validate server updates, an Identity Authentication Service (IAS) App2App trust relationship must be established between the Joule client and the target agent server/application sending updates.
>
> This trust setup can be automated by using the Joule booster for applications interfacing with Joule. For information about setting up a relationship manually, see Consuming APIs from Other Applications.

Multi-Turn Conversations: Context and Task Handling

If an agent requires runtime context handling to support multi-turn conversations/user interactions during the agent execution, the usage of context and task IDs (generated by the agent server) is enabled for agent request calls. You can capture the agent response and store the context ID and task ID into runtime variables (this is also supported in dialog functions as part of scripting) that are returned in the response from the agent server.

In the agent request action call, you can send the additional parameters in the body section of the agent request action call to propagate the context ID and the task ID mapping to the respective runtime variables where the values are captured from the agent response.

```
-actions:
  - type: agent-request
    system_alias: <agent_api> # mandatory for remote agents (BYOA)
    agent_type: remote | local # defaults to local as for BAF/local agents, file
exists in Joule capability
    body: > #optional
      {
        contextId: <? context_ID ?> # optional - enables propagation of context
ID to agent server for runtime context handling
        taskId: <? task_ID ?> # optional - enables propagation of task ID to
agent server
      }
    name: invoices/my_invoice_agent # mandatory for local agents
    path: /travel_assistant # optional for remote agents to refer to specific
agent end point or resource
    result_variable: "apiResponse" # A2A Response
```

> **Note**
>
> For extensibility developers, there is no need to explicitly handle the empty value checks for the initial agent call where the context ID and task ID are empty. This is handled internally by Joule.

4 Compile

4.1 Compile the Design-Time Artifact

Compile the Design-Time Artifact (DTA) to generate a Runtime Artifact (RTA) as a `.daar` file.

Prerequisites

1. You have installed the Joule Studio Command Line Interface as described in Install and Update the Joule Studio CLI [page 4].
2. You have logged into the Joule Studio CLI - for more, see joule login) .

Compiling DTA into RTA

1. Navigate to the root of your capability folder (folder with `capability.sapdas.yaml`).
2. Run the following command in the terminal to compile the DTA:
   ```
   joule compile
   ```

5 Deploy

5.1 Deploy the Runtime Artifact

A runtime artifact (RTA) is generated after you compile the design-time artifact (DTA). You can deploy the RTA into your Joule tenant to create an assistant or add a new version of the assistant to an existing one.

Prerequisites

- You have the `extensibility_developer` role assigned in SAP BTP to deploy the digital assistant into your Joule tenant. For more information, see Assign Roles.
- You have installed the Joule Studio CLI as described in Install and Update the Joule Studio CLI [page 4].
- You have logged in to Joule Studio CLI.

Deploy an RTA

1. Navigate to the root folder in which your RTA is created (with extension .daar).
2. Run the deploy command to deploy the assistant, for more information, for more, see Joule Studio CLI deploy command.
   For more information, see Joule Studio CLI Commands [page 6].

After the assistant is successfully deployed in your tenant, you can test the assistant by starting a conversation. You can use the standalone Web Client to chat with your assistant and test the conversation flow. For more information, see Using the Joule Web Client.

Constraints

- You must provide the name of your RTA file (including complete path), however, the assistant name is optional. If the assistant name is not provided, the assistant name from the RTA file will be considered.
- The assistant name is case-insensitive. It can have 3 to 50 characters, and can only contain alphanumeric characters and underscores (no consecutive underscores).
- Your RTA file size does not exceed 50 MB.
- The deployment process is timed-out after 60 minutes. You must re-trigger the deployment if the deployment takes longer.
- A new assistant will be created, if an assistant with provided name does not exist in the tenant.
- You can deploy up to 30 assistant artifacts in a tenant.

- If an assistant with the same name already exists in the tenant:
  - The deployed RTA version will be added as new version for the existing artifact. (Use the command `joule get <digital assistant name>` to check the details of the latest version of the assistant)
  - All new conversations will be initiated with the latest version of the Joule however, the conversations that are already in progress with the earlier version will continue uninterrupted till they are completed or expired.
  - The earlier version of the Joule will be deleted once all active conversations are completed or expired.
- Parallel deployments are not allowed for the same assistant artifact.
- Capabilities sent for deployment must have artifacts belonging to the same major schema version. Capabilities with the same major artifact version with different minor versions such as 3.4.0 and 3.10.0 are supported in the same Joule assistant for deployment. For more information, see Design Time Artifact (DTA) Specification [page 35].

> **Note**
>
> Extensibility developers cannot deploy assistants named `sap_digital_assistant`.

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.
About the icons:

- Links with the icon : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
  - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
  - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.
The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

www.sap.com/contactsap

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see https://www.sap.com/about/legal/trademark.html for additional trademark information and notices.